

ARIN 5201 & MAIE 5421

Machine Learning for Computer Vision

Wenhan LUO

9/18/2026

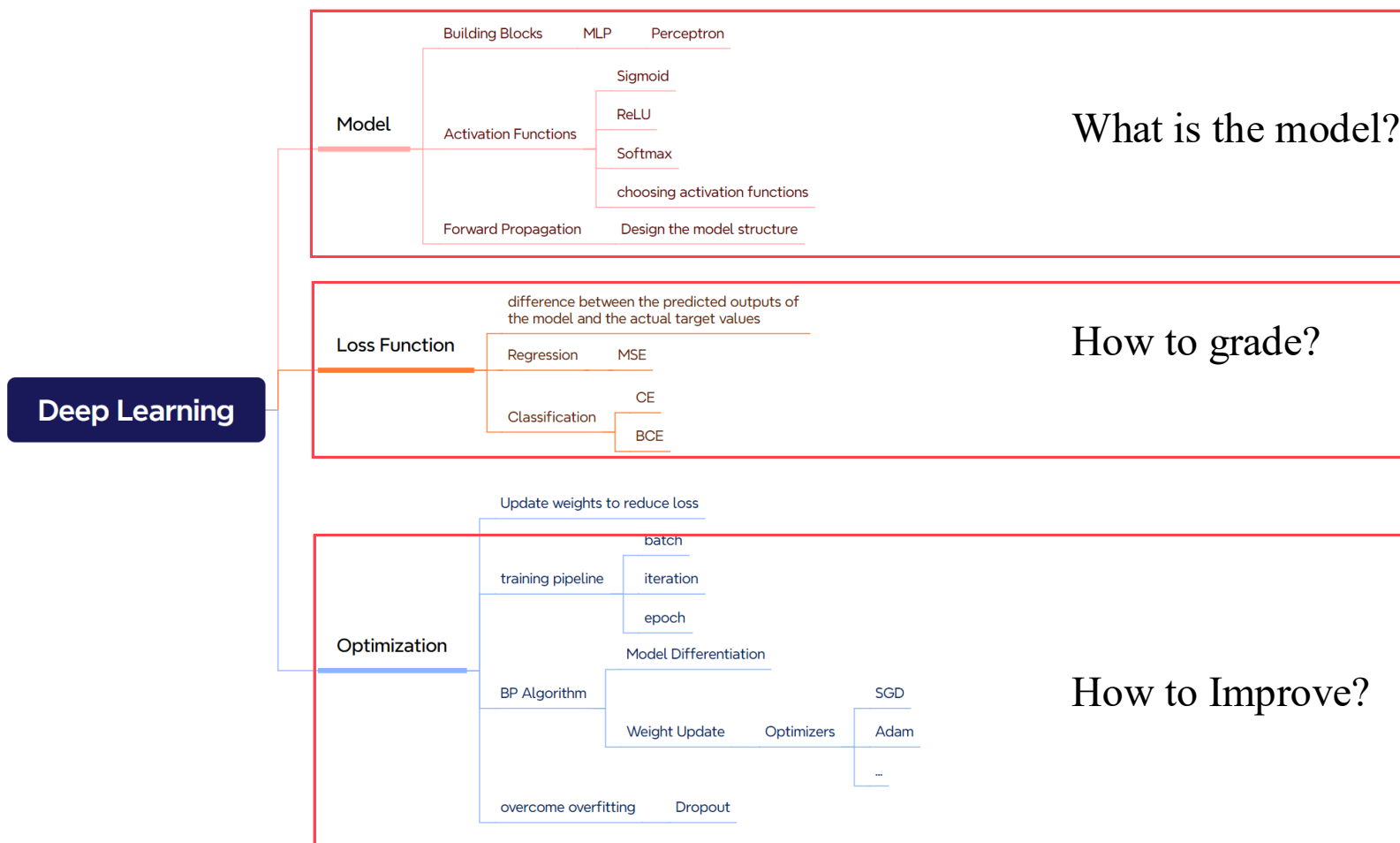
Content

Topic	Week
Machine Learning and Computer Vision Basics	
Introduction to Computer Vision	1
Machine Learning Elements	2
Deep Learning	3-4
 Advanced Topics in Computer Vision	
Image Classification with CNNs	5-6
Detection & Segmentation	7
Attention Mechanism & Transformer	8
Generative Models	9-10
Multi-modal Learning	11
Final project representation	12-13

Deep Learning

Overview

Think about Human Learning

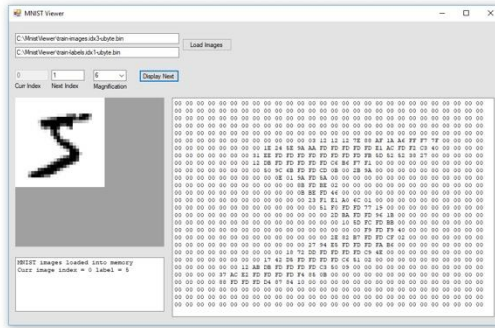
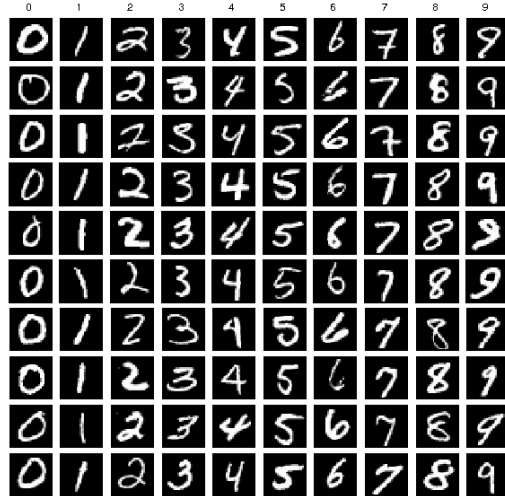


Deep Learning Outline

- A Journey of Deep Learning via MLP (A Simple Model)
 - **Multi-layer Perceptron (MLP)**
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks (Other Models)
 - Resnet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

Multi-layer Perceptron (MLP)

Let us define a task (the “destination”) first to start a MLP journey

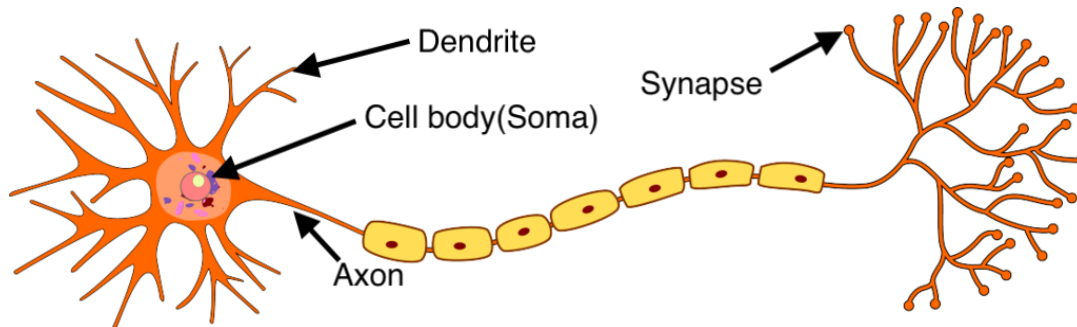


- Supposing that we want to classify the digit images from the MNIST dataset, where
 - the input is $28 \times 28 = 784$ -dim pixel with values in $[0, 255]$
 - the output is the categorical class label in $[0, 9]$
- Now we will design a MLP to take the 784-dim feature X as input, and predict the posterior probability $P(C_i|X)$ for each class.

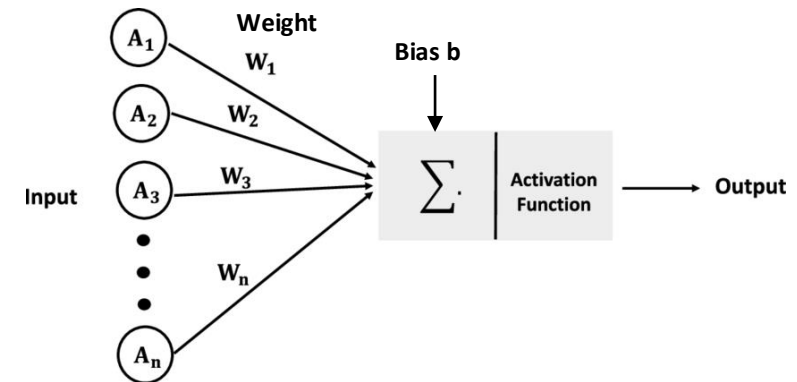
Multi-layer Perceptron (MLP)

Human Brain Analogy

- The perceptron is a mathematical model inspired by the information processing of a biological neuron.
- Just as **synapses** strengthen or weaken to learn in the human brain, perceptron adjusts its **weights**.



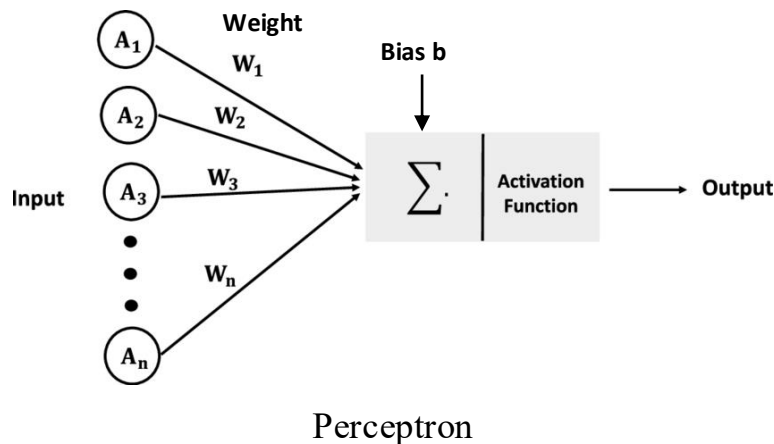
Biological neurons in the human brain



Analogy to Artificial Neuron, Perceptron

Multi-layer Perceptron (MLP)

Key Components of a Perceptron

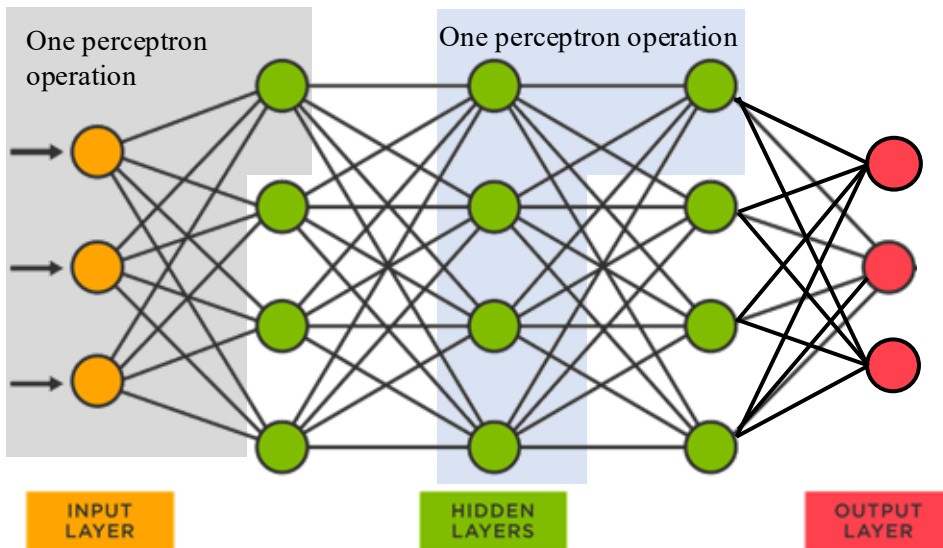


$$y = \phi(\mathbf{w} \cdot \mathbf{A} + b)$$

- **Input:** Multi-dimensional information, where A_n represents the n -th attribute. Can be either features, or from other perceptron outputs.
- **Weight:** The core of perceptron, which largely determines the flow of information and performance. Should be learned from data.
- **Bias:** Can be seen as a part of weight irrelevant to input.
- **Activation Function:** Imposing nonlinearity, making it to model complicated relations.
- **Output:** The aggregated information from the input.

Multi-layer Perceptron (MLP)

Building MLP from Perceptron



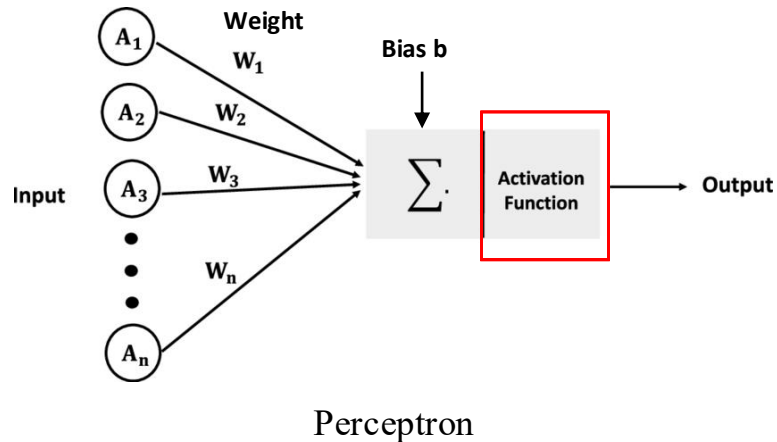
- A Multilayer Perceptron (MLP) is constructed by **stacking multiple perceptrons**, typically arranged in **layers**, where each perceptron in one layer is connected to all perceptrons in the next layer.
 - Input Layer
 - Hidden Layers
 - The neurons in the hidden layers are generated by different weights given the previous layer.
 - Output Layer
- MLP is also usually called as **“fully connected layer”** or **“fully connected network”** when used as a building block.

Deep Learning Outline

- A Journey of Deep Learning via MLP
 - Multi-layer Perceptron (MLP)
 - **Activation Functions**
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks
 - Resnet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

Activation Functions

- A mathematical non-linear operation applied to a perceptron's input to determine its output.



- Critical for neural network's ability to process and learn from complex data.
- Can help to decide whether a neuron should be **activated or not**.

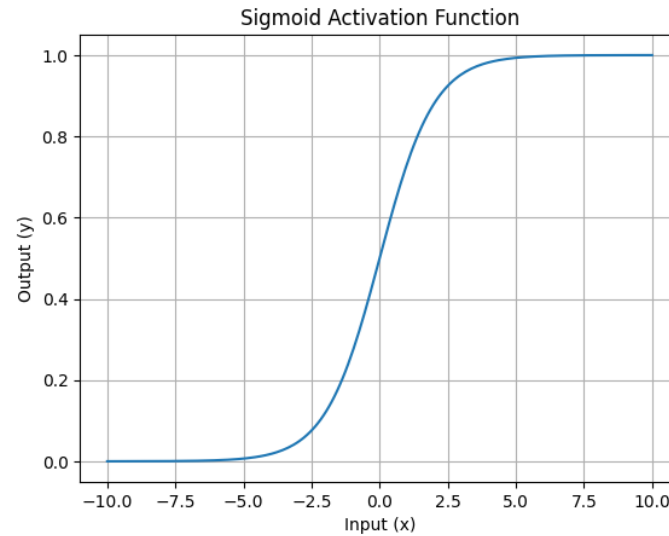
$$y = \phi(\mathbf{w} \cdot \mathbf{A} + b)$$

Activation Functions

Popular Activation Functions

Sigmoid Activation

$$\phi(x) = \frac{1}{1 + e^{-x}}$$



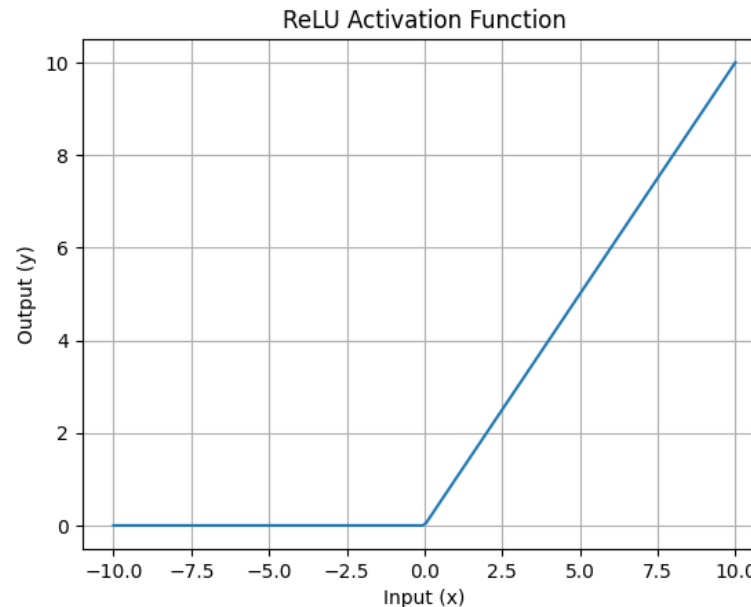
- Useful for
 - Binary classification (recall logistic regression)
 - Regression when the output is normalized to [0,1]

Activation Functions

Popular Activation Functions

ReLU Activation

$$\phi(x) = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise.} \end{cases}$$



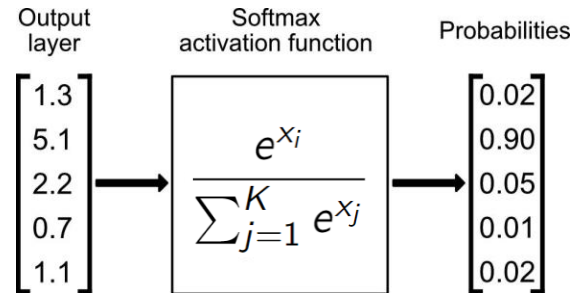
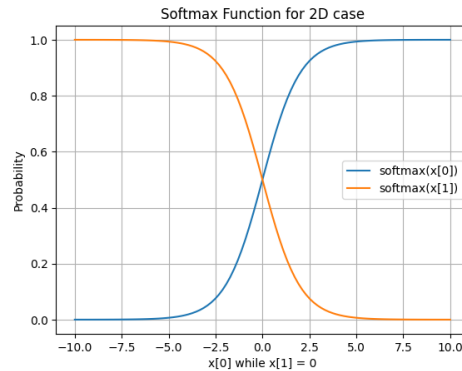
- Widely used in deep learning models because of computational efficiency $\max(x, 0)$.
- Another advantage is that the gradient over x is simple.

Activation Functions

Popular Activation Functions

Softmax Activation

$$\sigma(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

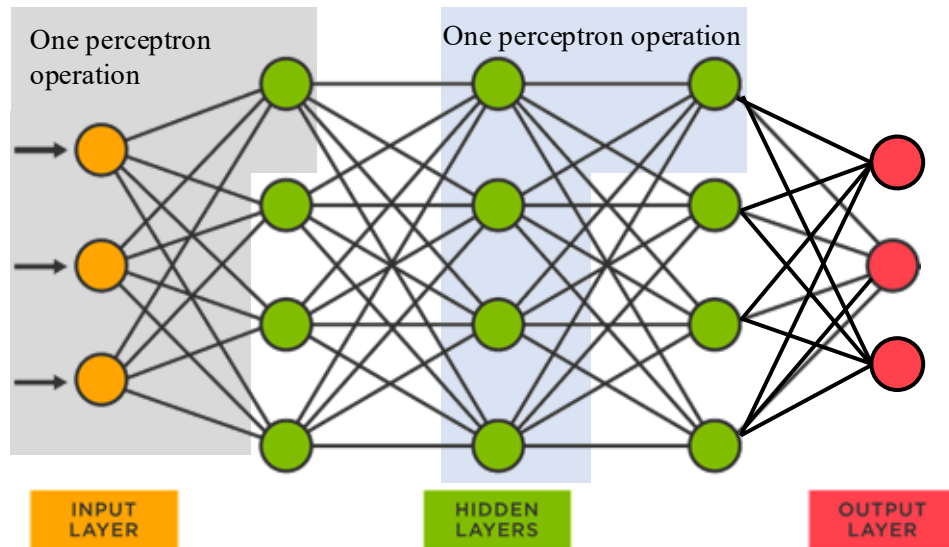


- The output is a vector where each element is in $[0, 1]$, and the sum of all elements is 1.
- Usually used in the output layer**, and provides a probability for each class **where classes are exclusive**.
 - Output layer of multi-class classification

Design An MLP for Digit Classification

Now let us think about our digit classification task

Input?



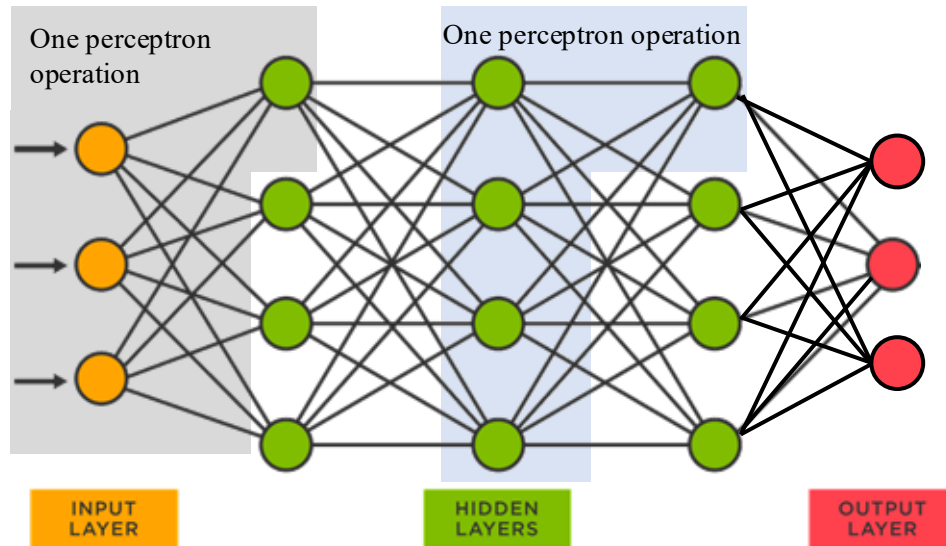
768-Dimension vector of image pixel values.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
25	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
26	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Design An MLP for Digit Classification

Now let us think about our digit classification task

Classifier?



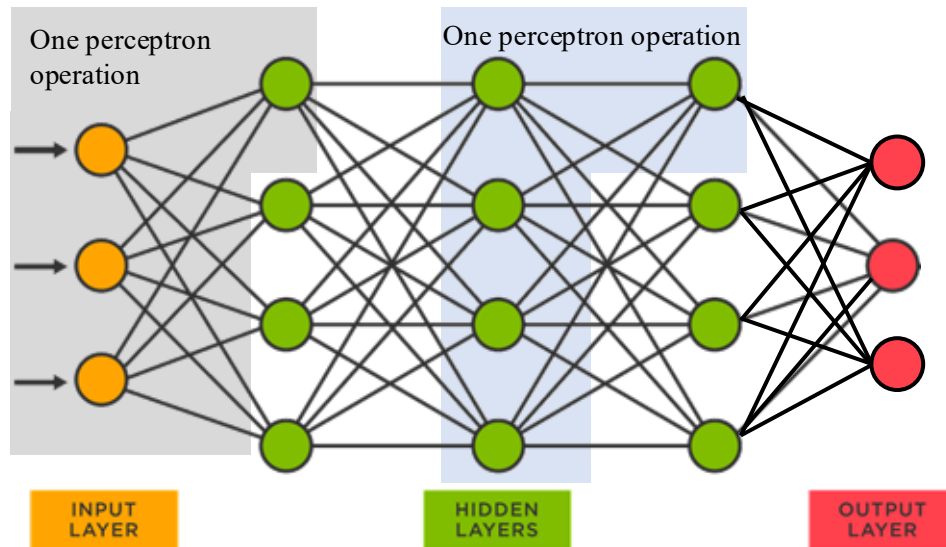
- MLP with 768-D input and 10-D output.
- Each output node indicates the probability on one class.

1	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
2	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
25	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
26	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Design An MLP for Digit Classification

Now let us think about our digit classification task

Output?



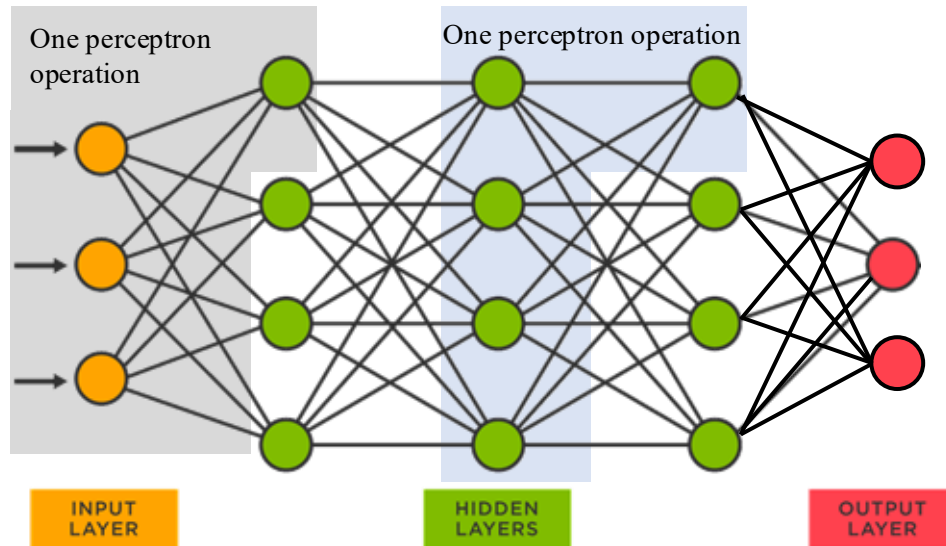
- 10-D output, each output node indicates the probability on one class.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
25	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

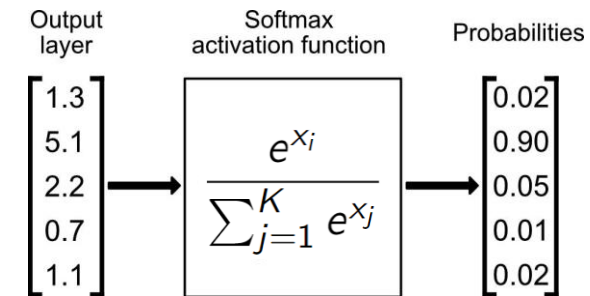
Design An MLP for Digit Classification

Now let us think about our digit classification task

Activation?



- We can use sigmoid activation function for input hidden layers
- Softmax will be used in the output layer

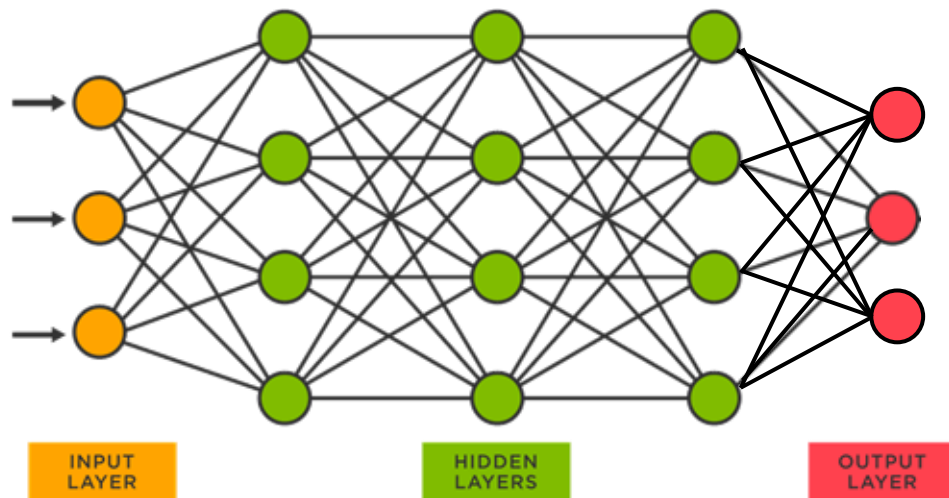


Deep Learning Outline

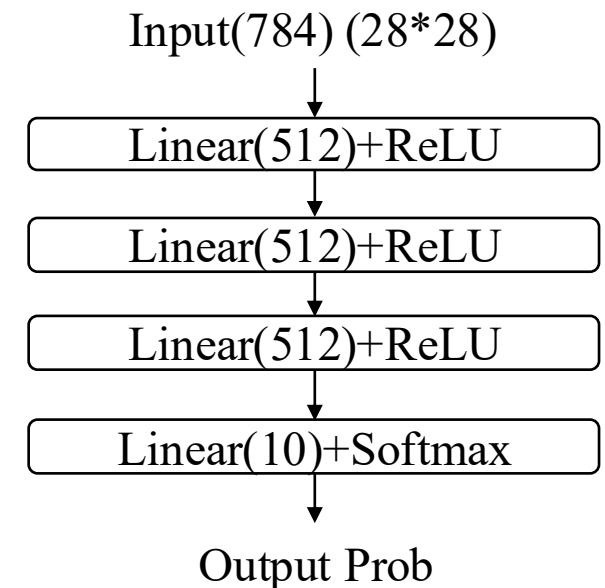
- A Journey of Deep Learning via MLP
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - **Forward Propagation**
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks
 - Resnet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

Forward Propagation

- Forward propagation computes the network's prediction for a given input, which defines the **overall information flow**.
- When many layers of neural networks are used, we call it “**Deep** neural network”.



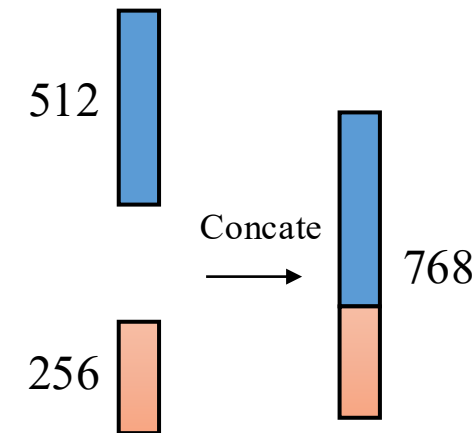
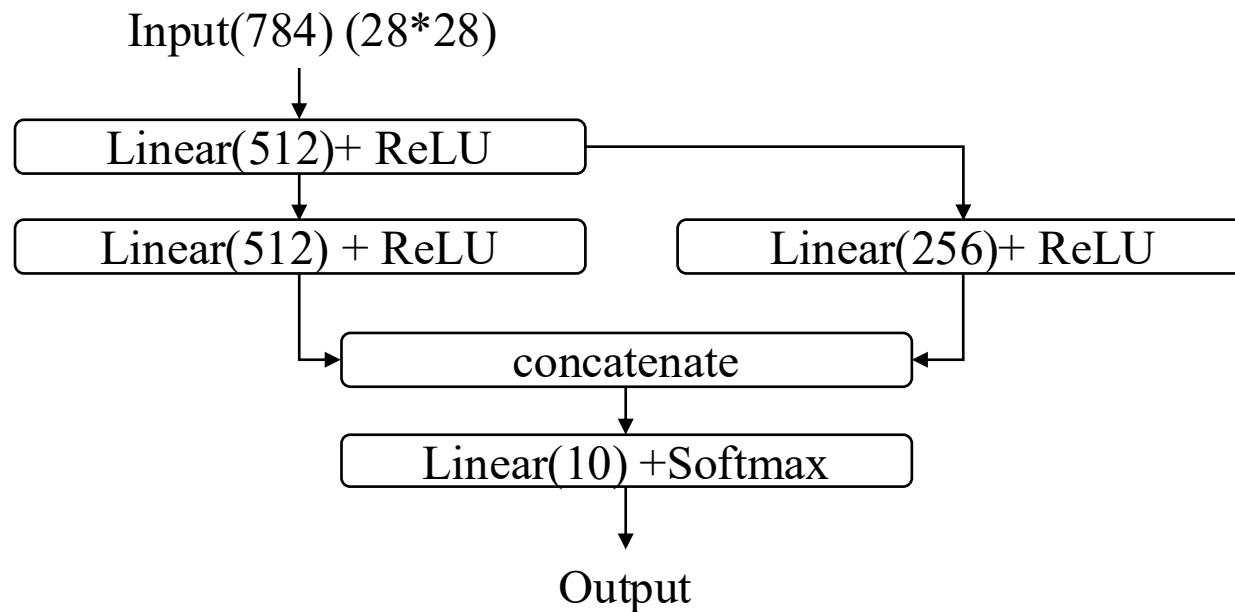
Another expression



- We assume we use 512 nodes with ReLU activation for each hidden layer.
- When talking about how many nodes are used, it also indicates the dimension of output for this layer.

Forward Propagation

In many cases, the model structure is more complicated than simple cascade forward connections.



In this case, we might want to first use two branches to capture different information of the input, and then combine them to produce the final output.

Forward Propagation

In many cases, the model structure is more complicated than simple cascade forward connections.

Designing the forward propagation is one of the most important parts of DL research.

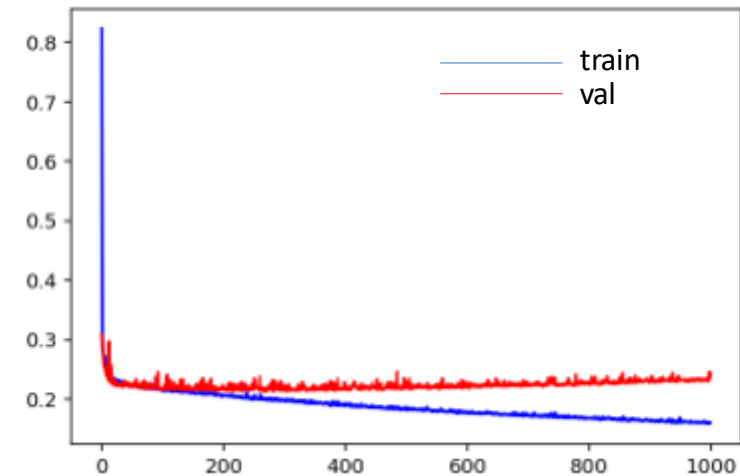
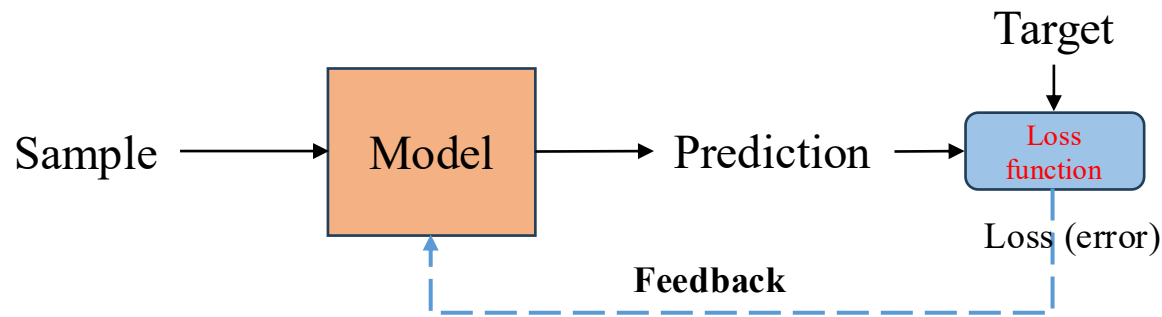
- More commonly referred as “model structure design”
- A model can consist of many sub-modules, and each module is composed as basic network building blocks and operations
 - MLP
 - CNN
 - RNN
 - Transformer
 - Concatenation
 - Norm
 - ...

Deep Learning Outline

- A Journey of Deep Learning via MLP
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - **Loss Functions**
 - Optimization with Backpropagation
- Advanced Deep Neural Networks
 - Resnet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

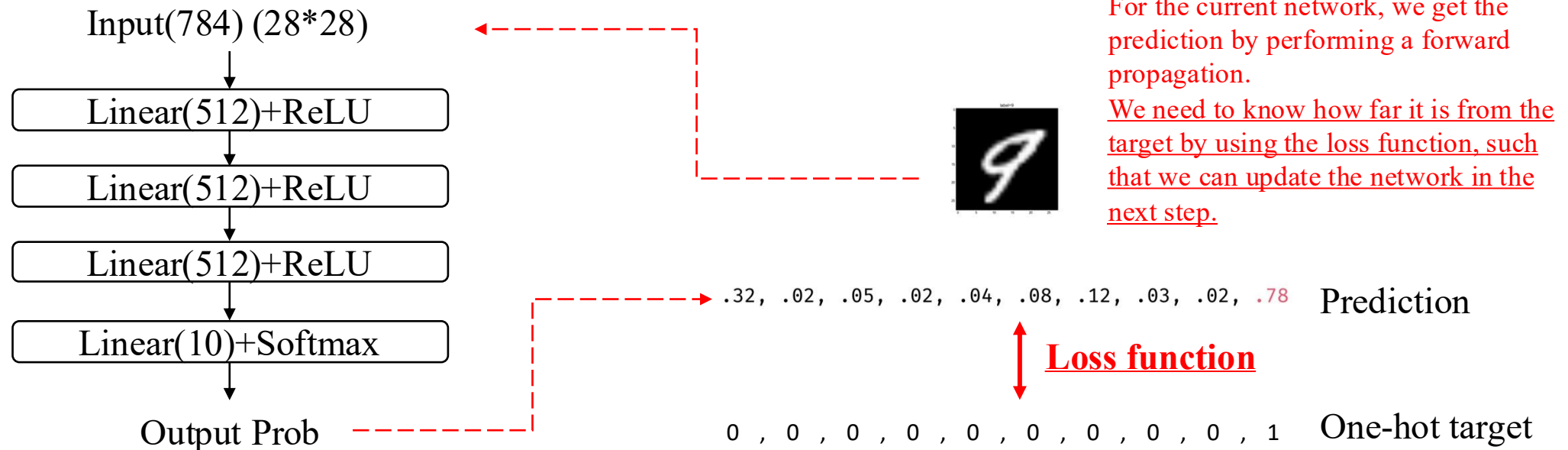
Loss Functions

- Loss function quantifies the difference between the predicted outputs of the model and the actual target values.
- A lower value indicates better performance.
- The loss function guides the updating of the model weights.



Loss Functions

- Loss function quantifies the difference between the predicted outputs of the model and the actual target values.
- A lower value indicates better performance.
- The loss function guides the updating direction of the model weights.



Loss Functions

Regression

Mean Squared Error (MSE)

- Calculates the average squared difference between predicted values and actual target values.
- Similar to SSR in traditional regression methods.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

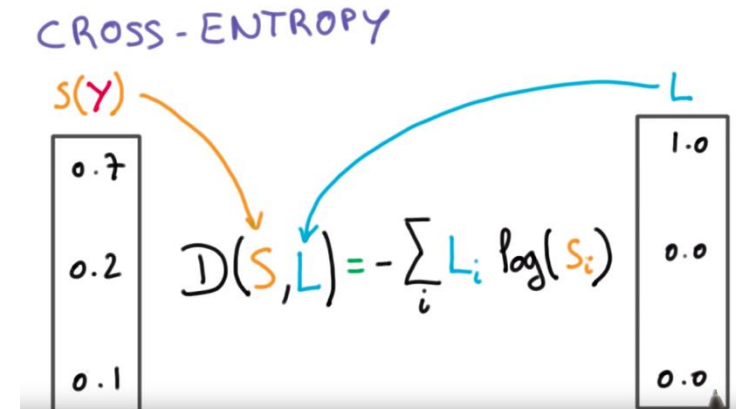
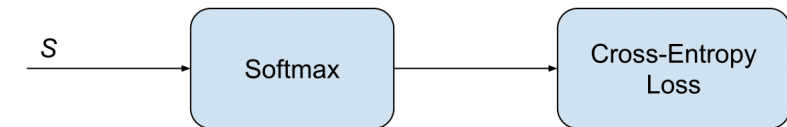
Loss Functions

Multi-Class Classification

Cross-Entropy (CE)

- A measure of the “surprise” from incorrect predictions, weighted by how often they occur.

$$E_p[\text{CE}_{\text{multi-class}}] = - \sum_{c=1}^M p(c) \log(\hat{p}(c))$$



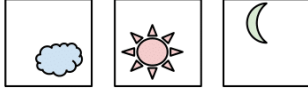
- Supposing the ground truth class for the input sample is c , the model **should give** high $\hat{p}(c)$.
- If the $\hat{p}(c)$ from model is small (incorrect prediction), the “surprise”, or loss function value $-\log(\hat{p}(c))$, is high.

Loss Functions

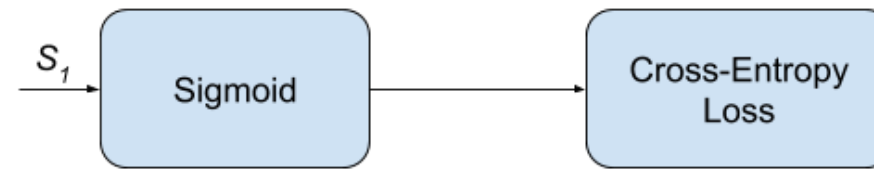
Multi-Label Classification

Binary Cross-Entropy (BCE)

- Use multi-hot vector to indicate the existence of multiple objects.
- Use Sigmoid activation rather than Softmax activation to normalize the probability for each class to $[0,1]$

	Multi-Class	Multi-Label
C = 3	Samples  Labels (t) [0 0 1] [1 0 0] [0 1 0]	Samples  Labels (t) [1 0 1] [0 1 0] [1 1 1]

$$CE_{\text{binary}}(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$



Loss Functions

Factors in loss functions

Factor	Description
Type of Learning Problem	Classification vs Regression; Binary vs Multiclass Classification.
Model Sensitivity to Outliers	Some loss functions are more sensitive to outliers (e.g., MSE), while others are more robust (e.g., MAE).
Desired Model Behavior	Influences how the model behaves, e.g., hinge loss in SVMs focuses on maximizing the margin.
Computational Efficiency	Some loss functions are more computationally intensive, impacting the choice based on available resources.
Convergence Properties	The smoothness and convexity of a loss function can affect the ease and speed of training.
The scale of the Task	For large-scale tasks, a loss function that scales well and can be efficiently optimized is crucial.

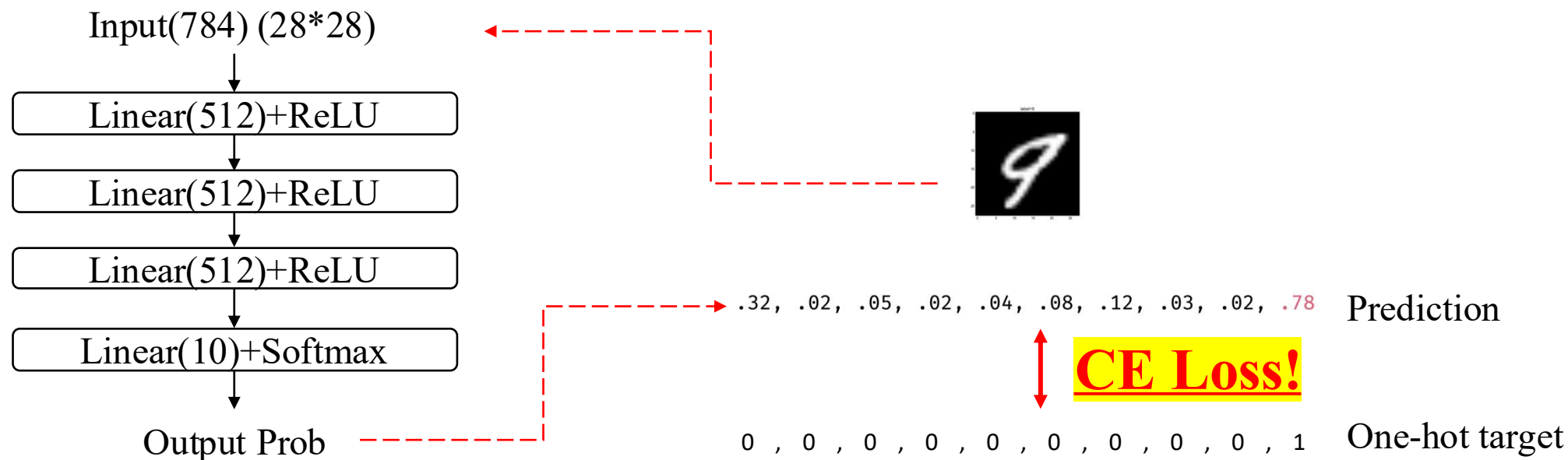
Loss Functions

**Choose suitable loss function
for your task**

Loss Function	Applicability to Classification	Applicability to Regression	Sensitivity to Outliers
Mean Squared Error (MSE)	✗	✓	High
Mean Absolute Error (MAE)	✗	✓	Low
Cross-Entropy	✓	✗	Medium
Hinge Loss	✓	✗	Low
Huber Loss	✗	✓	Medium
Log Loss	✓	✗	Medium

Loss Functions

- Now we can determine the loss function chosen for digits classification.

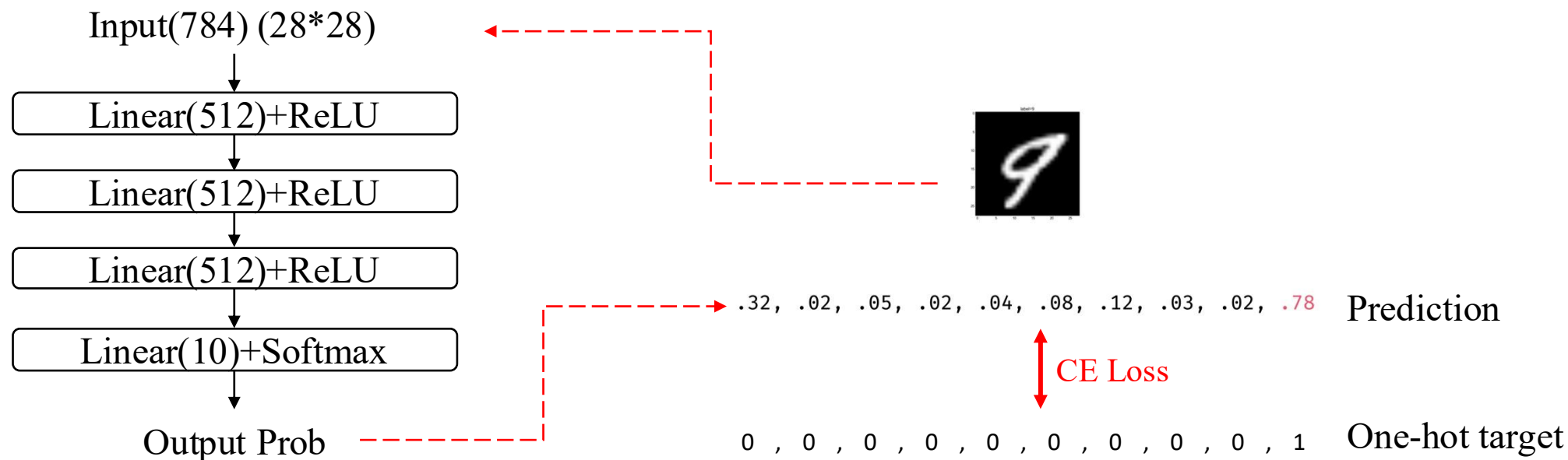


Deep Learning Outline

- A Journey of Deep Learning via MLP
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks
 - Resnet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

Model Training

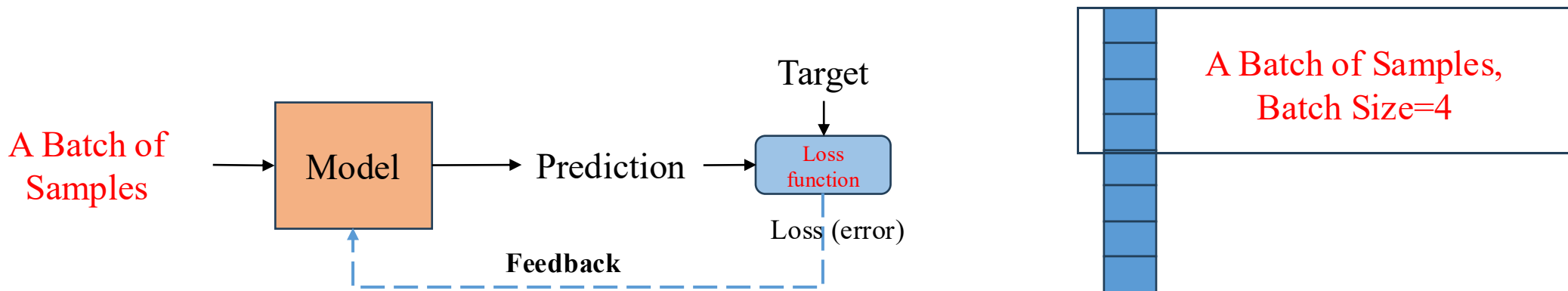
- Given the training data, defined deep neural network (DNN) and loss function,
 - How can we optimize the model weights to reduce the loss function?



Model Training

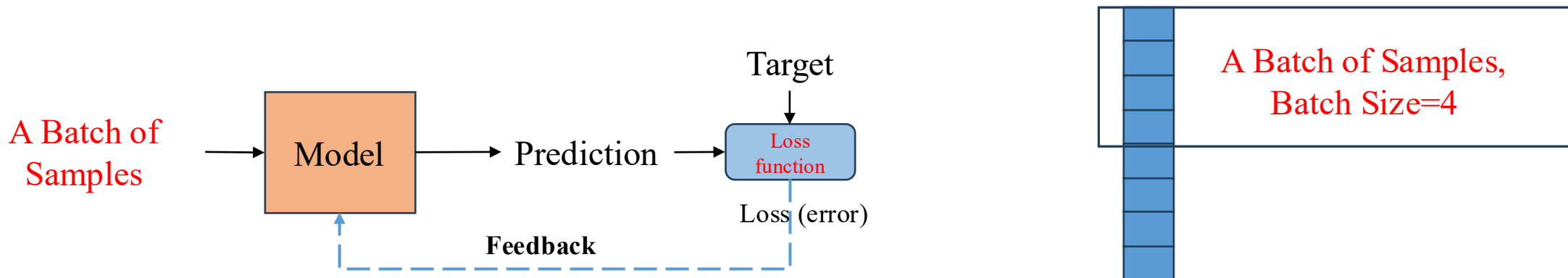
- **Batch**

- The training set is always huge, must be fed to the network in **batches**.
- Batch-wise training uses subsets of the training data.
- It strikes a balance between computational efficiency and the granularity of updates.



Model Training

- **Iteration**
 - Using one batch of data for model updating is called **one iteration**.

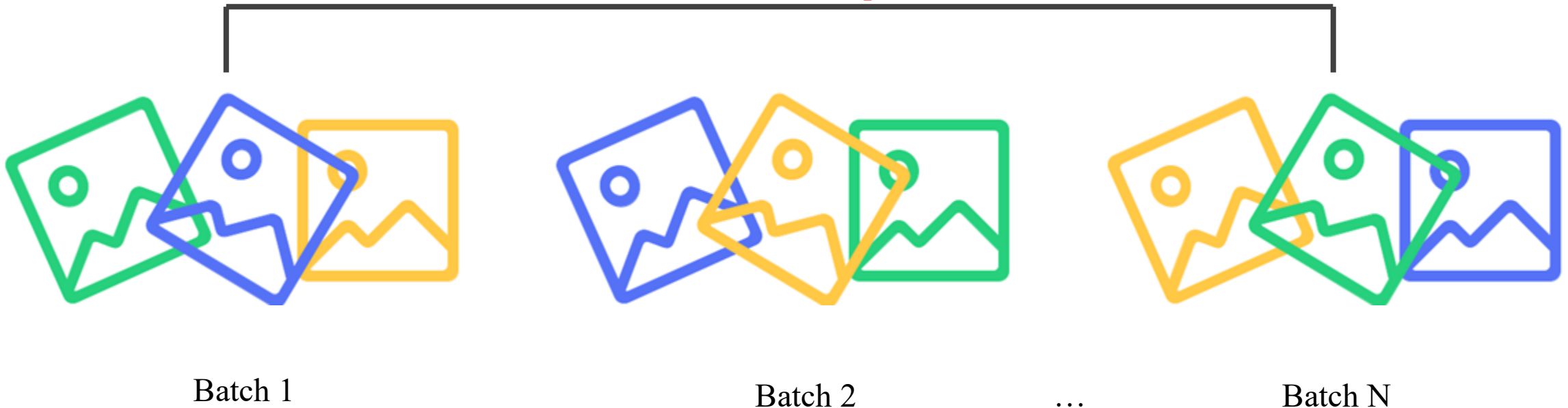


Model Training

- **Epoch**

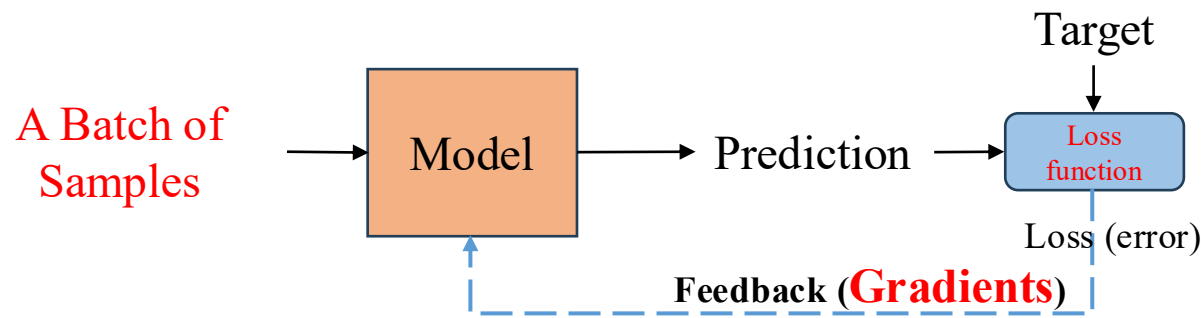
- The whole training set is divided into several batches
- One pass of all batches for model training is called **one epoch**.
- The model is generally trained for many epochs until convergence.

One Epoch



Backpropagation

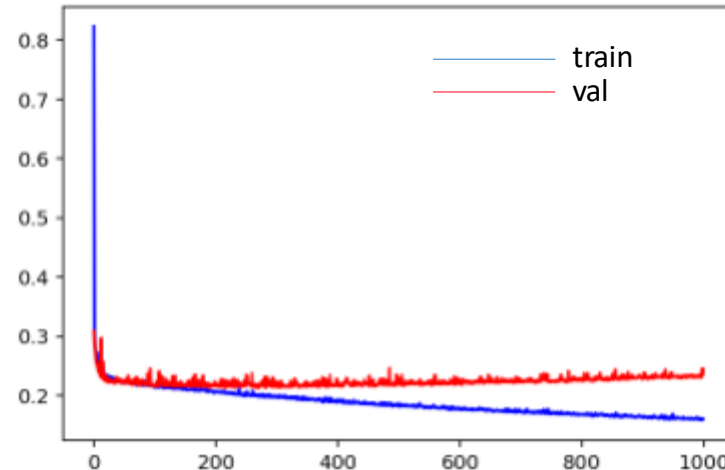
- Adjusting the weights based on the loss using the previous model weights.
 - **Forward Pass:** Input data is fed to the network to obtain the output.
 - **Loss Computation:** The error is calculated using the loss function.
 - **Model Differentiation:** Calculate gradients of the loss with respect to each weight.
 - **Weight Update:** Apply the gradients to update the weights using an optimization algorithm (e.g., SGD, Adam).



It is the gradients deduced from loss that help to update the weights

Backpropagation

- Adjusting the weights based on the loss using the previous model weights.
 - **Forward Pass:** Input data is fed to the network to obtain the output.
 - **Loss Computation:** The error is calculated using the loss function.
 - **Model Differentiation:** Calculate gradients of the loss with respect to each weight.
 - **Weight Update:** Apply the gradients to update the weights using an optimization algorithm (e.g., SGD, Adam).



Normally the training and validation loss will decrease over iterations until convergence.

Backpropagation

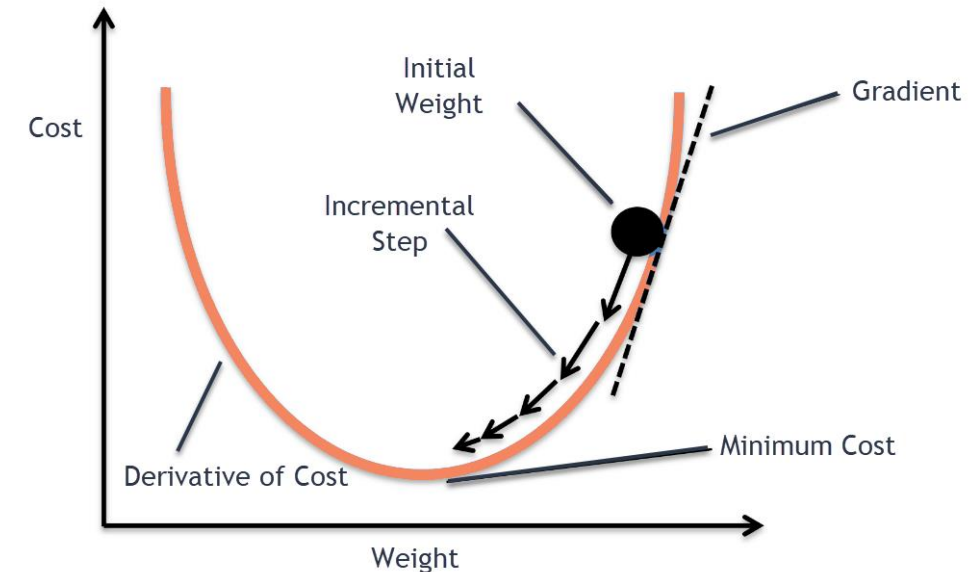
Optimizer

Stochastic Gradient Descent (SGD)

- **Stochastic:** Estimates the gradient based on a single sample or a small batch.

$$w = w - \eta \frac{\partial L}{\partial w}$$

- η is the learning rate for modeling training.



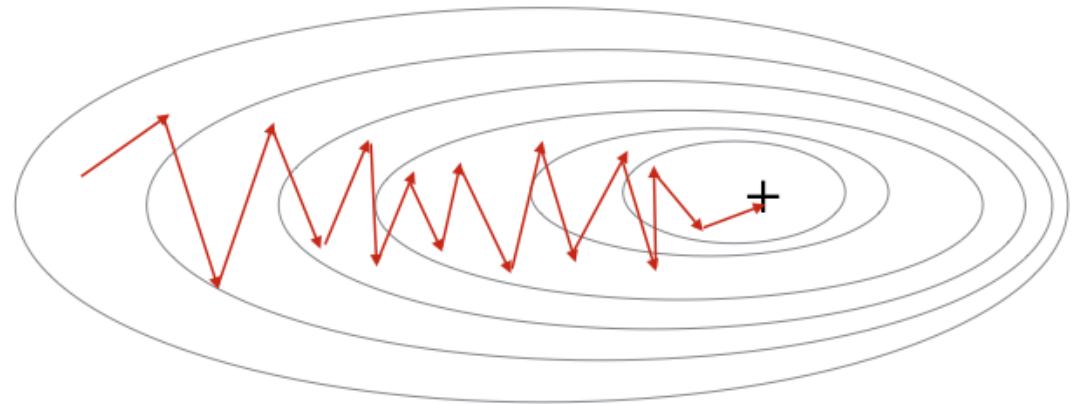
Backpropagation

Optimizer

Stochastic Gradient Descent (SGD)

Optimization Path

$$w = w - \eta \frac{\partial L}{\partial w}$$



- Since SGD updates parameters using only a small subset of data, the estimated gradient may not represent the true gradient across the entire dataset.
- As a result, the optimization trajectory is not a direct path towards the minimum of the loss function

Backpropagation

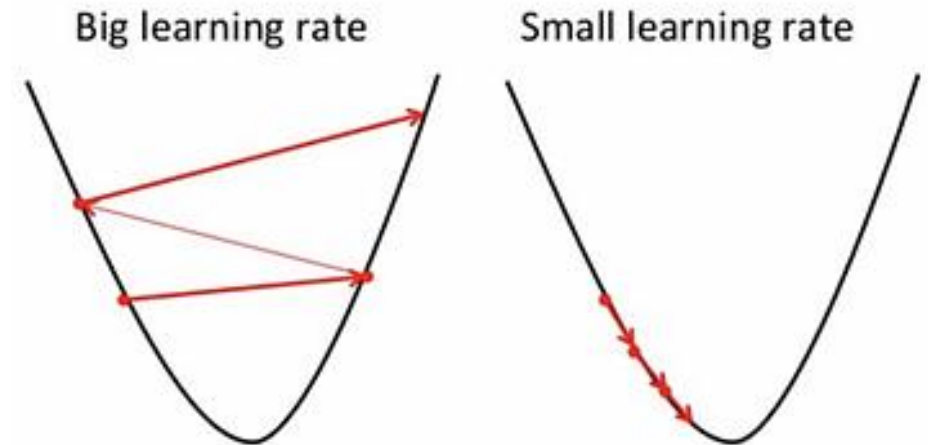
Optimizer

Stochastic Gradient Descent (SGD)

Learning Rate

$$w = w - \eta \frac{\partial L}{\partial w}$$

η is the learning rate (LR) for modeling training.
Large LR may lead to divergence, and small LR may make the optimization very slow.



Backpropagation

Optimizer

Stochastic Gradient Descent (SGD)

Problems

$$w = w - \eta \frac{\partial L}{\partial w}$$

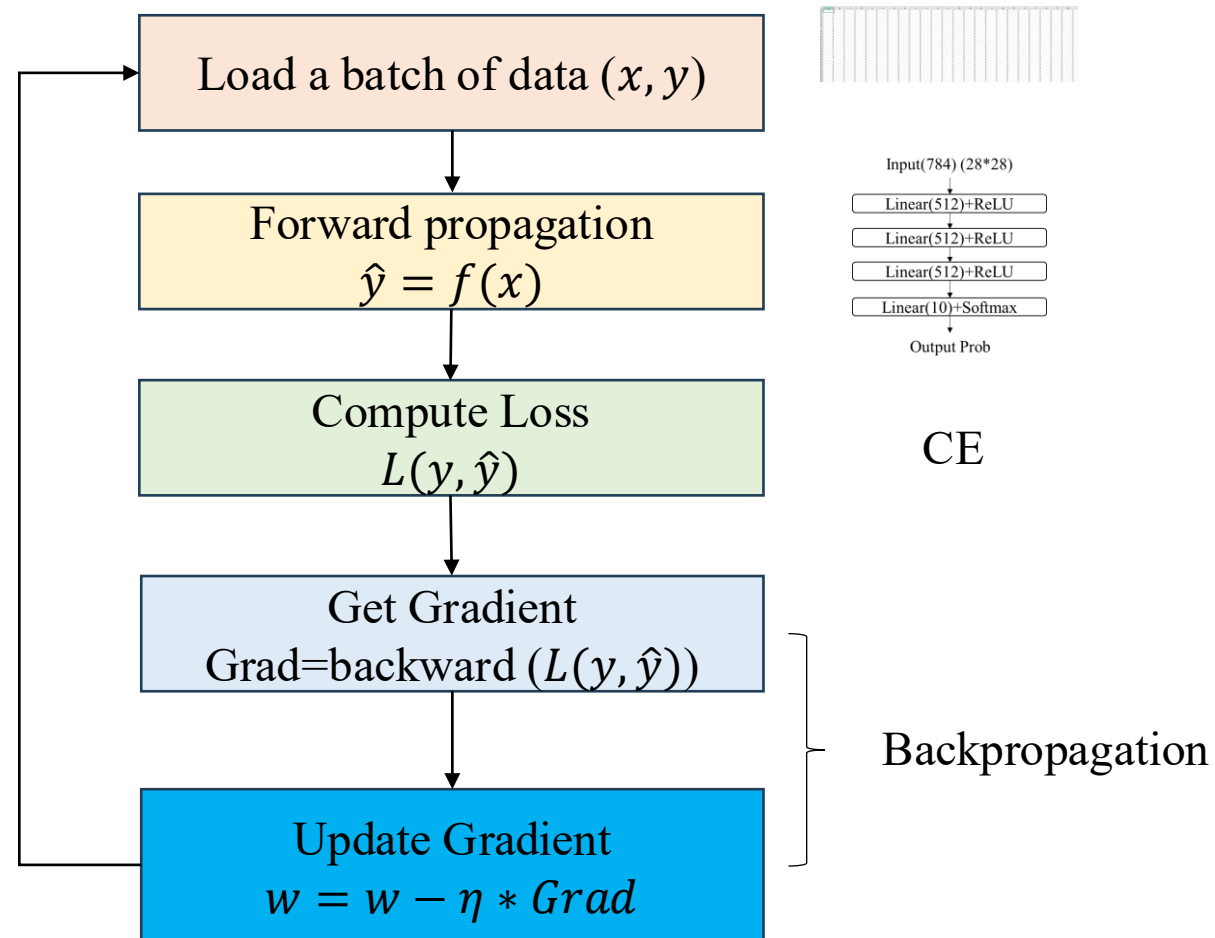
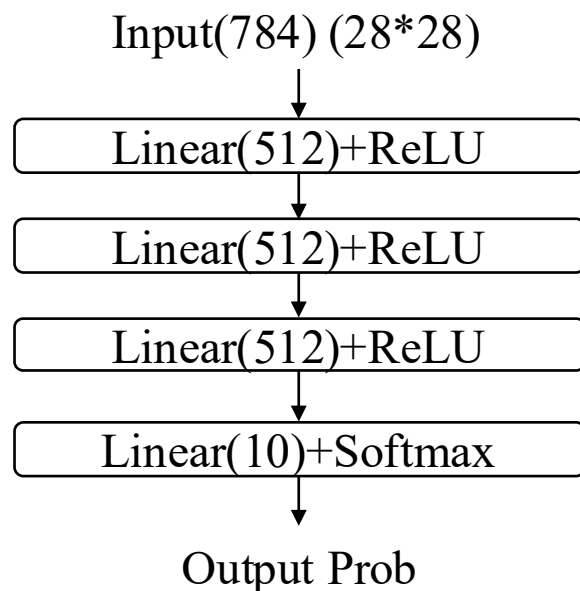
- Slow convergence speed.
- Need to pre-define the learning rate.

Improved Methods for choosing LR

- Adam
 - Adagrad
 - RMSprop
 - ...
-
- Adam has become the most popular optimizer
 - All these optimizers are implemented in DL frameworks such as PyTorch.

Backpropagation

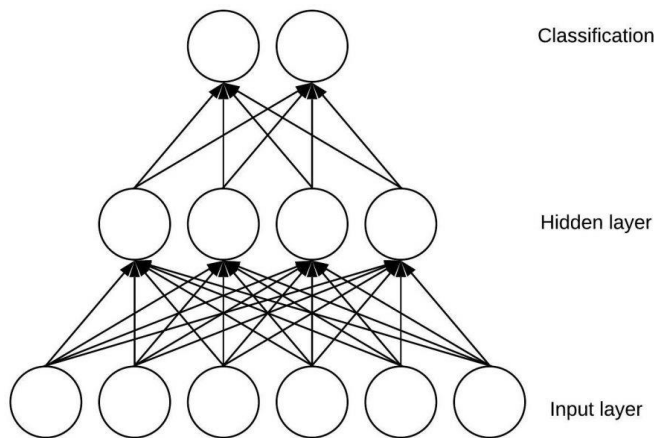
Training a digit image classifier



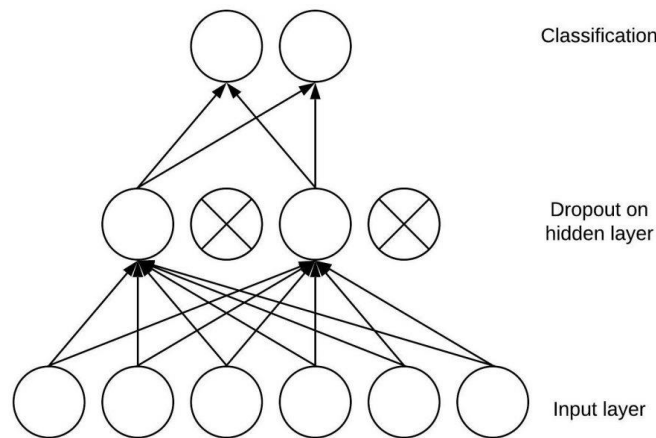
Coping with Overfitting When Training

Dropout

- The model can be too large as compared to the data size.
- One important DL technique to cope with overfitting is dropout.
- Controlled by “dropout rate” when defining layers.



Without Dropout

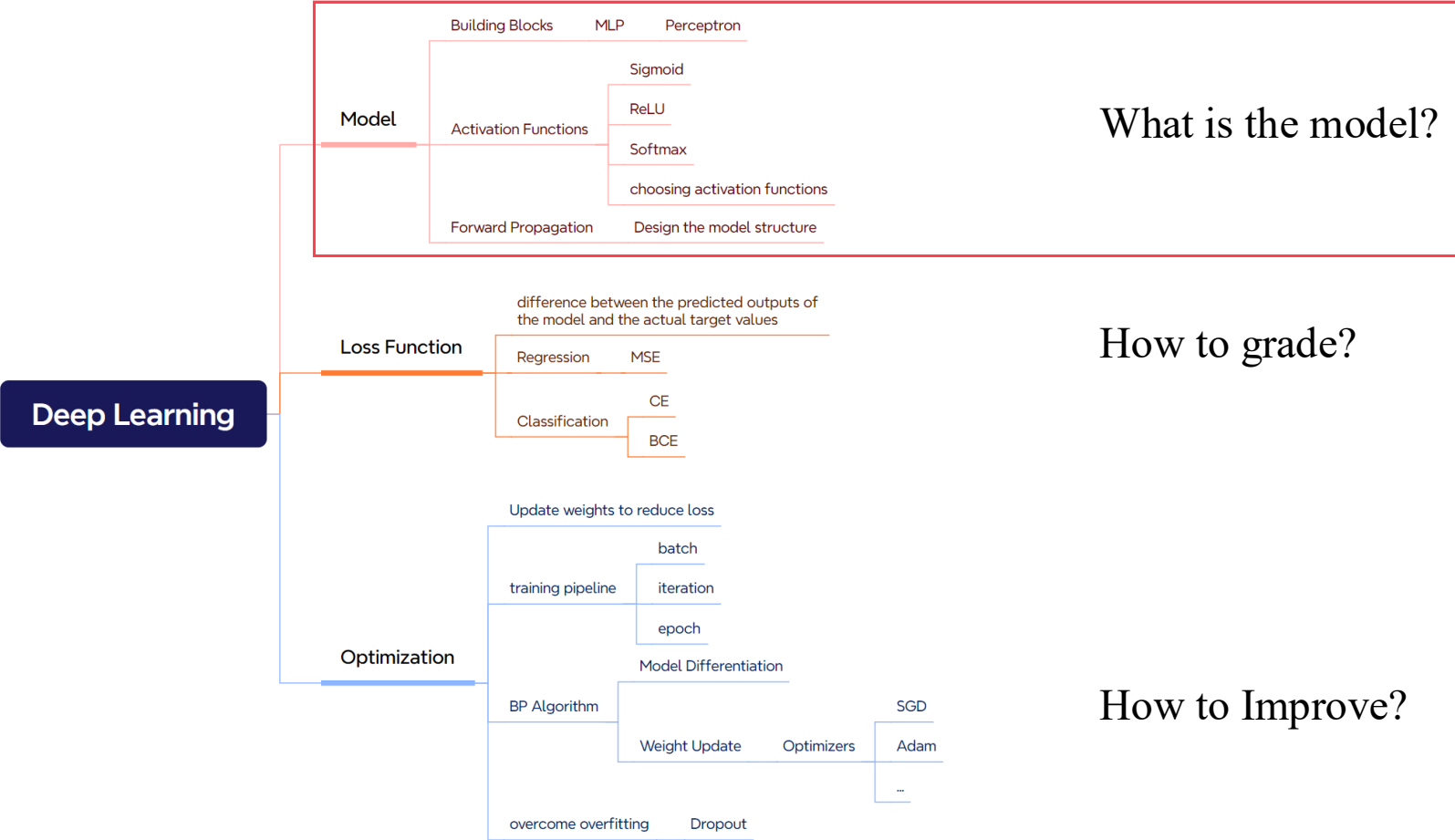


With Dropout

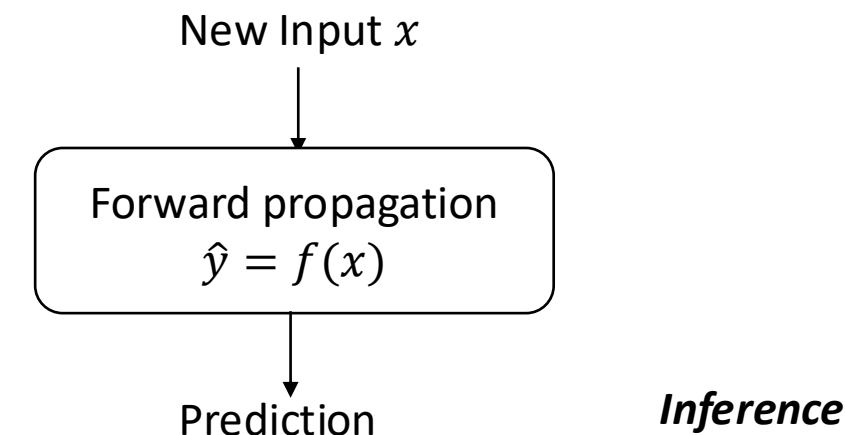
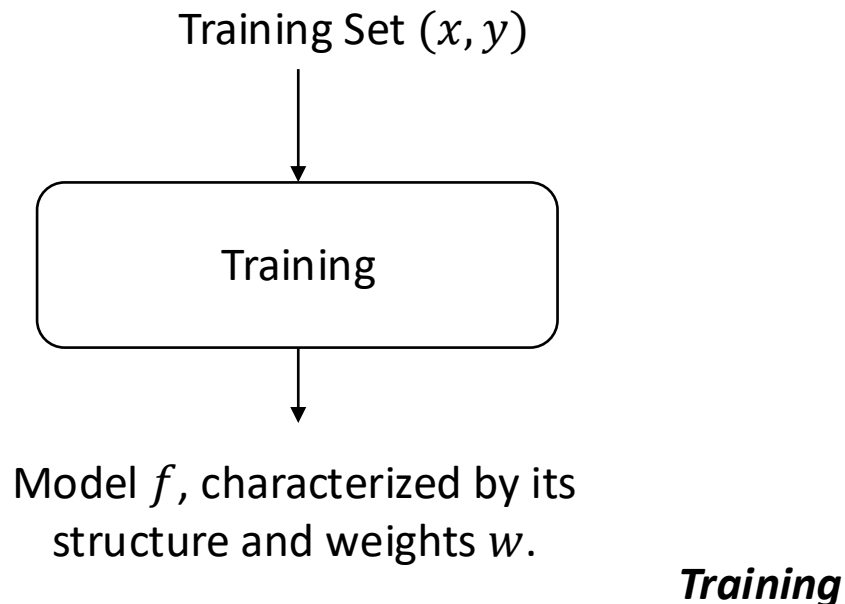
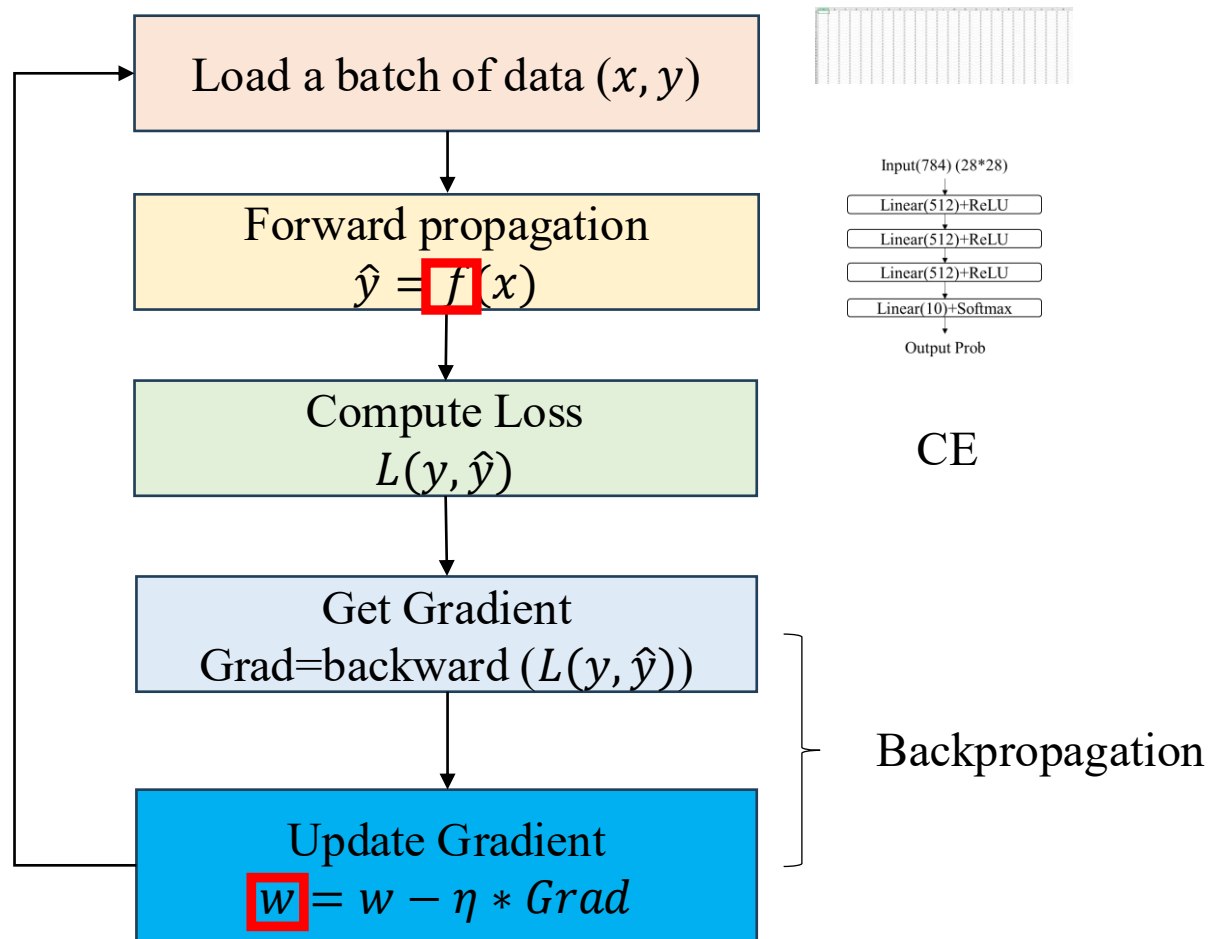
- Randomly dropping nodes (along with their connections) from the neural network during training
- Can be seen as a large ensemble of networks with shared weights

Overview

Think about Human Learning



Recap of Model Training



Deep Learning Outline

- A Journey of Deep Learning via MLP (A Simple Model)
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks (Other Models)
 - ResNet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

Deep Residual Net (ResNet)

ImageNet was once considered as one of the most challenging tasks for ML.



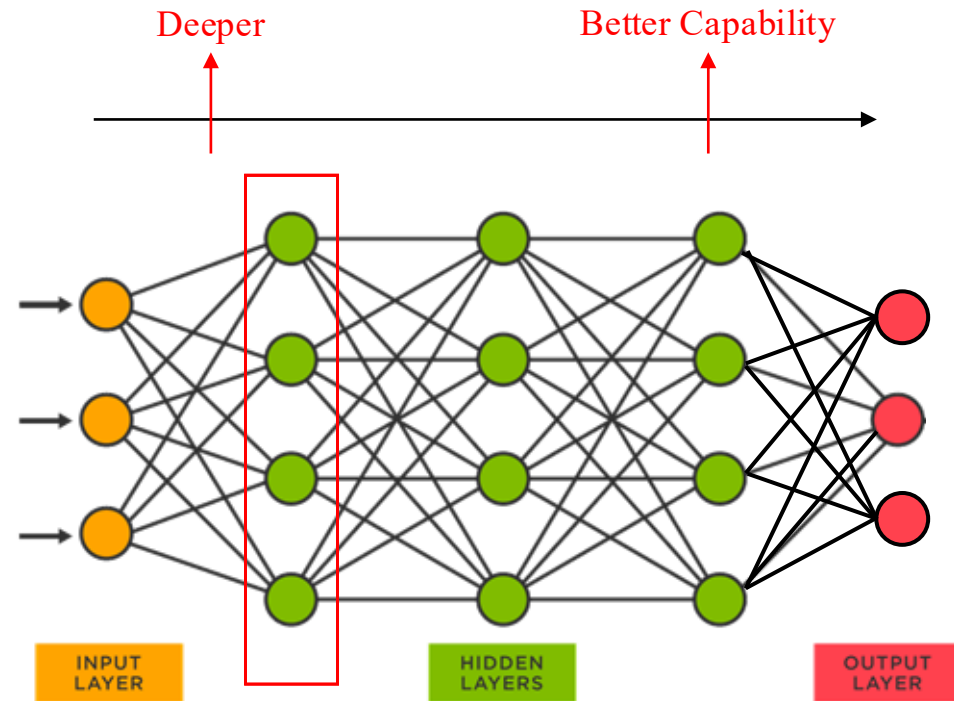
More than 14 million images have been hand-annotated for object classification

Year	Model	Top-5 error
2010	Hand-designed descriptors + SVM	28.20%
2011	Compressed Fisher Vectors + SVM	25.80%
2012	AlexNet	16.40%
2013	Variant of AlexNet	11.70%
2014	GoogLeNet	6.60%
2015	ResNet	4.50%

In 2015, ResNet achieved great success and was far ahead of other methods.

Deep Residual Net (ResNet)

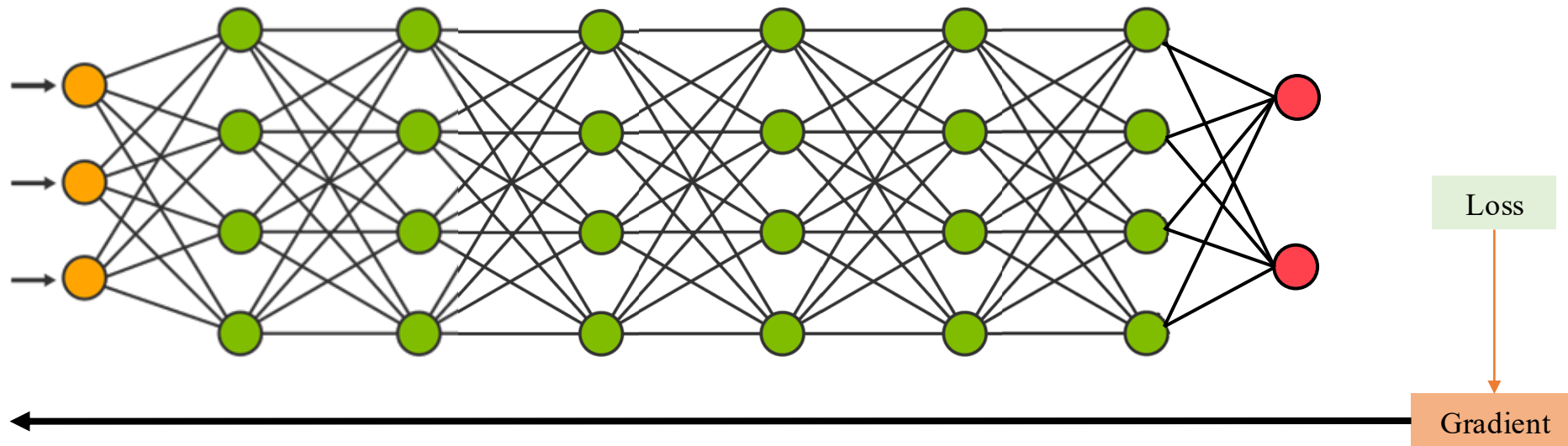
Making Network Deeper: Gradient Vanishing



- For MLP, each hidden layer can be seen as a feature extractor that makes the input information moving towards the discriminative features for the target.
- We may expect to increase the representation capability of the network by stacking many layers, making the network deep.

Deep Residual Net (ResNet)

Making Network Deeper: Gradient Vanishing



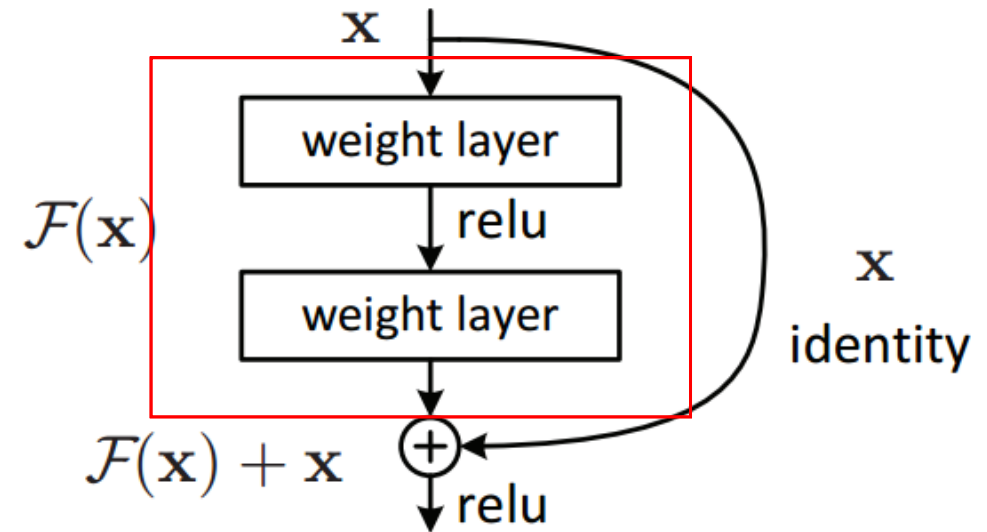
- When network gets deeper, it is hard to “influence” the top layers by backpropagating the gradients from bottom to up.
 - Multiplication of gradients over layers
 - The feedback from the “frontline” cannot be heard.
- The model **is unable to update effectively** for very deep cases: Gradient Vanishing

Deep Residual Net (ResNet)

Residual Block: Creating Highways with Skip Connection

- **Structure:** Each block consists of a skip connection that bypasses one or more layers.
- **Learning Residual Functions:** Instead of learning direct mappings, we learn residual functions with reference to the layer inputs, easing training.

Residual block is the core of ResNet which creates highways for backpropagation.

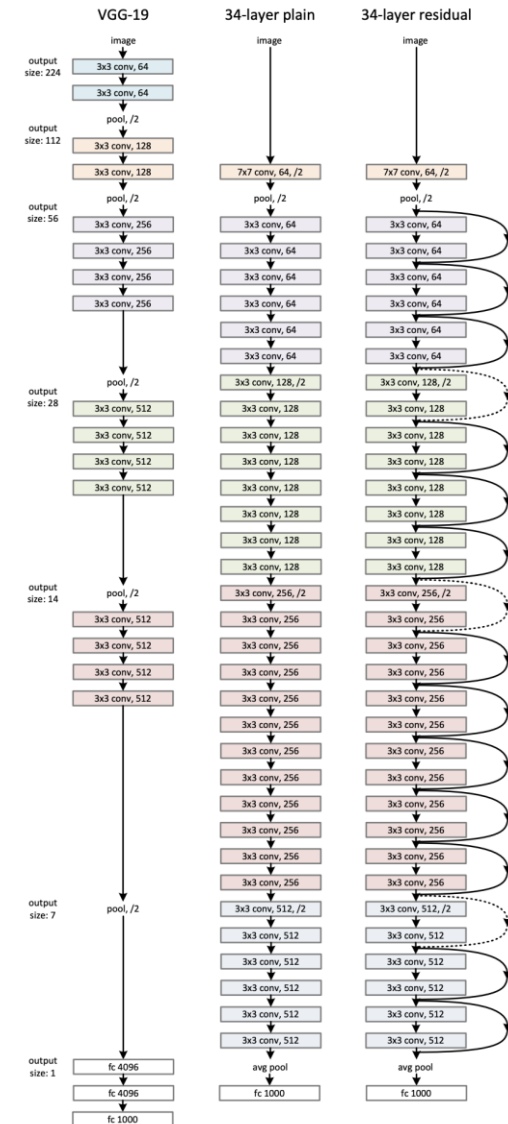


The network only needs to learn the “difference” between input and output.

Deep Residual Net (ResNet)

Residual Block: Creating Highways with Skip Connection

- The weight layer in the residual block can be other structures.
 - MLP
 - CNN
 - ...
- **Skip connections makes the ResNet be very deep!**
- **Popular ResNet Variants for image classification**
 - ResNet-18 & ResNet-34: Smaller-scale problems
 - ResNet-50: Balance of depth and computational efficiency
 - ResNet-101 and ResNet-152: For high-capacity model needs
- **ResNet's effectiveness is not limited to image classification.**

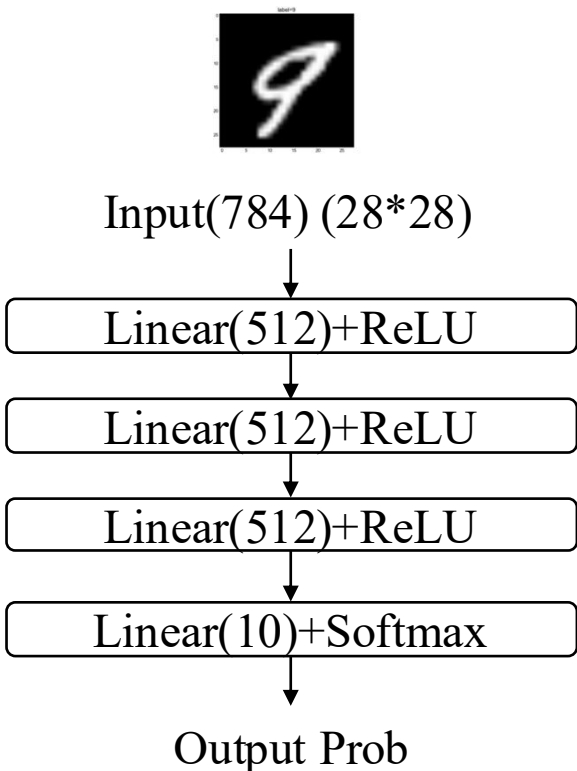


Deep Learning Outline

- A Journey of Deep Learning via MLP (A Simple Model)
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks (Other Models)
 - ResNet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

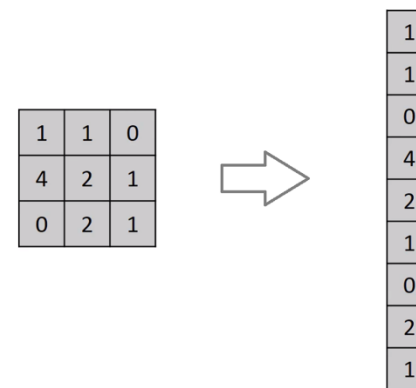
Convolutional Neural Network (CNN)

Problems of MLP



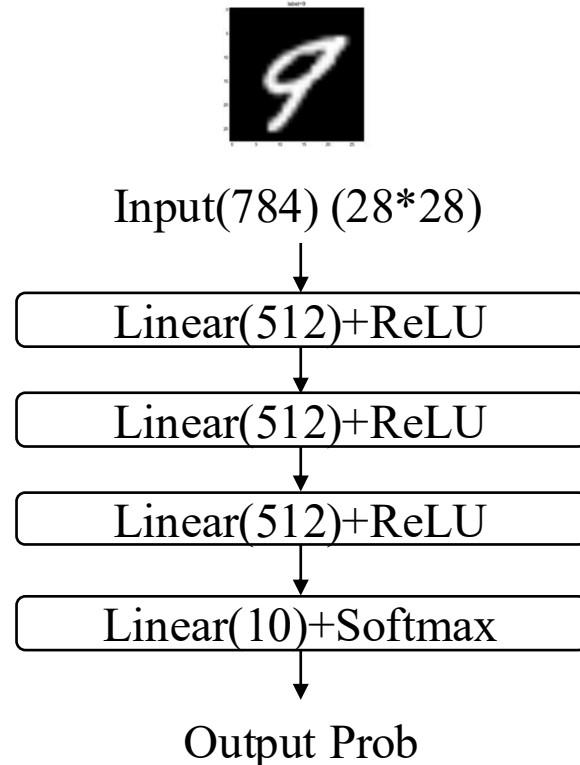
Let us take the handwritten digit classification as an example

- **Inefficiency in Handling Spatial Data:** MLPs treat input data as a flat vector, ignoring the 2D structure of images.



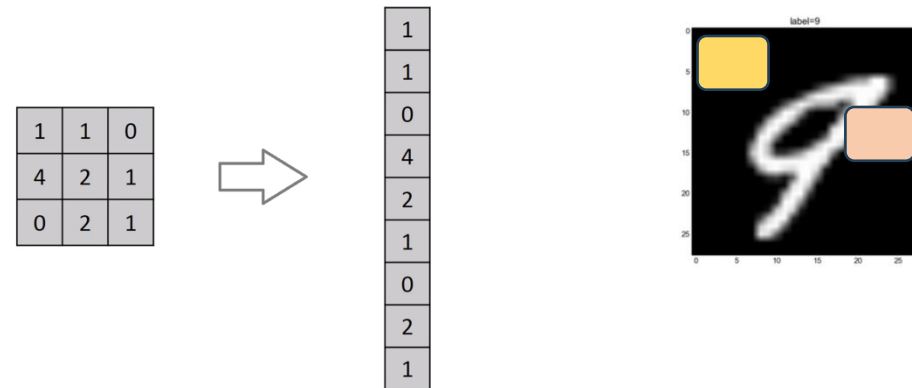
Convolutional Neural Network (CNN)

Problems of MLP



Let us take the handwritten digit classification as an example

- **High Number of Parameters:**
 - Fully connection leads to huge size expansion when the model gets deep and dealing with high-resolution images.
 - Parameters are not shared to handle similar operations!



In the MLP, because features are flattened, **we use distinct set of weights for similar operations on two marked regions.**

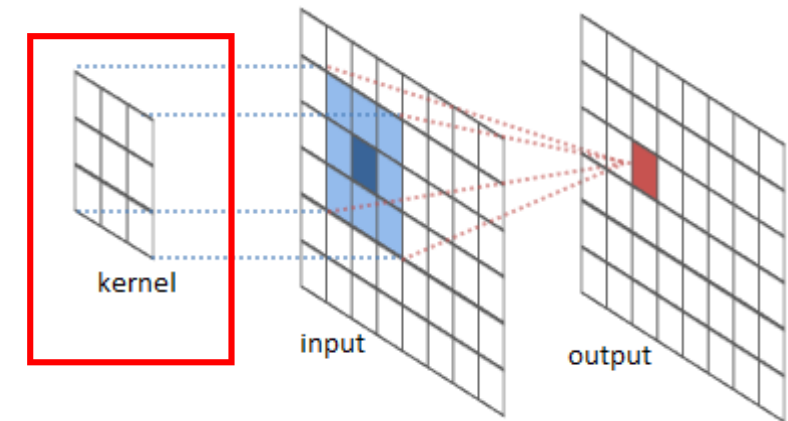
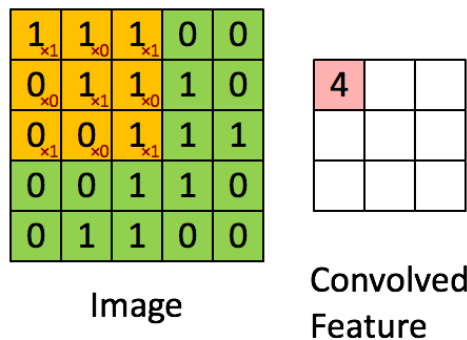
Convolutional Neural Network (CNN)

CNN: Designed for Spatial Data

Convolution Layer

Kernel

- Computes each element in the 2D output as the element-wise multiplication and summation between the 2D input, and the **kernel (or filter)**.
- In this way, the weights are shared for extracting features for different 2D regions.
- Activation function is applied to the linear weighting's output.



The same kernel moves to produce different elements in the output.

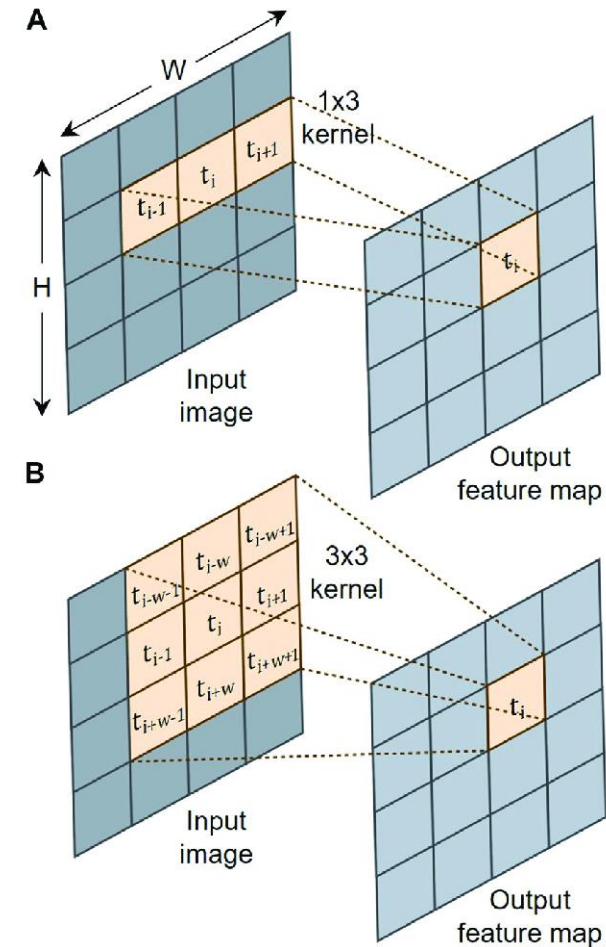
Convolutional Neural Network (CNN)

CNN: Designed for Spatial Data

Convolution Layer **Kernel**

Kernel Size

- The size of kernel determines how much information is used for each time to produce the output in the next layer.



Convolutional Neural Network (CNN)

CNN: Designed for Spatial Data

Convolution Layer Kernel

Stride Size

- The stride size determines the number of pixels by which the filter is moved across the input image during the convolution
- A **non-(1×1) stride size** makes **the resulting 2D feature smaller than the input, because it jumps**.
- It expands the area of the input image that contributes to each feature map, increases the “**receptive field**”.

Strided convolution

2	3	3	4	7	4	4	6	2	9
6	1	6	0	9	2	8	7	4	3
3	1	4	0	8	3	3	8	9	7
7	8	3	6	6	3	4			
4	2	1	8	3	4	6			
3	2	4	1	9	8	3			
0	1	3	9	2	1	4			

7×7

*

3	4	4
1	0	2
-1	0	3

3×3
Stride = 2

=

9		

Andrew Ng

The kernel **jumps!**

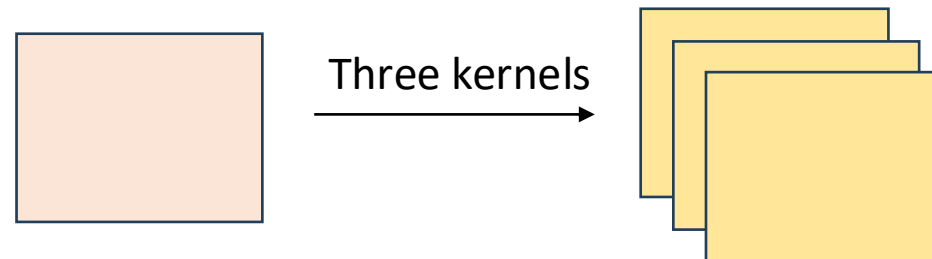
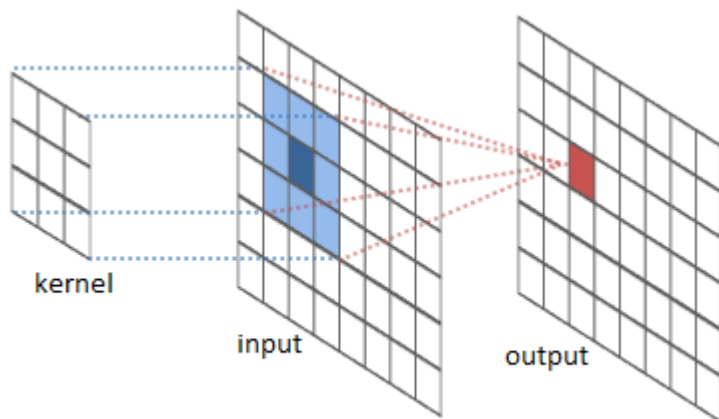
Convolutional Neural Network (CNN)

CNN: Designed for Spatial Data

Convolution Layer

Number of kernels

- We can use **multiple kernels** to extract different features for the same input!
- In this way, we have comprehensive description of different aspects of the same input feature.
- Multiple **channels** are produced.



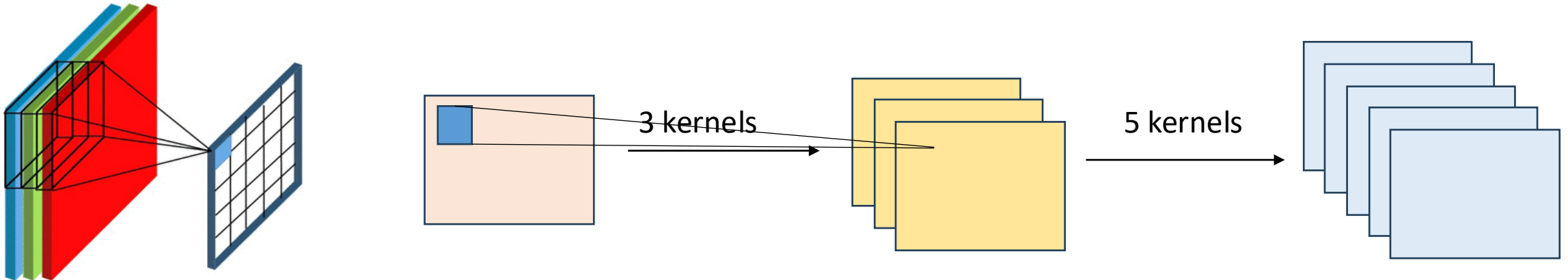
Convolutional Neural Network (CNN)

CNN: Designed for Spatial Data

Convolution Layer

Number of kernels

- The above operation can be generalized to multiple channels as input
- Each kernel is applied to all previous channels and sums up the results to produce one output.
- Stacking convolution layers extracts higher-level information (e.g., structures for image processing)



Convolutional Neural Network (CNN)

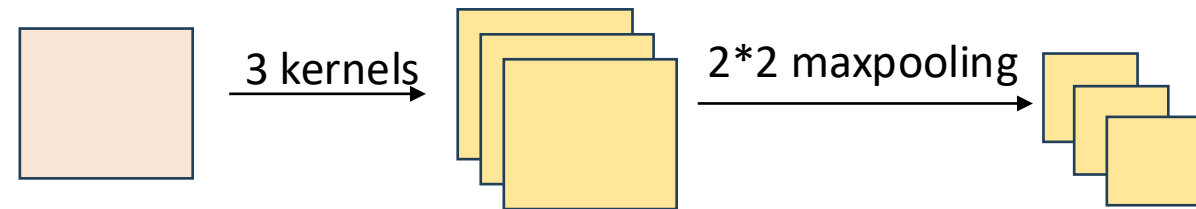
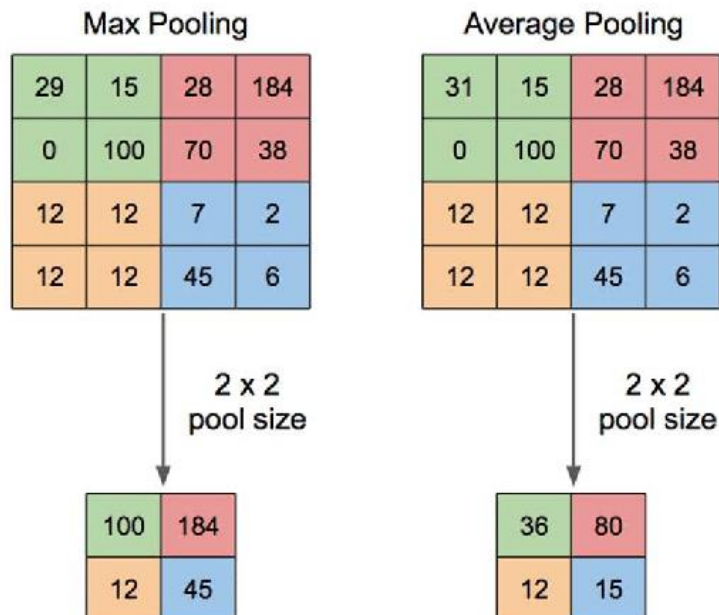
CNN: Designed for Spatial Data

Pooling Layer

- Reduce parameters
- Reduce pixel redundancy in local regions
- Further increase perceptive field

Types of Pooling Layer

- Max Pooling
- Average Pooling



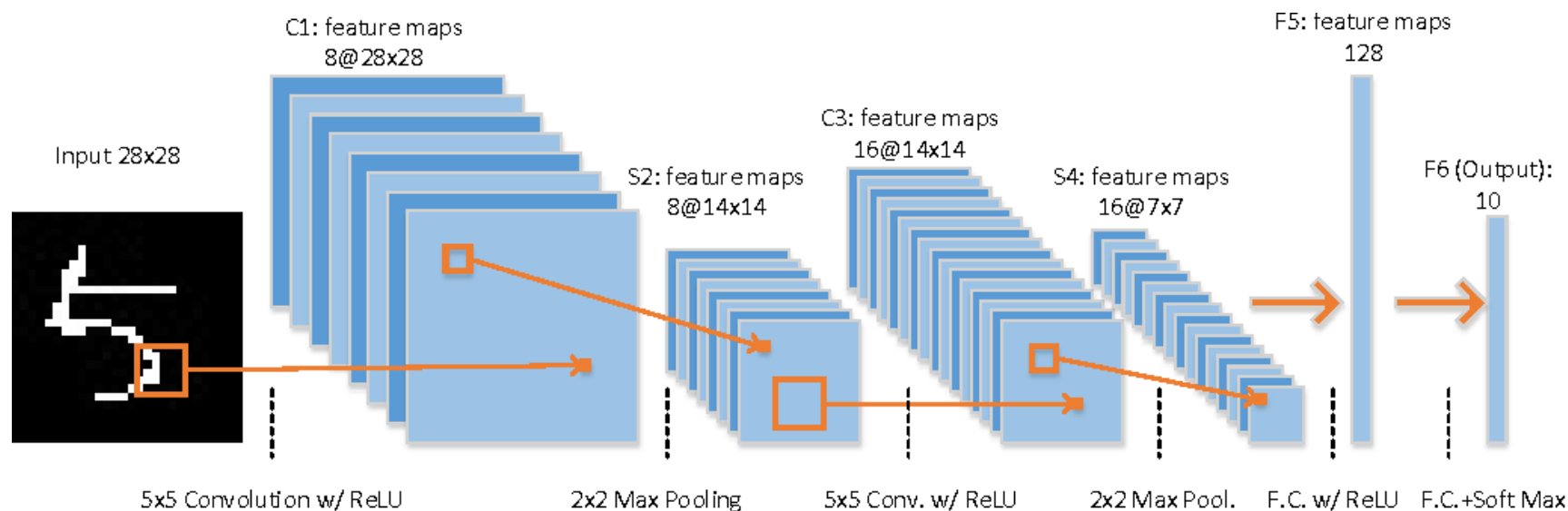
Final Effect of Pooling:

A large image is finally converted to small dimension features, making each feature represents the more global information

Convolutional Neural Network (CNN)

CNN: Designed for Spatial Data

CNN for handwritten digit recognition



The flatten is applied after a set of convolution and pooling, such that feature dimension is reduced significantly to capture the global information.

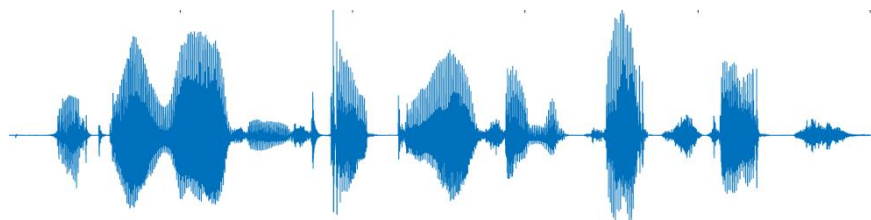
Deep Learning Outline

- A Journey of Deep Learning via MLP (A Simple Model)
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks (Other Models)
 - ResNet
 - CNN
 - RNN
 - Transformer
- Transfer Learning

Recurrent Neural Network (RNN)

Why RNN is needed? Modelling Sequential Data

- CNN mainly captures data with structures (e.g., images), while many other data is in the sequence form.
- Using MLP to model the sequence data is nearly impossible because the combination of feature dimensions for all time points, is extremely high.



Audio

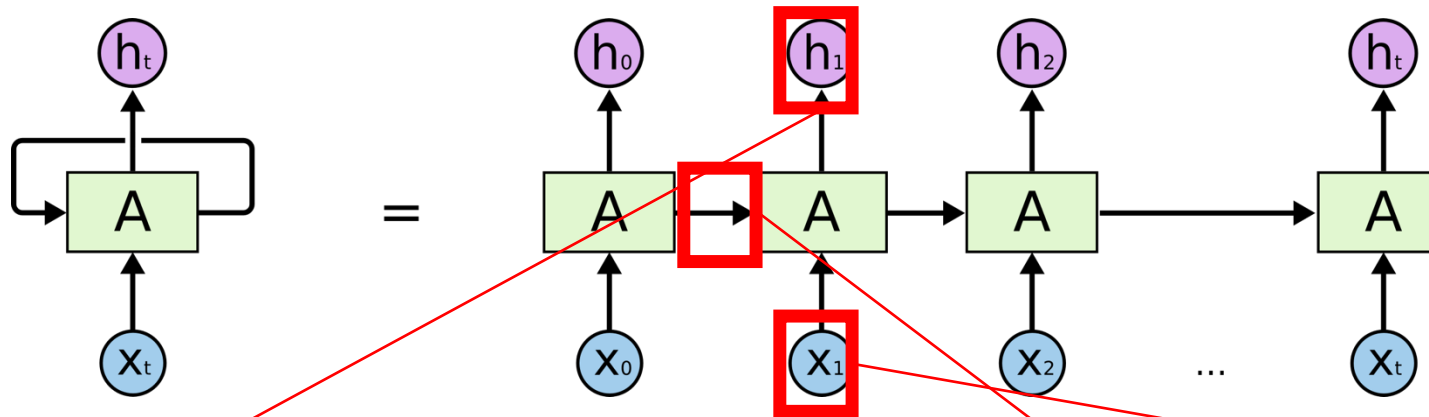
I read a book on Sunday

Text

Recurrent Neural Network (RNN)

What is the forward propagation of RNN?

- The idea of RNN, is to perform weight sharing over different time steps
- (Recall CNN performs weight sharing over different regions)

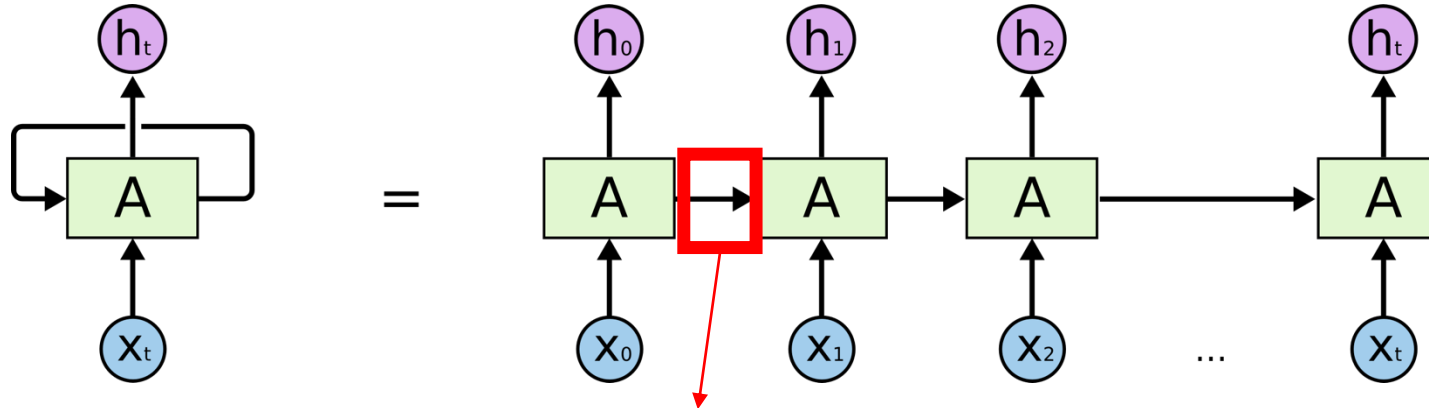


- For each time step, the network is based on its “**state**” and “**new input**” to produce the **output**.
- The process is **repeated recurrently**, so it is called recurrent neural network.

Recurrent Neural Network (RNN)

What is the forward propagation of RNN?

- The idea of RNN, is to perform weight sharing over different time steps
- (Recall CNN performs weight sharing over different regions)

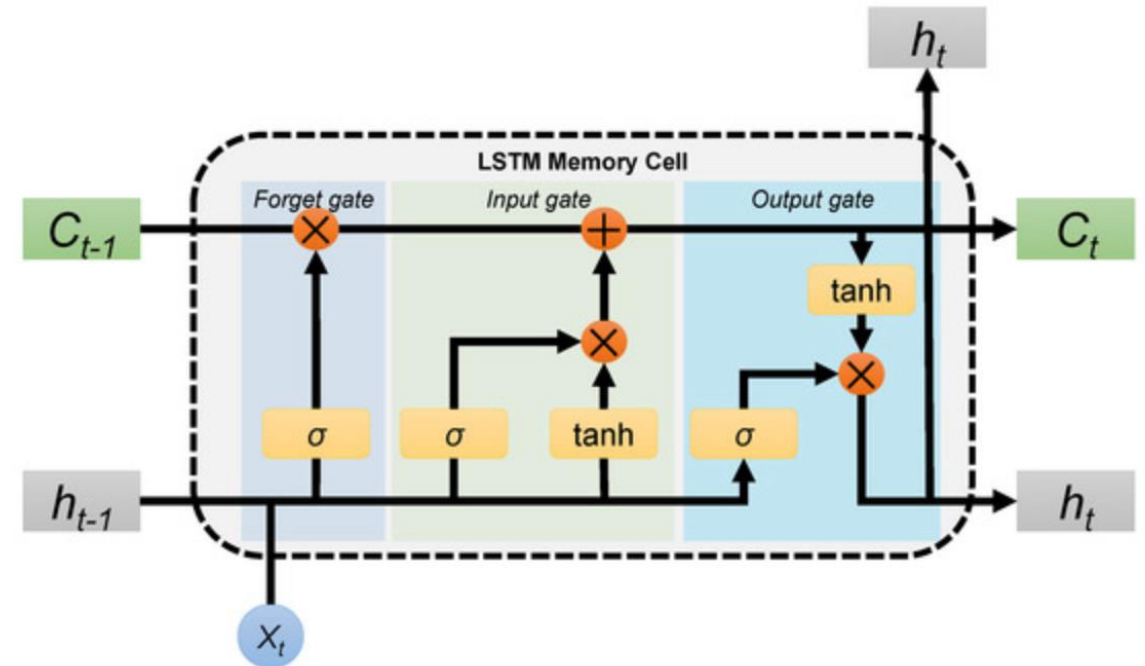
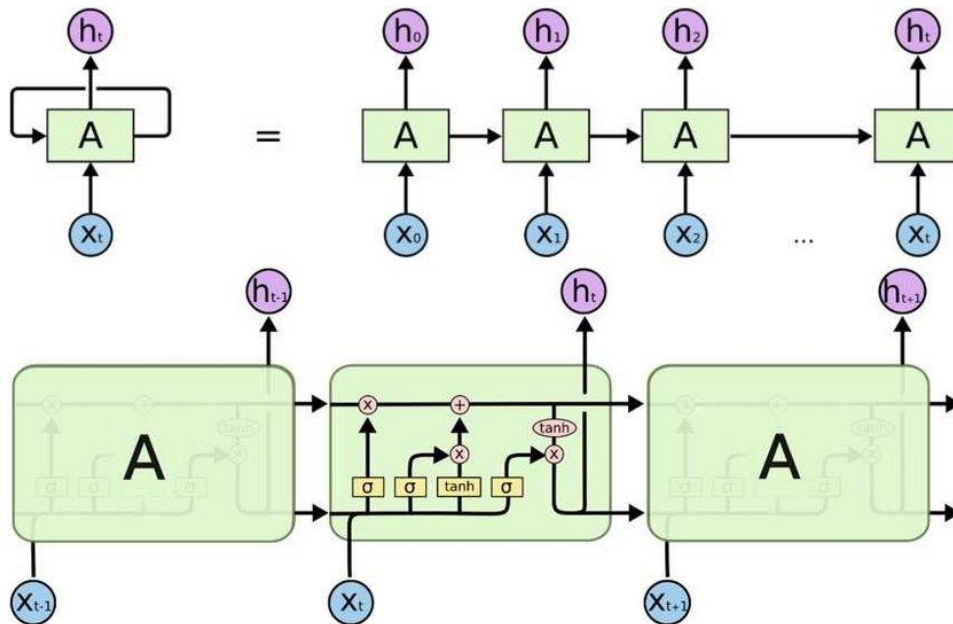


- The state, makes different steps correlated and being directional, making it suitable for sequence modelling.

Recurrent Neural Network (RNN)

How to combine the input and state to produce new state and output?

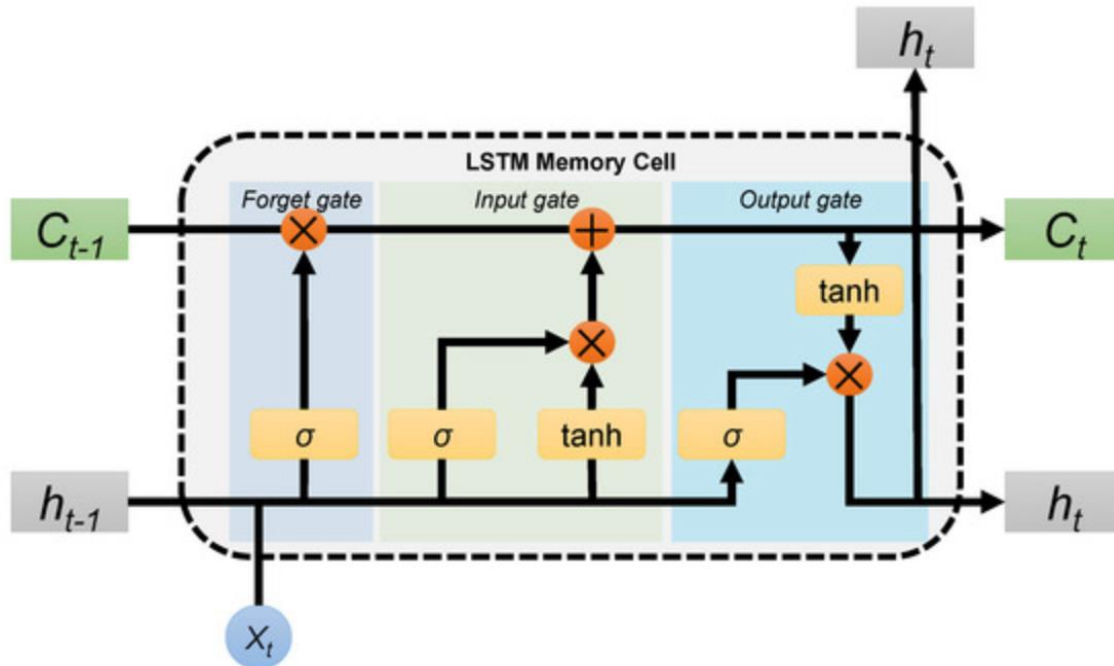
- Long Short-Term Memory Network (LSTM)



Recurrent Neural Network (RNN)

How to combine the input and state to produce new state and output?

- **Long Short-Term Memory Network (LSTM)**
 - Using Gates to Control Information over Sequences
 - Also takes the previous output as a part of input.



Forget Gate: Decides what information should be discarded from the cell state.

Input Gate: Decides which values to update

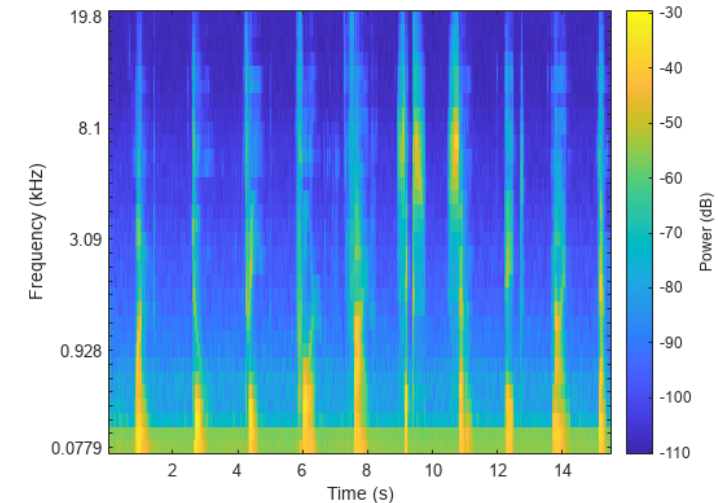
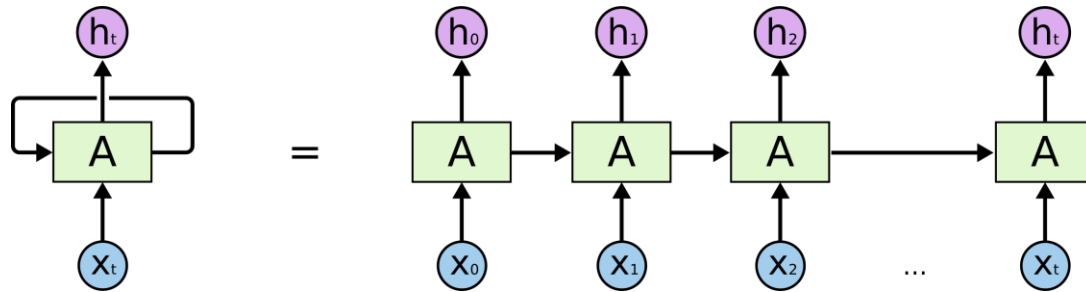
Cell State: Carries the information throughout the sequence processing

Output Gate: Decides what the output should be

Recurrent Neural Network (RNN)

What are the typical input and output dimensions?

- Long Short-Term Memory Network (LSTM)



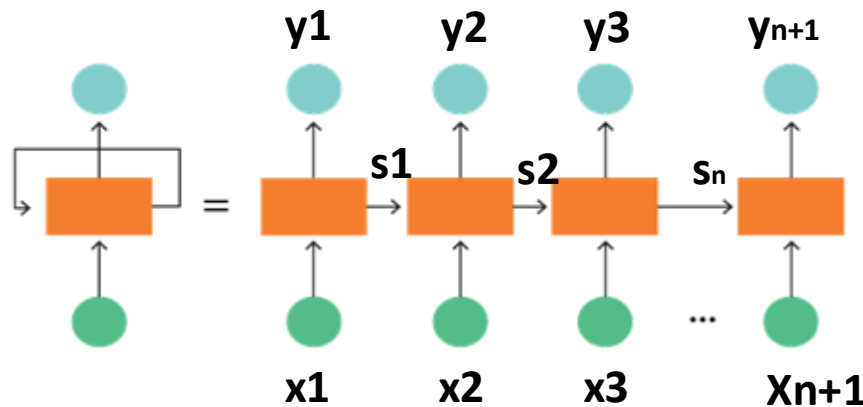
Supposing we want to train a speech recognizer.

- The input is typically 1000×80 , with 1000 the length of sequence, and 80 the dimension of feature at each time step
- The output is typically $1000 \times \mathcal{C}$, where $\mathcal{C}$ is the number of phoneme classes.
- **We can notice that the input and output are all sequences!**

Recurrent Neural Network (RNN)

Drawback

- The RNN is very **SLOW** for long sequences
 - We have to perform prediction step by step (1->2->3)
- This makes it not efficient for both training and inference.
- Modelling very long context can be difficult.



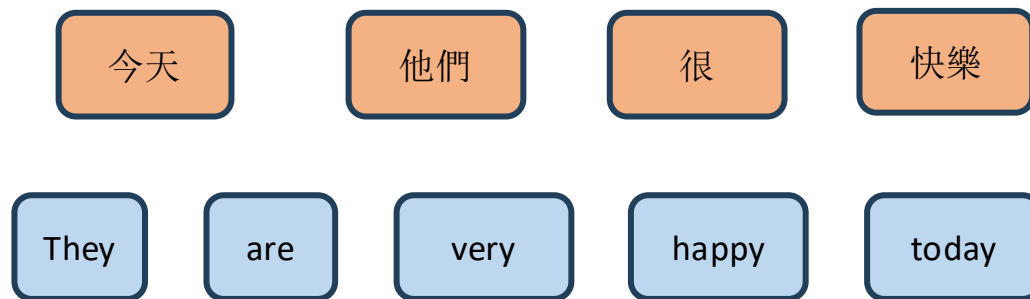
Deep Learning Outline

- A Journey of Deep Learning via MLP (A Simple Model)
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks (Other Models)
 - ResNet
 - CNN
 - RNN
 - **Transformer**
- Transfer Learning

Transformer

Why is Transformer needed? Efficient Modelling Sequential Data

- Considering the machine translation problem, which transforms Chinese sequence to English.

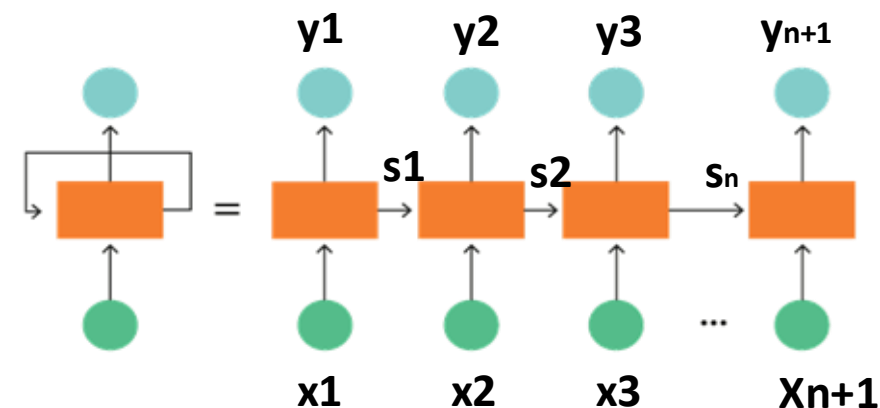
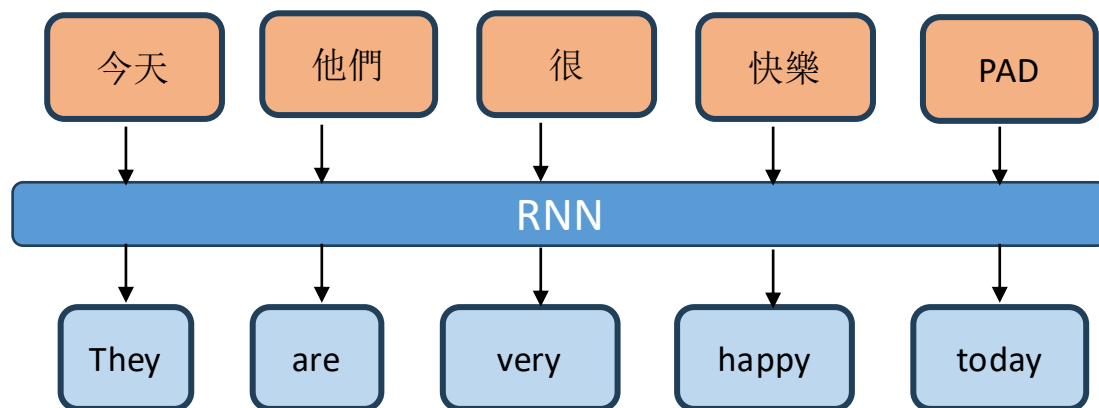


Machine Translation

Transformer

Why is Transformer needed? Efficient Modelling Sequential Data

- Using RNN framework:

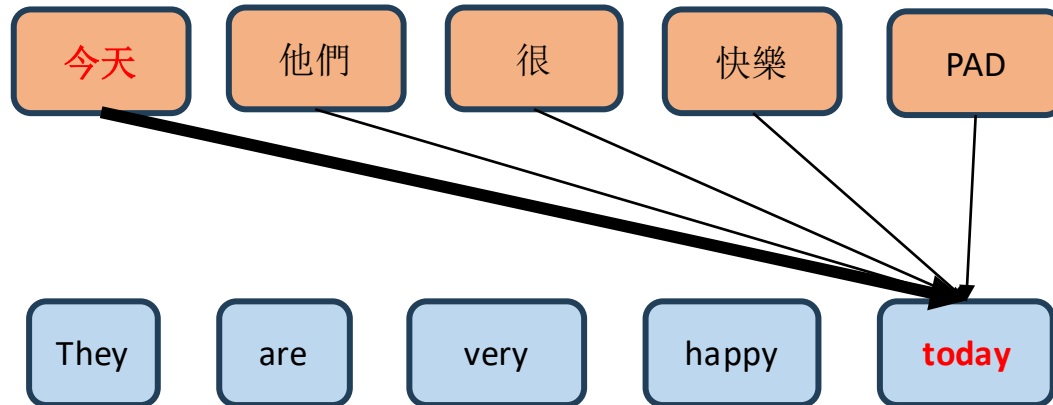


The input and output are first padded to the same length, and perform sequence-to-sequence prediction

Transformer

Why is Transformer needed? Efficient Modelling Sequential Data

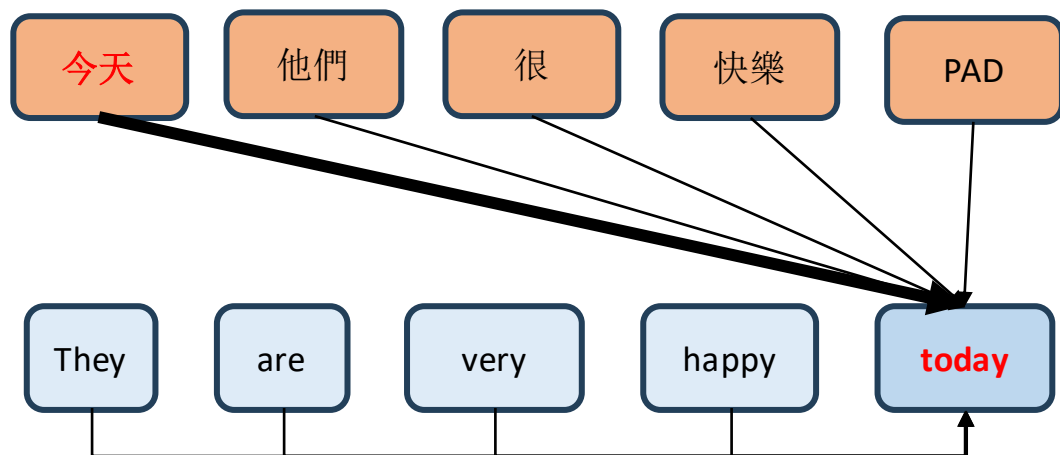
- Find the most relevant information for prediction!



- In fact, “today” is much more related to “今天(jin tian)” than other words, therefore, we can give more **attentions** directly to “今天 (jin tian)” when performing translation. (other words are also useful, for instance, to determine the order of output words.)
- Transformer wants to gather information without step-by-step prediction over time.

Transformer

A practical Transformer structure for machine translation



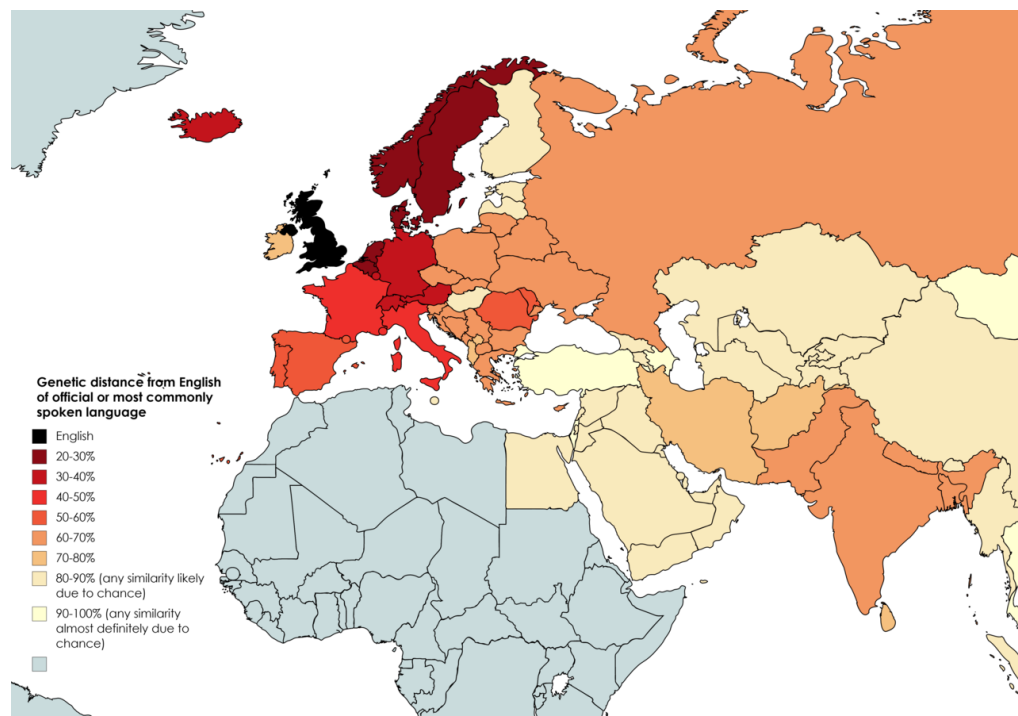
- In the actual translation, the **previous outputs** are also taken as the inputs for the next work prediction.
- Transformer mainly refers using attention for sequence modeling

Deep Learning Outline

- A Journey of Deep Learning via MLP (A Simple Model)
 - Multi-layer Perceptron (MLP)
 - Activation Functions
 - Forward Propagation
 - Loss Functions
 - Optimization with Backpropagation
- Advanced Deep Neural Networks (Other Models)
 - ResNet
 - CNN
 - RNN
 - Transformer
- **Transfer Learning**

Transfer Learning

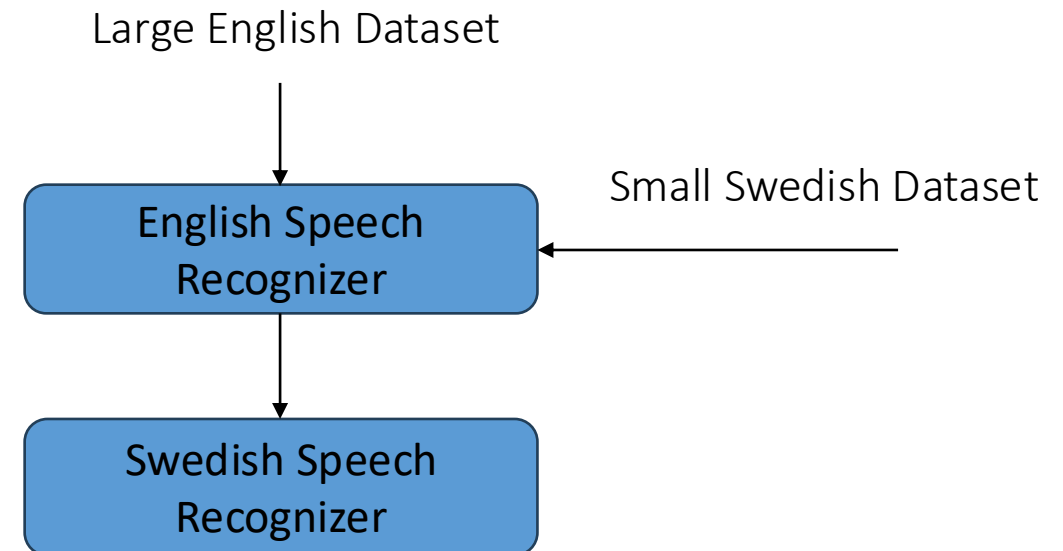
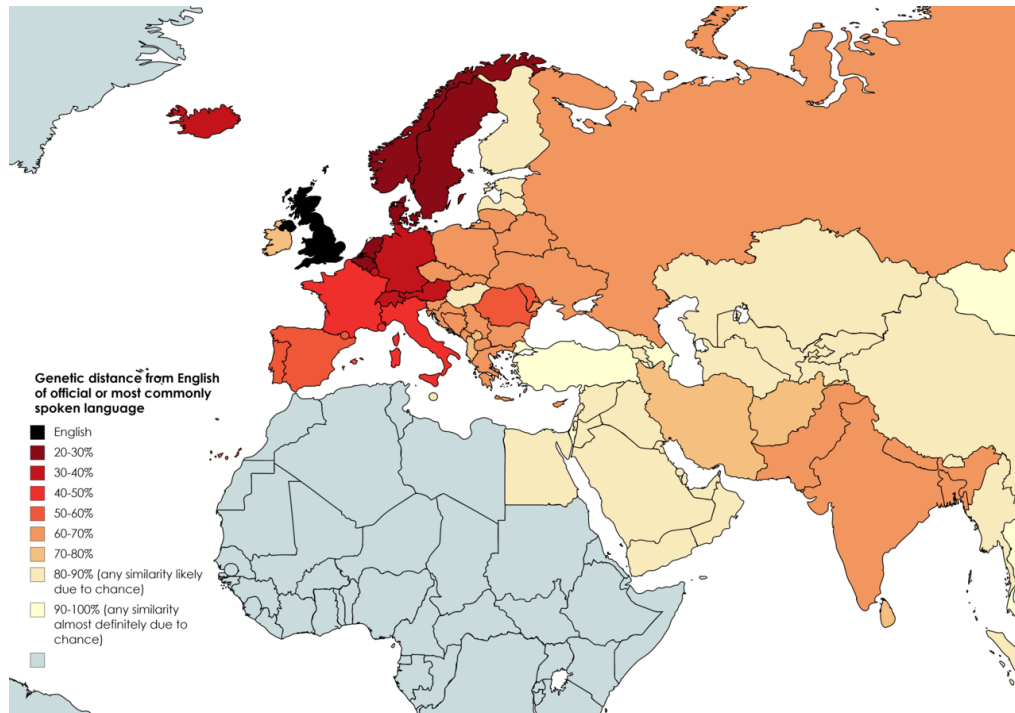
- Transfer knowledge from a related task that has already been learned to improve learning in a new task.



Europeans learn English faster than Chinese, because their languages are more similar to English

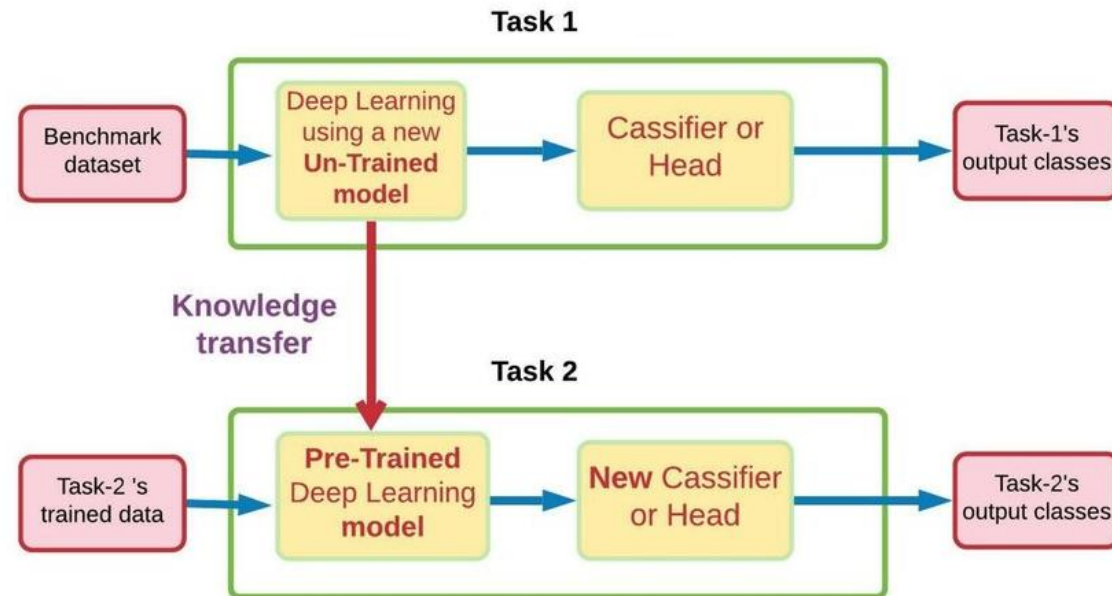
Transfer Learning

- Similarly, if you want to train a speech recognizer for Swedish
 - Swedish data is much less than English
 - We can make the model to recognize English first, and then learn Swedish on less data



Transfer Learning

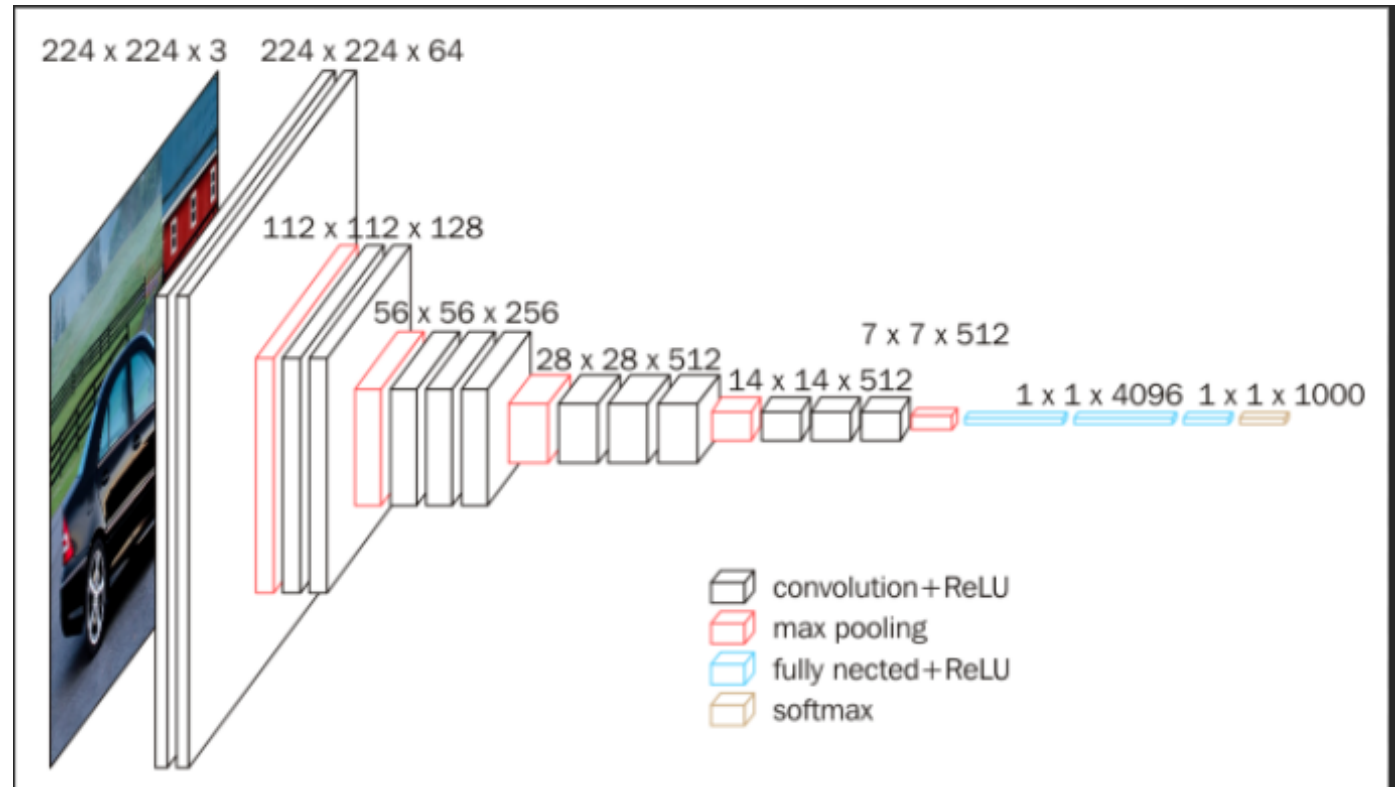
- Steps of Transfer Learning
 - **Pretraining:** Model training on Large-scale generic data
 - **Fine-Tuning:** Adaptation on Small-scale task for the specific task
- Transfer learning performs knowledge transfer among tasks by providing a good starting point.



Transfer Learning

Popular Pre-Trained Architectures

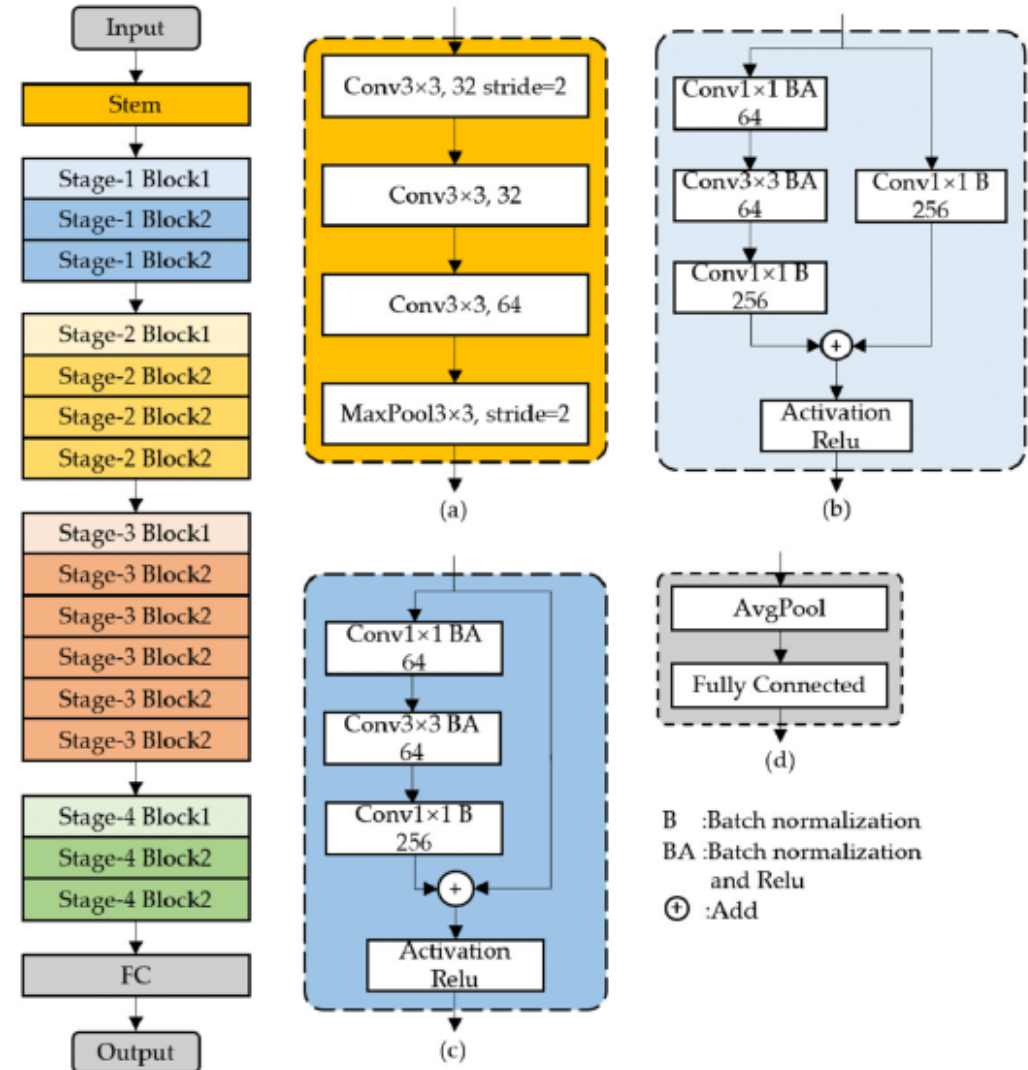
- **VGG (Visual Geometry Group):**
VGG is a family of convolutional neural networks that were first introduced in 2014. They are known for their simplicity and efficiency, and they have been used for a variety of tasks, including image classification, object detection, and segmentation



Transfer Learning

Popular Pre-Trained Architectures

- **ResNet (Residual Network):** ResNet is a family of convolutional neural networks that were introduced in 2015. They are known for their ability to learn deeper features, and they have been shown to achieve state-of-the-art results on a variety of tasks.

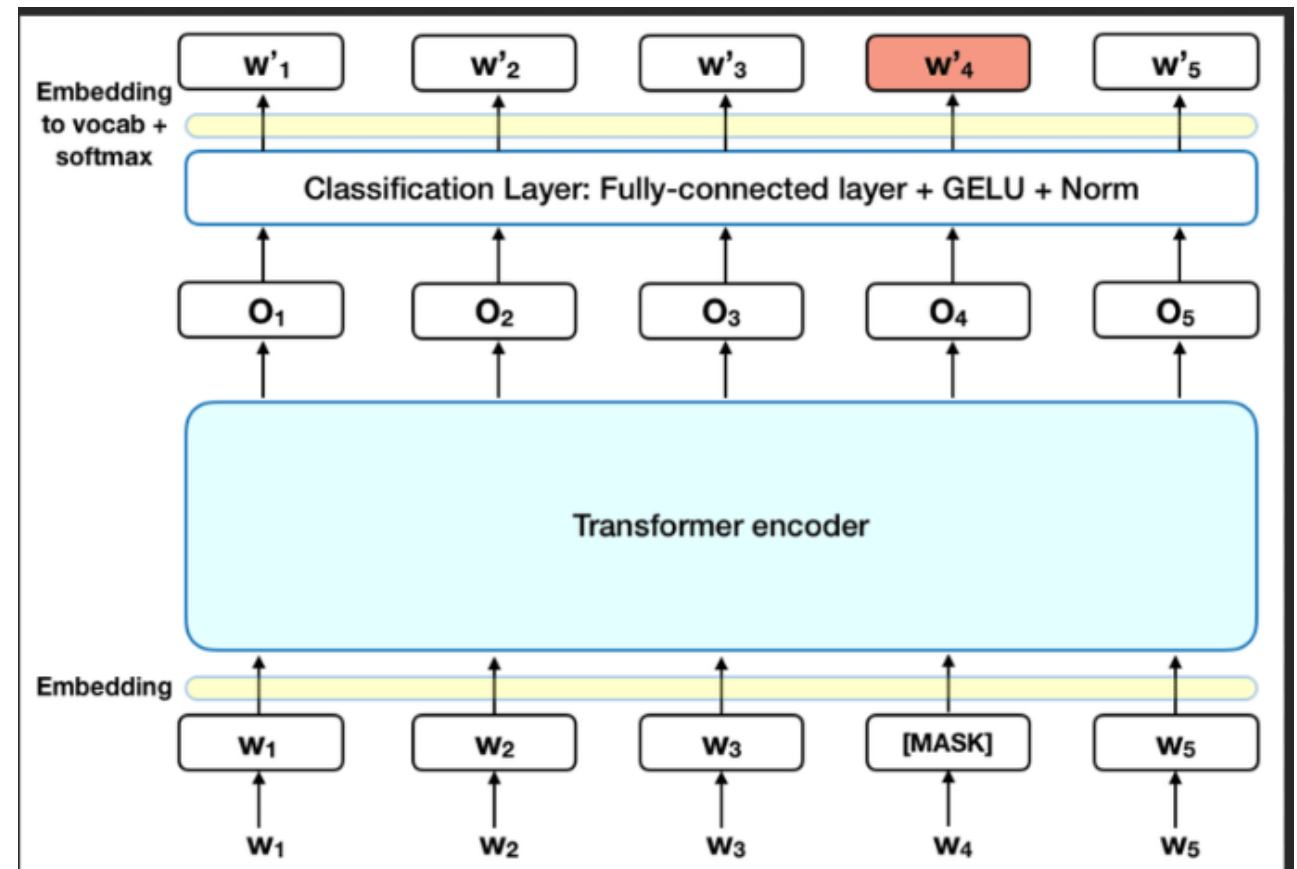


The architecture of ResNet-50-vd. (a) Stem block; (b) Stage1-Block1; (c) Stage1-Block2; (d) FC-Block.

Transfer Learning

Popular Pre-Trained Architectures

- **BERT (Bidirectional Encoder Representations from Transformers):** BERT is a language model that was introduced in 2018. It is known for its ability to learn long-range dependencies in text, and it has been used for a variety of natural language processing tasks, including question-answering, sentiment analysis, and text summarization.

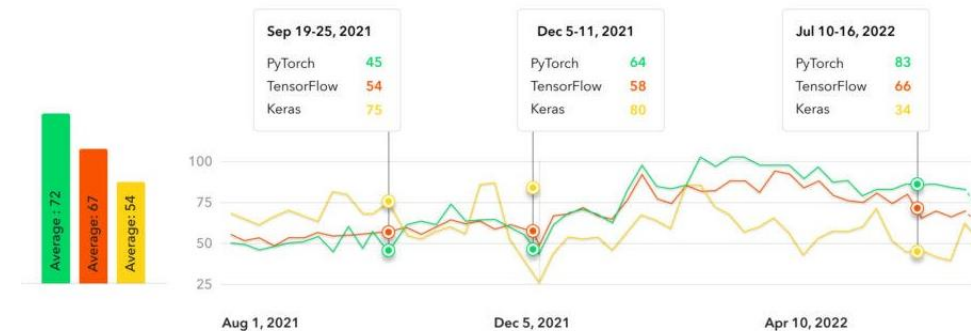


What is Pytorch?

- Pytorch is a powerful, popular and easy-to-use free open source Python library for developing and evaluating deep learning (DL) models.
- It allows you to define and train neural network models easily.
- While many DL frameworks are available, the pipelines are very similar. Here we take Pytorch as an example.

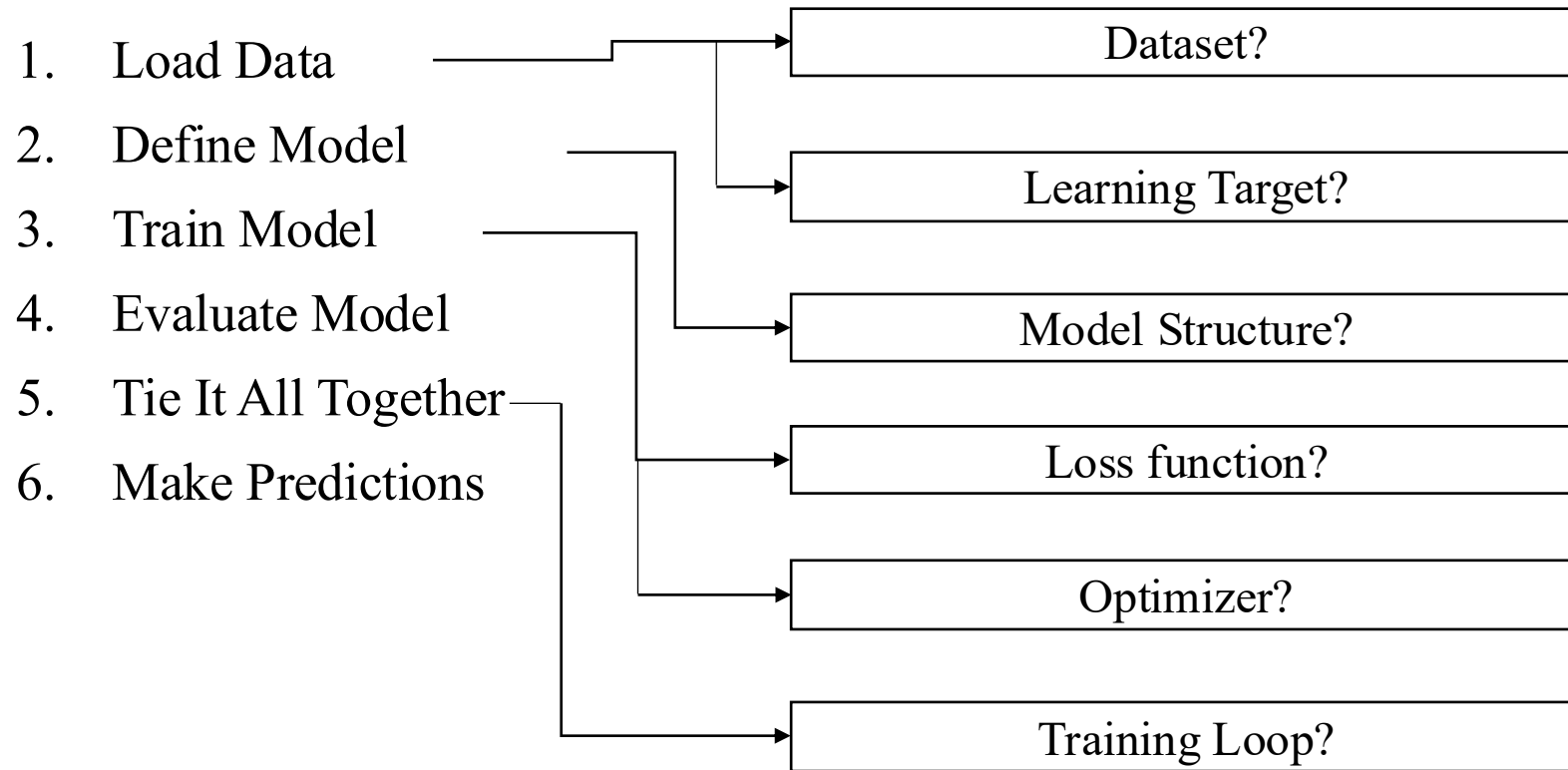


Top 3 Deep Learning Frameworks



Pytorch has become the most popular DL framework

Key Components of Pytorch Pipeline



Install and Import Pytorch

- In Colab, run

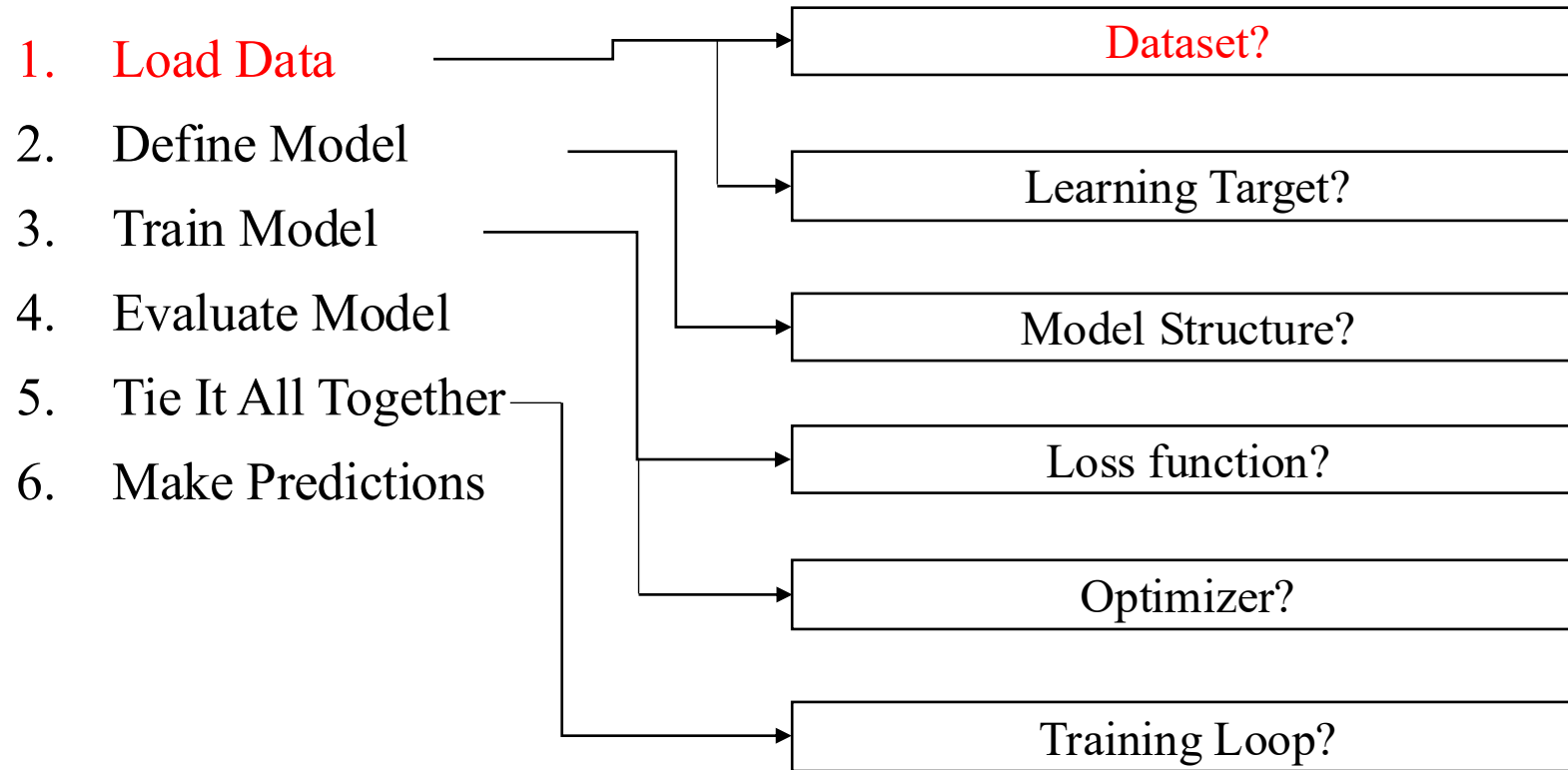
```
pip install torch  
import torch
```

Key Components of Pytorch Pipeline

- We will learn each step based on a famous task:
 - **MNIST handwritten digits recognition**



Key Components of Pytorch Pipeline



Loading Datasets

- Let us first download the data from the Canvas:

```
train-images-idx3-ubyte.gz: training set images (9912422 bytes)  
train-labels-idx1-ubyte.gz: training set labels (28881 bytes)  
t10k-images-idx3-ubyte.gz: test set images (1648877 bytes)  
t10k-labels-idx1-ubyte.gz: test set labels (4542 bytes)
```

These files contains the pixel values and labels for each digit image. The pixel values range from 0 to 255.

Loading Datasets

- Using the codes in `convert.py` to convert the raw dataset to the csv files, we can check the details of the dataset:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
25	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
26	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

label

Pixel values (features)

mnist_test.csv

Try converting the dataset by yourself and check the csv files.

Loading Datasets

- Although there are some wrapped API to load the MNIST dataset, we will introduce **the general ways of loading a dataset**.
- Define a customized [data generator](#) to load the data sample one by one.

Defining a data generator

In many cases the dataset is huge such that it is impossible to load the entire dataset into the memory before training.

- Speech Recognition: 20,000 hours of data

We can define a customized “data generator” to specify the rules of creating one sample.

Loading Datasets

Defining a data generator

In PyTorch, a data generator is typically implemented using the Dataset and DataLoader classes. The Dataset class requires three methods `__init__`, `__len__`, and `__getitem__`.

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    'Dataset for PyTorch'
    def __init__(self, param1, param2):
        # Initialize variables according to the input parameters
        # Any number of parameters are supported
        # some initialization code

    def __len__(self):
        # Return the total number of samples in the dataset
        # some length calculation code

    def __getitem__(self, index):
        # Generate one sample based on the index
        # some item retrieval code
        return sample
```

Please also learn
the definition of
Class in Python

Loading Datasets

Defining a data generator

__init__

In PyTorch, instead of loading all the data at once, we typically define a custom Dataset class that reads one line (or a chunk of lines) from a file when generating each sample.

```
import torch
from torch.utils.data import Dataset
import csv
import random

class CSVDataset(Dataset):
    Dataset for PyTorch loading from CSV one line at a time'
    def __init__(self, csv_path):
        # Store the path to the CSV file
        self.csv_path = csv_path
        # Get the number of lines in the CSV file
        self.num_samples = self._get_num_samples()
        # List of indices to allow random shuffling
        self.indexes = list(range(self.num_samples))
```

In the init function, we can define any class variables that can be used elsewhere in the class

Now we define where to find the data, how many data are included, and the index of data sample

Loading Datasets

Defining a data generator

__init__

In PyTorch, instead of loading all the data at once, we typically define a custom Dataset class that reads one line (or a chunk of lines) from a file when generating each sample.

```
def _get_num_samples(self):  
    with open(self.csv_path, 'r') as file:  
        info = file.readlines()  
        return len(info)-1 # exclude the header line
```

The initialization also relies on a function called `_get_num_samples`, which can be defined separately

Loading Datasets

Defining a data generator

- len
- The function returns the total number of samples in the dataset
- Given this value the data generator knows whether all the data has been used for one-epoch training.

```
def __init__(self, csv_path):  
    # Store the path to the CSV file  
    self.csv_path = csv_path  
    # Get the number of lines in the CSV file  
    self.num_samples = self._get_num_samples()  
    # List of indices to allow random shuffling  
    self.indexes = list(range(self.num_samples))  
  
def _get_num_samples(self):  
    with open(self.csv_path, 'r') as file:  
        info = file.readlines()  
    return len(info)-1 # exclude the header line  
  
def __len__(self):  
    # Return the total number of samples in the dataset  
    return self.num_samples
```

In this example, we can easily get the number of samples from `self.num_samples` calculated in `init`.

Loading Datasets

Defining a data generator

__getitem__

- Return a training sample according to the index value;
- The index value will iterate from 0 to L-1, where L is the value returned by `__len__`;
- `__getitem__` function defines the rules of creating the sample given the index

```
def __getitem__(self, index):  
    # Translate the random index to the corresponding line in the file  
    line_number = self.indexes[index]  
    with open(self.csv_path, 'r') as file:  
        reader = csv.reader(file)  
        for _ in range(line_number + 1): # Skip to the desired line  
            line = next(reader)  
    # Convert the line to a tensor  
    data = torch.tensor([int(i) for i in line], dtype=torch.float32)  
    feat = data[1:]/255.0  
    label = data[0]  
    return feat, label
```

Now we read the index-th line of the csv file and read the data. The feature values are normalized, and the first element is the label.

Loading Datasets

Defining a data generator
Now combine everything:

```
import torch
from torch.utils.data import Dataset
import csv
import random

class CSVDataset(Dataset):
    # Dataset for PyTorch loading from CSV one line at a time
    def __init__(self, csv_path):
        # Store the path to the CSV file
        self.csv_path = csv_path
        # Get the number of lines in the CSV file
        self.num_samples = self._get_num_samples()
        # List of indices to allow random shuffling
        self.indexes = list(range(self.num_samples))
    def _get_num_samples(self):
        with open(self.csv_path, 'r') as file:
            info = file.readlines()
            return len(info)-1 # exclude the header line
    def __len__(self):
        # Return the total number of samples in the dataset
        return self.num_samples

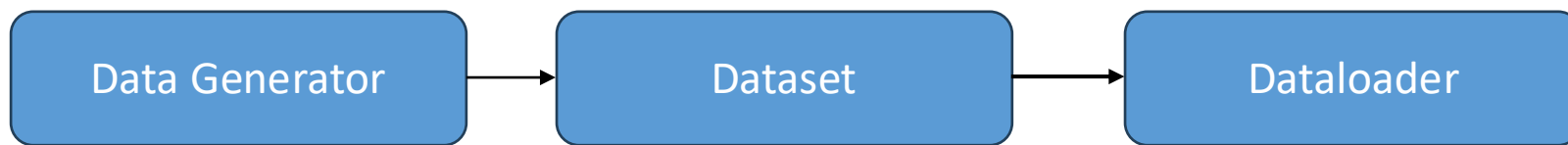
    def __getitem__(self, index):
        # Translate the random index to the corresponding line in the file
        line_number = self.indexes[index]
        with open(self.csv_path, 'r') as file:
            reader = csv.reader(file)
            for _ in range(line_number + 1): # Skip to the desired line
                line = next(reader)
            # Convert the line to a tensor
            data = torch.tensor([int(i) for i in line], dtype=torch.float32)
            feat = data[1:]/255.0
            label = data[0]
            return feat, label
```

Loading Datasets

Using a data generator

With the data generator, we can further define the “**dataloader**”.

Now we first clarify some similar concepts



How one sample is fetched

A collection of such samples

Fetching **batches of samples** for training

Then we can define the dataloader to load the data for training.

```
from torch.utils.data import Dataset, DataLoader
```

```
dataset = CSVDataset('path_to_csv.csv')
```

```
dataloader = torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=True)
```

Loading Datasets

Using a data generator

With the data generator, we can further define the “**dataloader**”.

Then we can define the dataloader to load the data for training.

```
from torch.utils.data import Dataset, DataLoader

dataset = CSVDataset('path_to_csv.csv')
dataloader = torch.utils.data.DataLoader(dataset, batch_size=8, shuffle=True)
```

batch_size: how many samples are included in one batch

shuffle: whether we will randomize the order of fetching samples in one epoch.

Loading Datasets

Using a data generator

The training batches can be fetched from the dataloader for training.

```
# Create datasets
dataset = CSVDataset('mnist_test.csv')

# Splitting the dataset into train and validation sets
train_size = int(0.9 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(dataset, [train_size,
val_size])
# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=8, shuffle=False)
# Extract a batch of data
for i, data in enumerate(train_loader):
    inputs, labels = data
    print(labels)
    if i == 2: # Just to show 2 batches
        break
```

Try run the codes for multiple times and check the outputs. Why each run gives a different output?

Loading Datasets

1. Define a dataset
 1. Init
 2. len
 3. getitem
2. Create a dataset
3. Create a dataloader
4. Fetch data batches

```
import torch
from torch.utils.data import Dataset, DataLoader
import csv
import random

class CSVDataset(Dataset):
    # Dataset for PyTorch loading from CSV one line at a time
    def __init__(self, csv_path):
        # Store the path to the CSV file
        self.csv_path = csv_path
        # Get the number of lines in the CSV file
        self.num_samples = self._get_num_samples()
        # List of indices to allow random shuffling
        self.indexes = list(range(self.num_samples))
    def _get_num_samples(self):
        with open(self.csv_path, 'r') as file:
            info = file.readlines()
            return len(info)-1 # exclude the header line
    def __len__(self):
        # Return the total number of samples in the dataset
        return self.num_samples

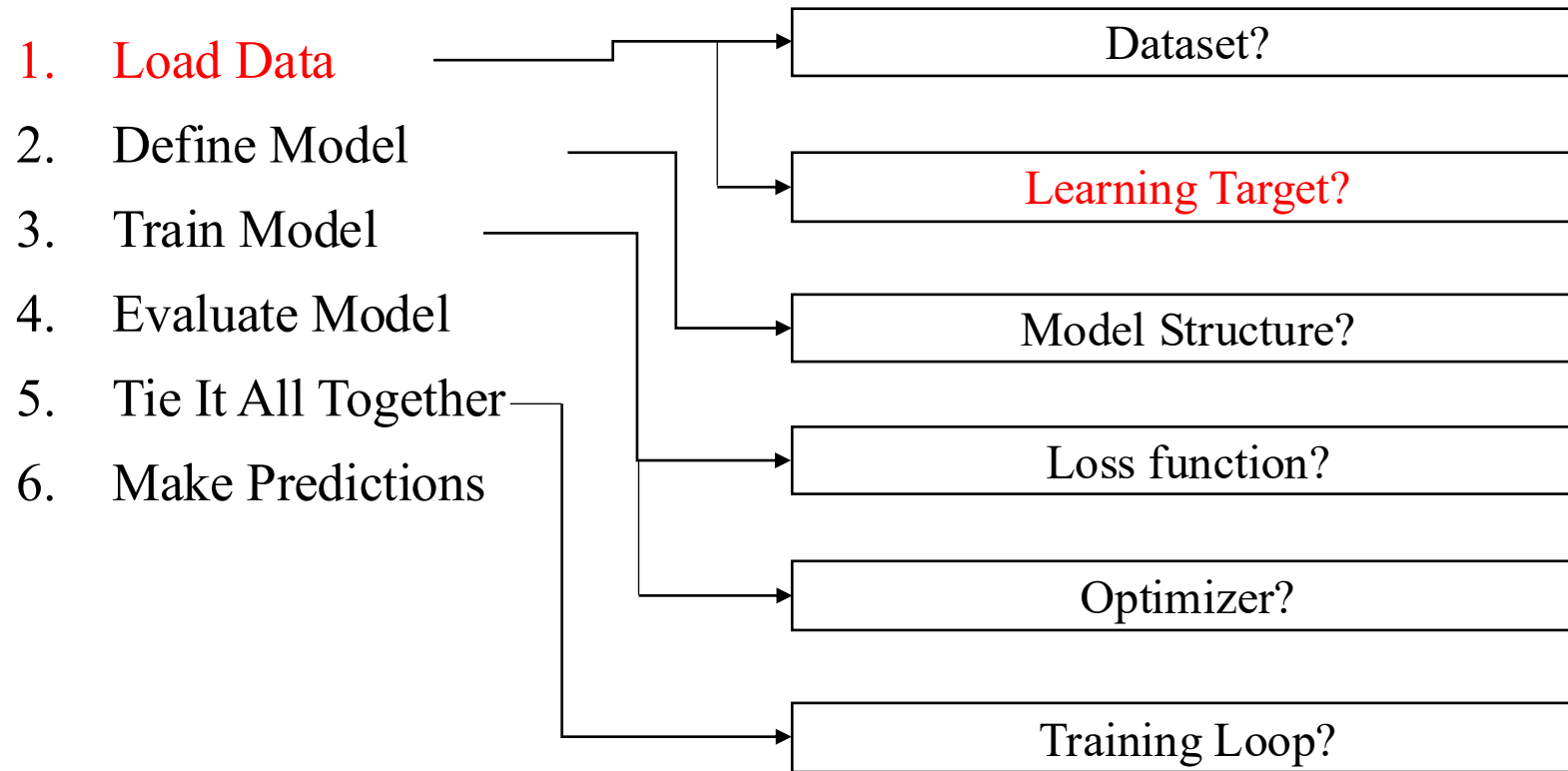
    def __getitem__(self, index):
        # Translate the random index to the corresponding line in the file
        line_number = self.indexes[index]
        with open(self.csv_path, 'r') as file:
            reader = csv.reader(file)
            for _ in range(line_number + 1): # Skip to the desired line
                line = next(reader)
            # Convert the line to a tensor
            data = torch.tensor([int(i) for i in line], dtype=torch.float32)
            feat = data[1:]/255.0
            label = data[0]
            return feat, label

# Create datasets
dataset = CSVDataset('mnist_test.csv')
# Splitting the dataset into train and validation sets
train_size = int(0.9 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(dataset, [train_size, val_size])

# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)

# Extract a batch of data
for i, data in enumerate(val_loader):
    inputs, labels = data
    print(labels)
    if i == 1: # Just to show 2 batches
        break
```

Key Components of Pytorch Pipeline



Loading Datasets

Defining the learning target

The learning target is defined as the label when creating a sample.

```
def __getitem__(self, index):
    # Translate the random index to the corresponding line in the file
    line_number = self.indexes[index]
    with open(self.csv_path, 'r') as file:
        reader = csv.reader(file)
        for _ in range(line_number + 1): # Skip to the desired line
            line = next(reader)
    # Convert the line to a tensor
    data = torch.tensor([int(i) for i in line], dtype=torch.float32)
    feat = data[1:]/255.0
    label = data[0]
    return feat, label
```

In our case, we want to classify the digits, therefore, the digit value itself, stored in the first element in each line, is taken as the learning target

Exercise

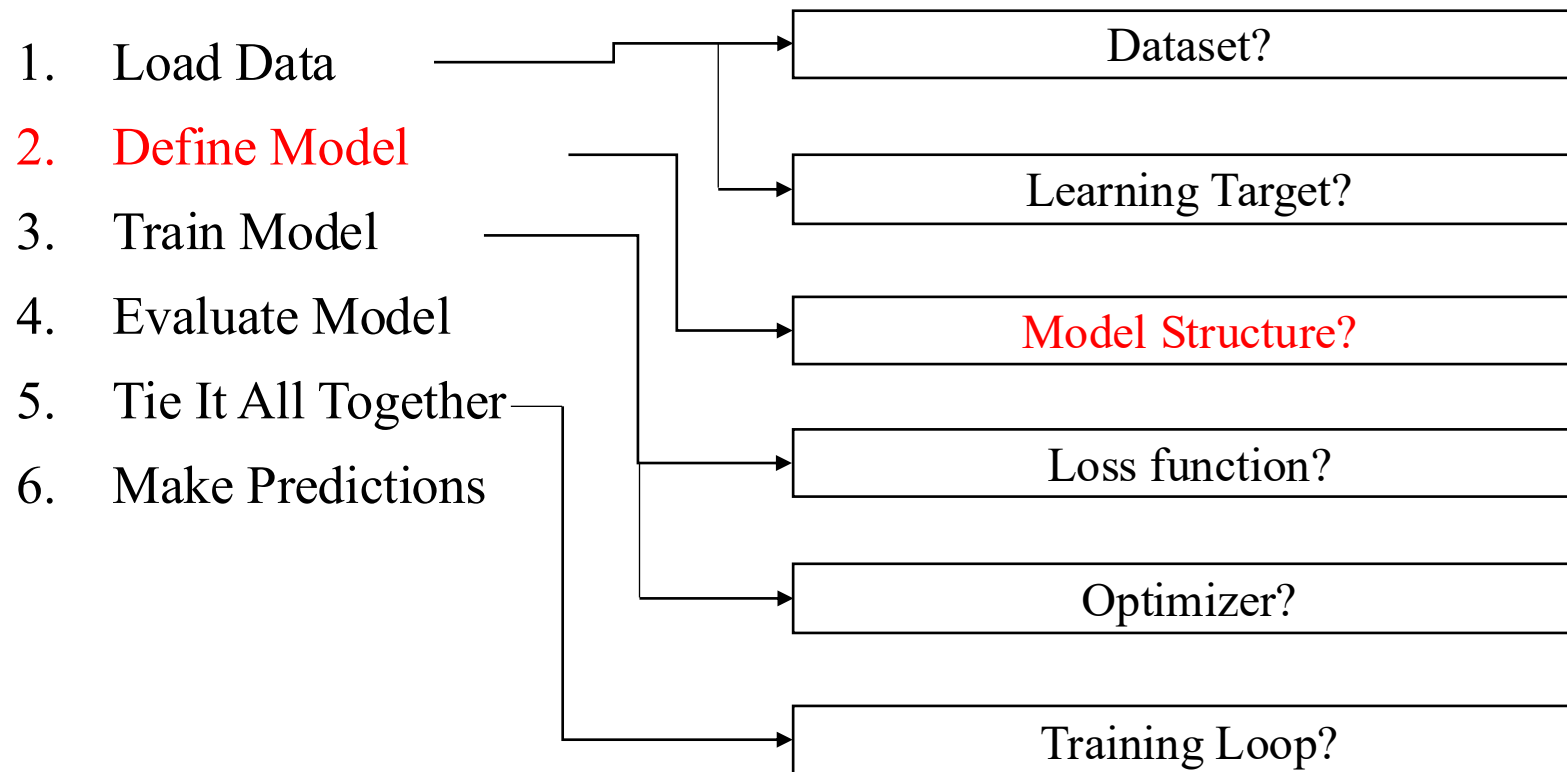
1. Rather than identifying the actual number, we want to classify whether the digit image contains an odd number. Modify the data generator to produce correct data samples from the MNIST dataset.

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Exercise

1. Rather than identifying the actual number, we want to classify whether the digit image contains an odd number. Modify the data generator to produce correct data samples from the MNIST dataset.
2. We want to make the input feature normalized to $[-1, 1]$. What shall we do?

Key Components of Pytorch Pipeline



Define Pytorch Model

A model is the core of deep learning, is composed of many building blocks and defines the computation logics from input to output.

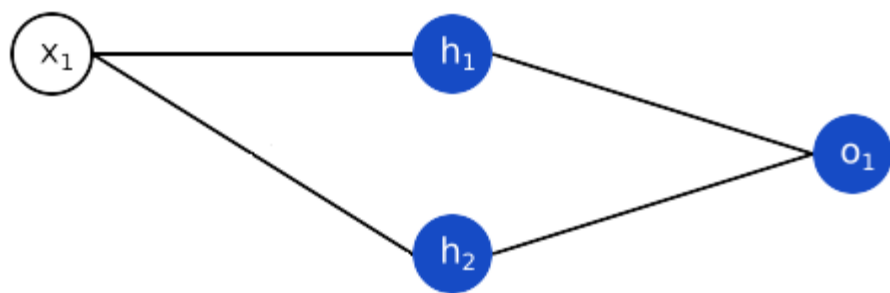
Two different ways of defining a deep neural network:

- **Sequential API**
- Functional API



Define Pytorch Model

Input Layer Hidden Layer Output Layer



Sequential API

```
import torch.nn as nn

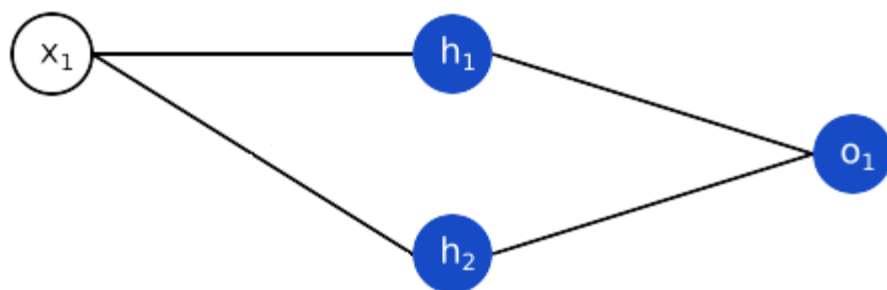
# Define a simple sequential model
model = nn.Sequential(
    nn.Linear(in_features=784, out_features=128),
    # An input layer with 784 inputs and 128 outputs
    nn.ReLU(),
    # A ReLU activation layer
    nn.Linear(in_features=128, out_features=10),
    # A hidden layer with 128 inputs and 10 outputs
    nn.Softmax(dim=1)
    # A Softmax activation layer
)

# Print the model architecture
print(model)
```

```
Sequential(
  (0): Linear(in_features=784, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=10, bias=True)
  (3): Softmax(dim=1)
)
```

Define Pytorch Model

Input Layer Hidden Layer Output Layer



You can omit the key words
in_features, out_features

Sequential API

```
import torch.nn as nn

# Define a simple sequential model
model = nn.Sequential(
    nn.Linear(784, 128), # An input layer with
                        784 inputs and 128 outputs
    nn.ReLU(),          # A ReLU activation layer
    nn.Linear(128, 10), # A hidden layer with
                        128 inputs and 10 outputs
    nn.Softmax(dim=1)   # A Softmax activation layer
)

# Print the model architecture
print(model)
```

Define Pytorch Model

Sequential API

```
import torch.nn as nn

# Define a simple sequential model
model = nn.Sequential(
    nn.Linear(in_features=784, out_features=128), # An input layer with 784 inputs and 128 outputs
    nn.ReLU(), # A ReLU activation layer
    nn.Dropout(),
    nn.Linear(in_features=128, out_features=10), # A hidden layer with 128 inputs and 10 outputs
    nn.Softmax() # A Softmax activation layer
)

# Print the model architecture
print(model)
```

```
Sequential(
  (0): Linear(in_features=784, out_features=128, bias=True)
  (1): ReLU()
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=128, out_features=10, bias=True)
  (4): Softmax(dim=None)
)
```

We can add also dropout layer to prevent overfitting

Define Pytorch Model

Sequential API

```
import torch
# Example input: a batch of 64 flattened MNIST images
input_data = torch.randn(64, 784)

# Forward pass through the model
output = model(input_data)

# Print the output size
print(output.size()) # Should print torch.Size([64, 10]), indicating 64 samples and 10 output classes
```

Now let us checkout the forward pass through this model!

What is “64”?

64 is the batch size

Define Pytorch Model

A model is the core of deep learning, is composed of many building blocks and defines the computation logics from input to output.

Two different ways of defining a deep neural network:

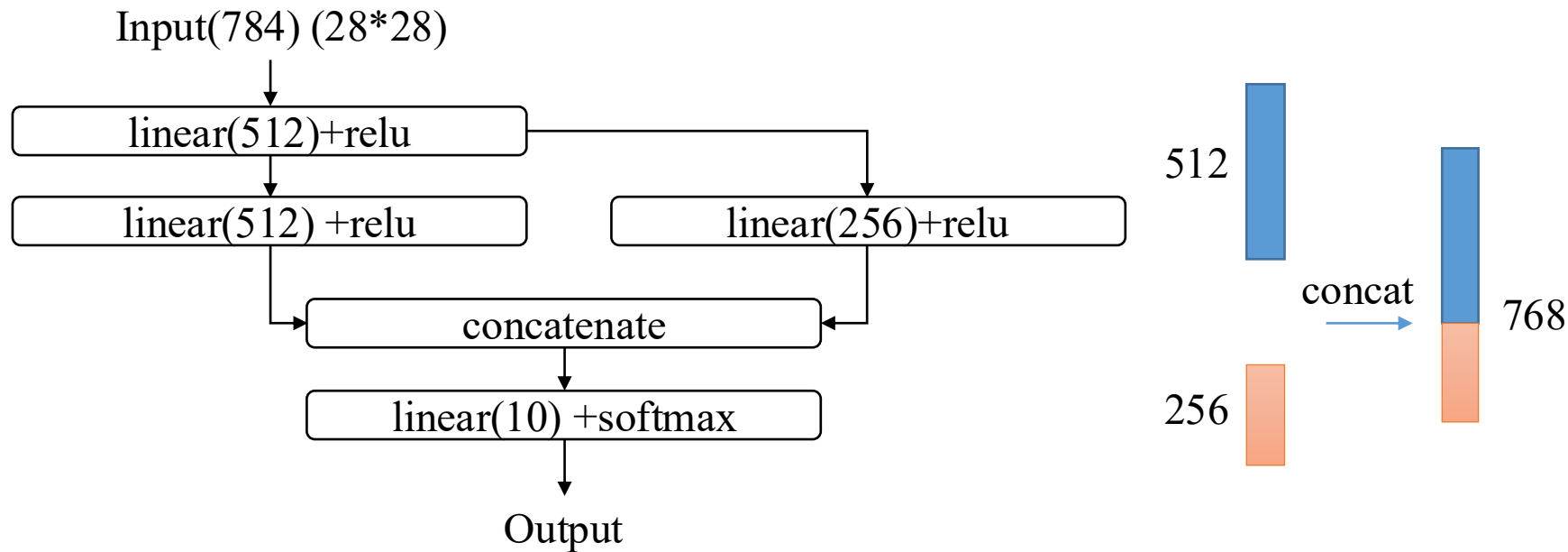
- Sequential API
- **Functional API**



Define Pytorch Model

Functional API

- In many cases, the model structure is more complicated than forward connections.

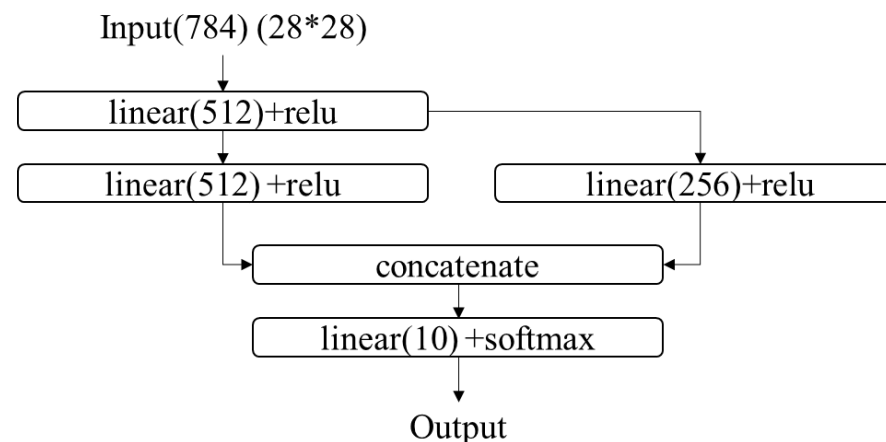


Define Pytorch Model

Functional API

Functional API makes it possible to
“package” a model as class

Again, similar to data generator, we should define
two important functions: **init** and **forward**



`__init__`

Define building blocks of the model
and other preset information (e.g.,
hyper parameters)

`forward`

Define the forward process of the model using the building blocks

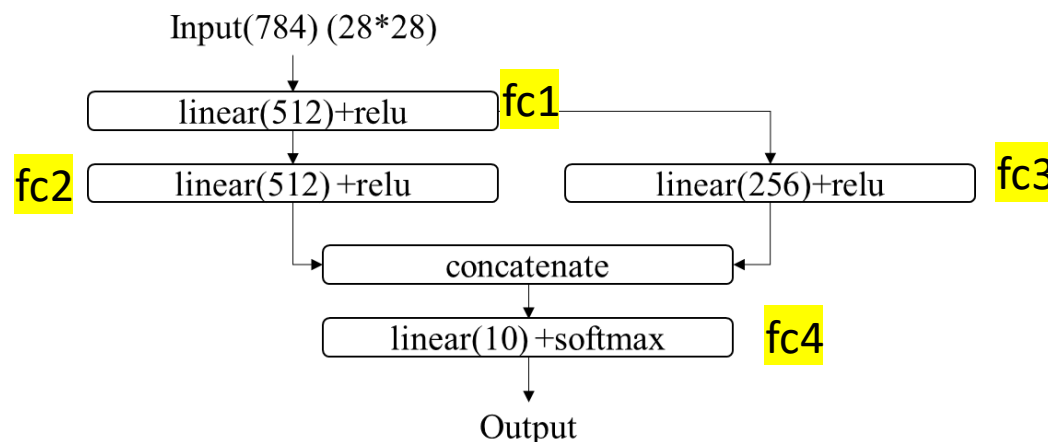
Define Pytorch Model

Functional API

`__init__`: Define building blocks of the model

```
import torch
from torch import nn
```

```
class MNISTModel(nn.Module):
    def __init__(self, in_shape, out_shape):
        super(MNISTModel, self).__init__()
        self.fc1 = nn.Linear(in_shape, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, 256)
        self.fc4 = nn.Linear(768, out_shape)  # 768 is the
        concatenated size of fc2 and fc3 outputs
```



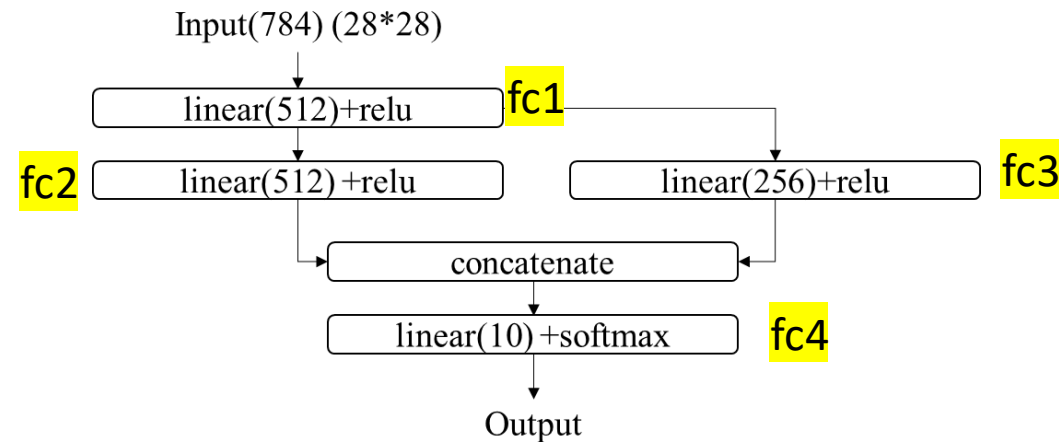
A default line, using the class name to declare the parent class `nn.Module`, you may ignore this.

Define Pytorch Model

Functional API

forward: Define the forward process of the model using the building blocks

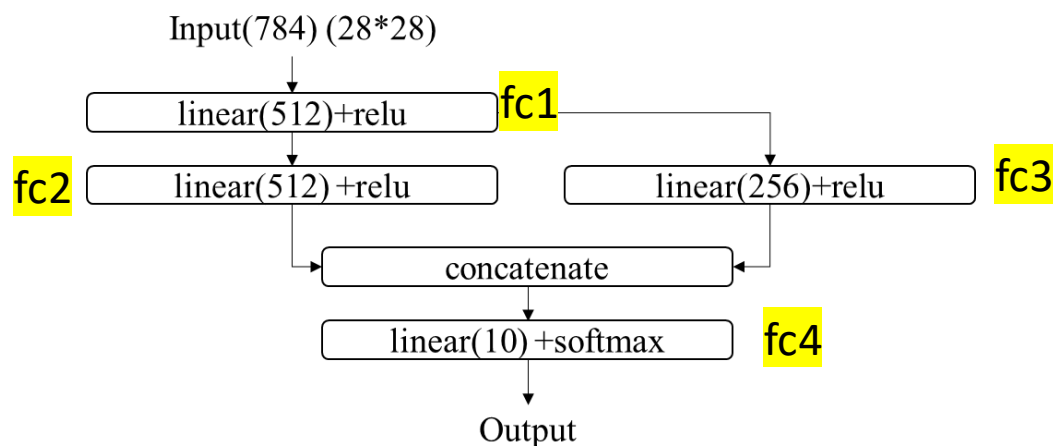
```
def forward(self, x):  
    x = torch.relu(self.fc1(x))  
    x1 = torch.relu(self.fc2(x))  
    x2 = torch.relu(self.fc3(x))  
    x3 = torch.cat((x1, x2), dim=1)  
    x = torch.softmax(self.fc4(x3), dim=1)  
    return x
```



In the forward function, we define how to get output according to the input, i.e., the model.

Define Pytorch Model

Functional API



Quick summary: init, forward

```

import torch
from torch import nn

class MNISTModel(nn.Module):
    def __init__(self, in_shape, out_shape):
        super(MNISTModel, self).__init__()
        self.fc1 = nn.Linear(in_shape, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, 256)
        self.fc4 = nn.Linear(768, out_shape)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x1 = torch.relu(self.fc2(x))
        x2 = torch.relu(self.fc3(x))
        x3 = torch.cat((x1, x2), dim=1)
        x = torch.softmax(self.fc4(x3), dim=1)
        return x
  
```

Define Pytorch Model

Functional API

With the model class, we specify the input and output shape, and finally define the model

```
model = MNISTModel(784, 10)
```

```
import torch
from torch import nn

class MNISTModel(nn.Module):
    def __init__(self, in_shape, out_shape):
        super(MNISTModel, self).__init__()
        self.fc1 = nn.Linear(in_shape, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, 256)
        self.fc4 = nn.Linear(768, out_shape)  # 768 is the concatenated size of fc2 and fc3 outputs
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x1 = torch.relu(self.fc2(x))
        x2 = torch.relu(self.fc3(x))
        x3 = torch.cat((x1, x2), dim=1)
        x = torch.softmax(self.fc4(x3), dim=1)
        return x

# Assuming input shape is 784 (28x28 pixels flattened) and output shape is 10 (digits 0-9)
model = MNISTModel(784, 10)
print(model)

MNISTModel(
  (fc1): Linear(in_features=784, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=512, bias=True)
  (fc3): Linear(in_features=512, out_features=256, bias=True)
  (fc4): Linear(in_features=768, out_features=10, bias=True)
)
```

Define Pytorch Model

Functional API

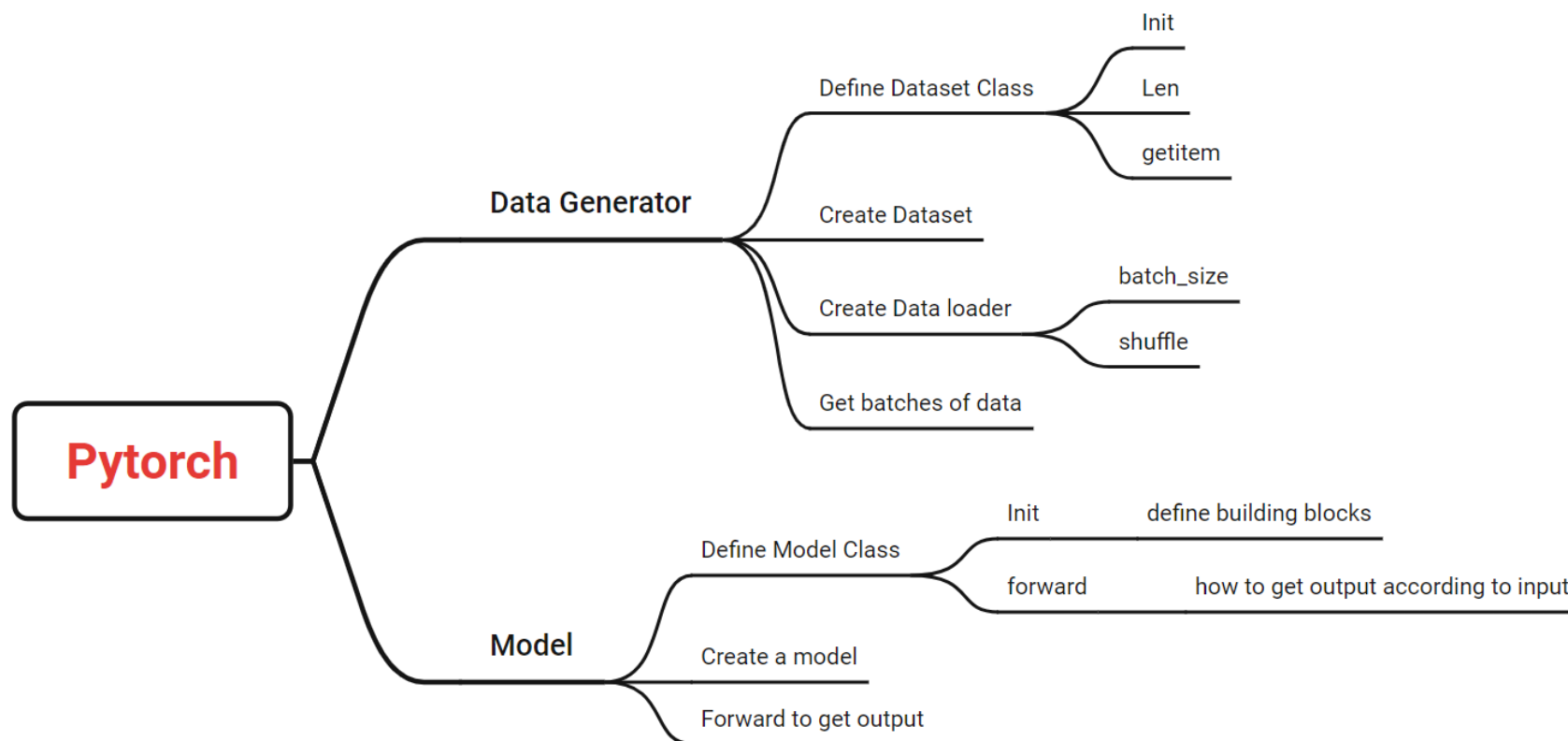
```
import torch
# Example input: a batch of 64 flattened MNIST images
input_data = torch.randn(64, 784)

# Forward pass through the model
output = model(input_data)

# Print the output size
print(output.size()) # Should print torch.Size([64, 10]), indicating 64 samples and 10 output classes
```

Again, we can check out the output size when using the actual data.

Summary

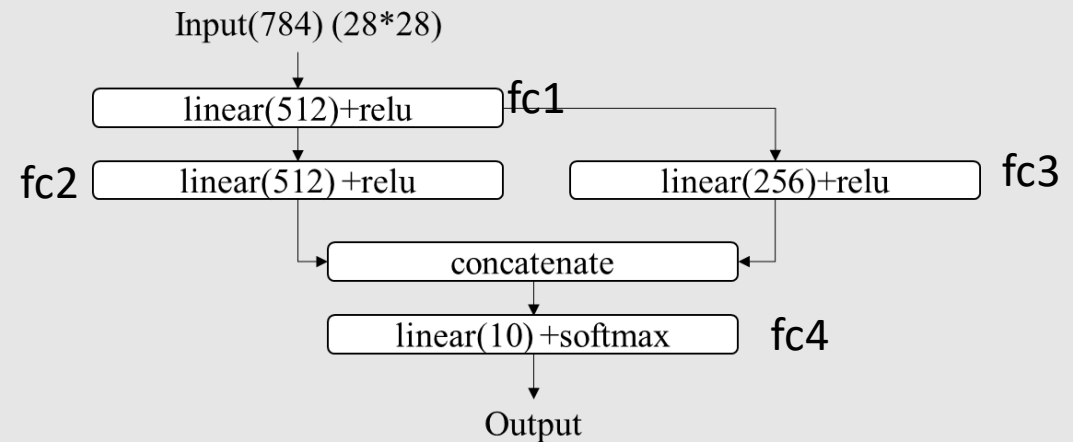


Define Pytorch Model

Functional API

forward: Define the forward process of the model using the building blocks

```
def forward(self, x):  
    x = torch.relu(self.fc1(x))  
    x1 = torch.relu(self.fc2(x))  
    x2 = torch.relu(self.fc3(x))  
    x3 = torch.cat((x1, x2), dim=1)  
    x = torch.softmax(self.fc4(x3), dim=1)  
    return x
```



In the forward function, we define how to get output according to the input, i.e., the model.

Now we will learn using different layers in the Pytorch!

Define DNN Model

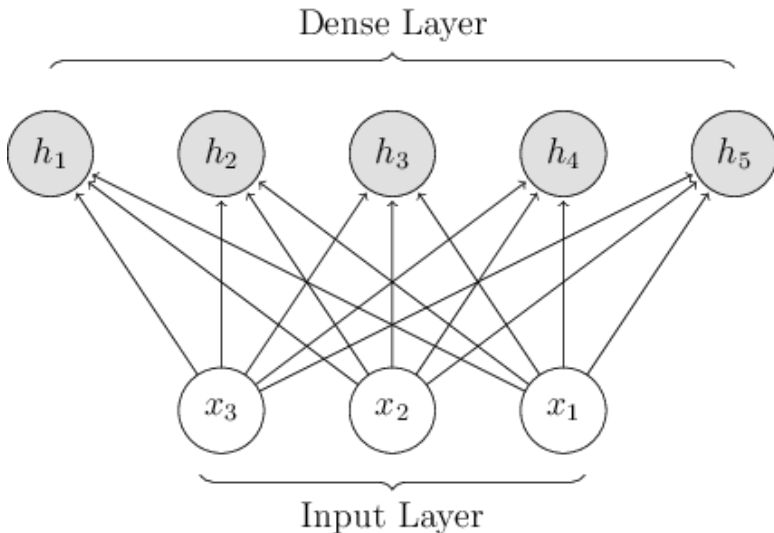
Network Layers

A DNN is built from many different types of layers, which mainly include

- Linear layers (Dense)
- RNN layers
- CNN layers

Define DNN Model

Linear layers (Dense)



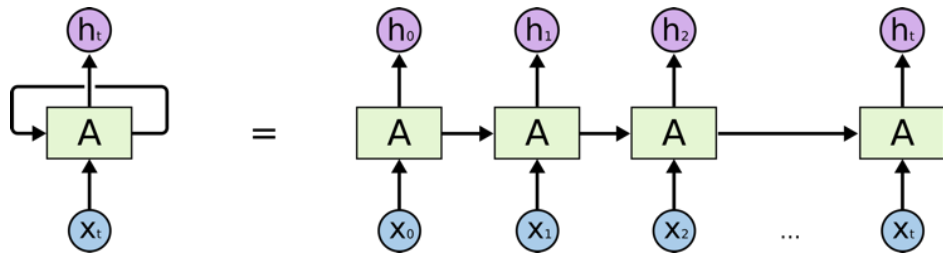
```
torch.nn.Linear(in_features, out_features)
```

```
m = nn.Linear(20, 30)  
input = torch.randn(128, 20)  
output = m(input)  
print(output.size())
```

```
→ torch.Size([128, 30])
```

Define DNN Model

RNN layers



- RNN layers takes the sequence as input and output a new sequence.
- Mainly used to model sequential data (speech, stock price, etc)
- The most used RNN is LSTM (Long Short-Term Memory).

```
torch.nn.LSTM(input_size, hidden_size, num_layers=1,
batch_first=False)
```

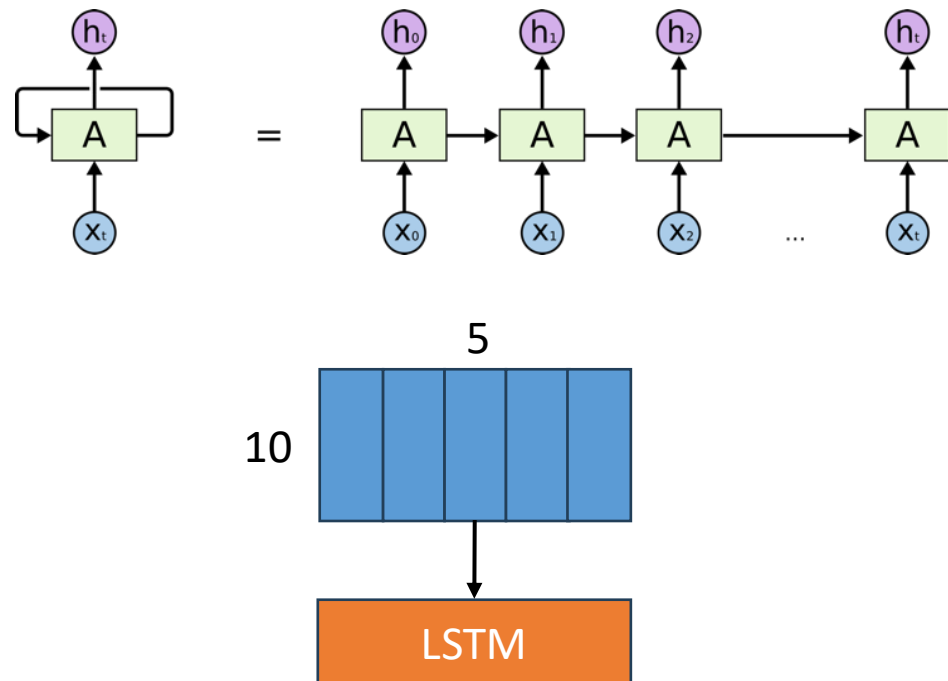
Input: a tensor with dim $(L, B, input_size)$, L is the sequence length, B is the batch size, and $input_size$ is the feature dimension. If *batch_first=True*, the dim is $(B, L, input_size)$

Output

- a tensor with dim $(L, B, hidden_size)$ for *batch_first=False*, and $(B, L, hidden_size)$ for *batch_first=True*.
- (h_0, c_0) , which are hidden states and cell states.

Define DNN Model

RNN layers



```
torch.nn.LSTM(input_size, hidden_size, num_layers=1)
```

```
>>> lstm = nn.LSTM(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> output, _ = lstm(input)
```

An LSTM is defined with input dim as 10, hidden dim as 20, 2 hidden LSTM layers.

The input sample's size is 5*10, with 5 as the sequence length and 10 as the input dimension. The batch size is 3.

We only get the output tensor and drop the hidden and cell states.

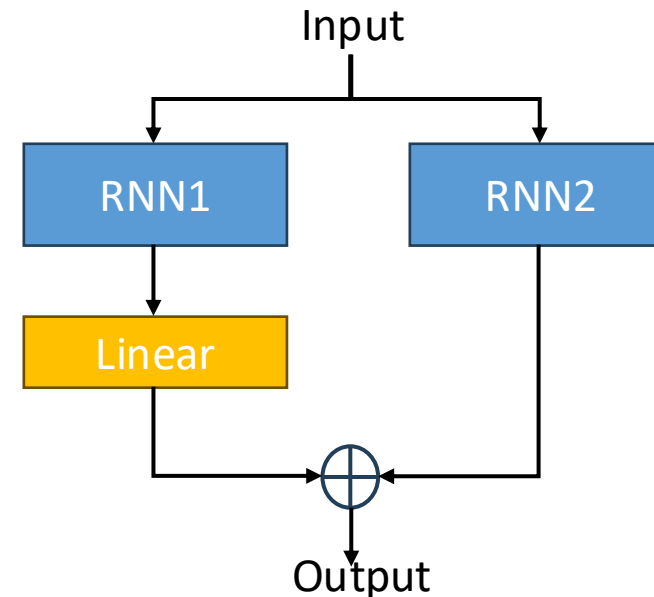
Guess what is the output dimension?

Run the code and check the results.

Exercise

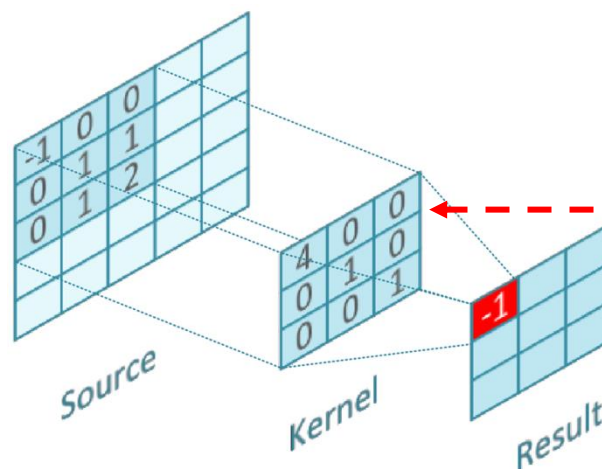
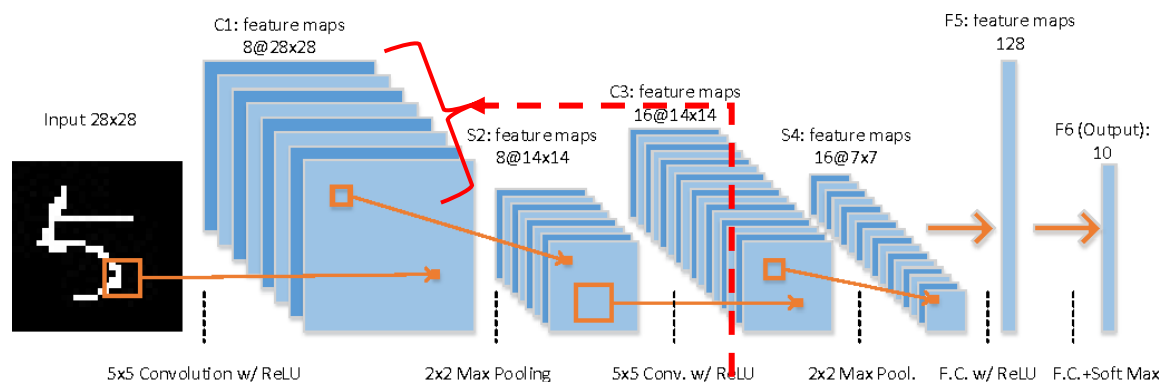
Write the forward function for the network, the batch size is 32:

1. Considering we have an input sequence with size (1000,20), design a 3 layer LSTM RNN1 to make the output feature dimension to 60.
2. Likewise, define a 2 layer LSTM RNN2 to make the output feature dimension to 30.
3. Convert the output dimension of RNN1 to 30 using linear layers, and add it with the RNN2 output to yield the final output.



Define DNN Model

CNN layers



Strided convolution

$$\begin{bmatrix} 2 & 3 & 4 & 7 & 4 & 6 & 2 & 9 \\ 6 & 4 & 0 & 9 & 1 & 8 & 7 & 4 & 3 \\ 3 & 1 & 4 & 0 & 8 & 3 & 3 & 8 & 9 & 7 \\ 7 & 8 & 3 & 6 & 6 & 3 & 3 & 4 & & \\ 4 & 2 & 1 & 8 & 3 & 4 & 6 & & & \\ 3 & 2 & 4 & 1 & 9 & 8 & 3 & & & \\ 0 & 1 & 3 & 9 & 2 & 1 & 4 & & & \end{bmatrix} \times \begin{bmatrix} 3 & 4 & 4 \\ 1 & 0 & 2 \\ -1 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 91 & & \\ & & \\ & & \end{bmatrix}$$

Handwritten notes: 7×7 (under the first matrix), 3×3 (under the second matrix), and $\text{Stride} = 2$ (below the second matrix).

Andrew Ng

- Filters (how many do we generate)
- filter size (how large do we compute)
- Strides (how far do we move)

Define DNN Model

CNN layers

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`

- Parameters: number of kernels in input (input channel), number of kernels in output (output channel), kernel size, stride size, padding option, normally can be chosen as “same”.
- Input: $(B, C_{in}, H_{in}, W_{in})$ with B as the batch size, C_{in} as input channel, H_{in}, W_{in} as input size.
- Output: $(B, C_{out}, H_{out}, W_{out})$ defined similarly to the input

Define DNN Model

CNN layers

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`

```
import torch
import torch.nn as nn
import numpy as np
```

```
# Convert NumPy array to PyTorch tensor
x = torch.randn(16, 1, 28, 28)
```

```
# Define the first convolutional layer
n_filter = 16
filter_size = 2
```

```
cnn_layer1 = nn.Conv2d(in_channels=1, out_channels=n_filter, kernel_size=filter_size,
padding='same')
```

```
# Apply the first convolutional layer to the input
y1 = cnn_layer1(x)
```

Define DNN Model

CNN layers

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`

```
# Define the second convolutional layer
```

```
n_filter = 32
```

```
filter_size = 4
```

```
cnn_layer2 = nn.Conv2d(in_channels=1, out_channels=n_filter,  
kernel_size=filter_size, padding='same')
```

```
# Apply the second convolutional layer to the input
```

```
y2 = cnn_layer2(x)
```

```
# Print the shapes of the tensors
```

```
for ii in [x, y1, y2]:
```

```
    print(ii.size())
```

```
torch.Size([16, 1, 28, 28])
```

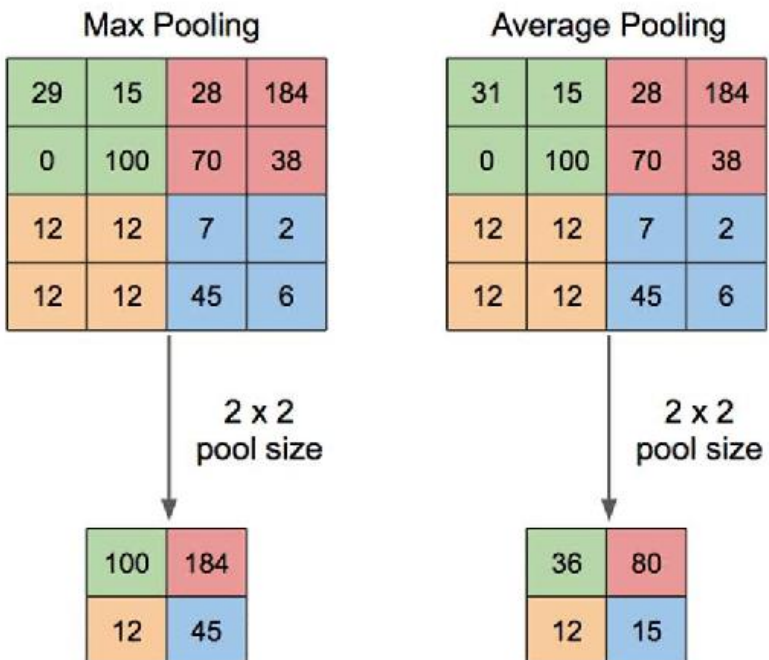
```
torch.Size([16, 16, 28, 28])
```

```
torch.Size([16, 32, 28, 28])
```

Define DNN Model

Pooling layers

`torch.nn.MaxPool2d(kernel_size)`



```
import torch
import torch.nn as nn
import numpy as np

# Convert NumPy array to PyTorch tensor
x = torch.randn(16, 1, 28, 28)

# Define the first convolutional layer
n_filter = 16
filter_size = 2
cnn_layer1 = nn.Conv2d(in_channels=1, out_channels=n_filter,
                        kernel_size=filter_size, padding='same')
# Apply the first convolutional layer to the input
y1 = cnn_layer1(x)

# Define the max pooling layer
pooling_layer = nn.MaxPool2d(kernel_size=2)

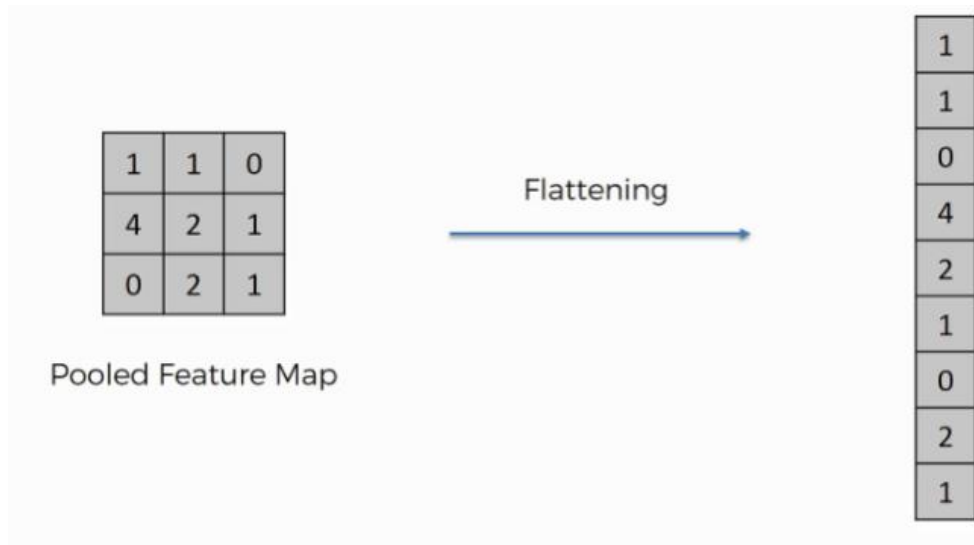
# Apply the max pooling layer to the output of the convolutional
layer
y2 = pooling_layer(y1)

# Print the shapes of the tensors
for ii in [x, y1, y2]:
    print(ii.size())
```

Define DNN Model

Other Common Operations

Reshape



```
# Flatten the tensor using reshape  
y = x.reshape(x.size(0), -1)
```

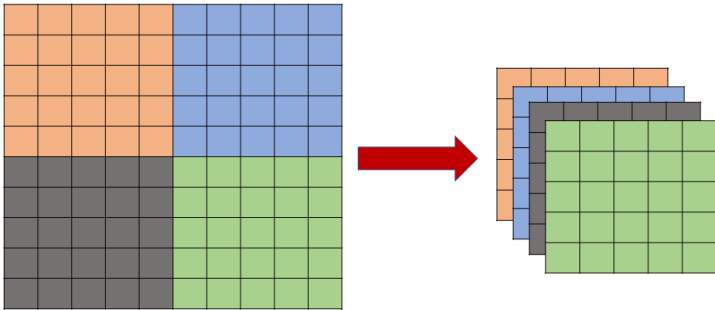
x.size(0) is the batch size

-1 means letting the machine to determine the dimension

Define DNN Model

Other Common Operations

Reshape



```
import torch
```

```
x = torch.randn(16, 1, 28, 28, dtype=torch.float32)
```

```
# First reshape operation
```

```
y = x.reshape(x.size(0), -1, 14, 14)
```

```
print(y.size())
```

```
# Second reshape operation
```

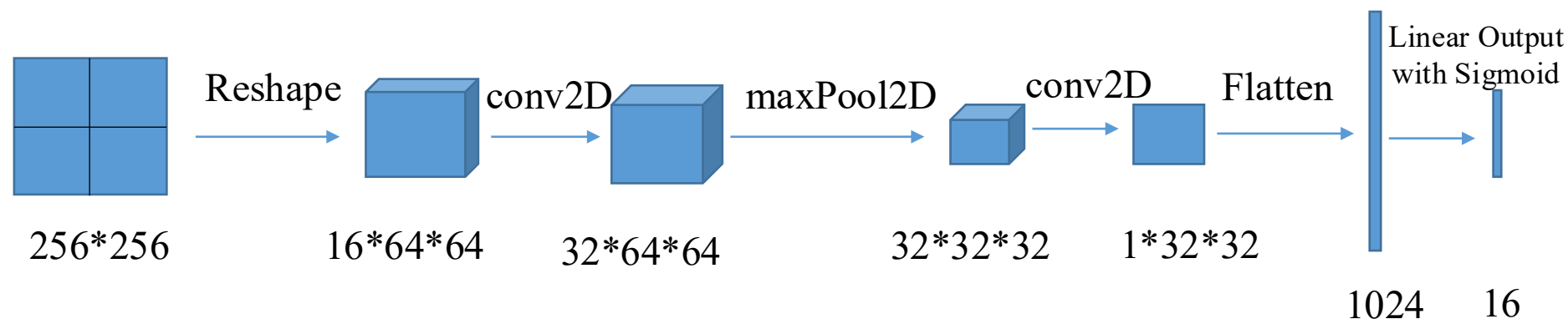
```
y = x.reshape(-1, 1)
```

```
print(y.size())
```

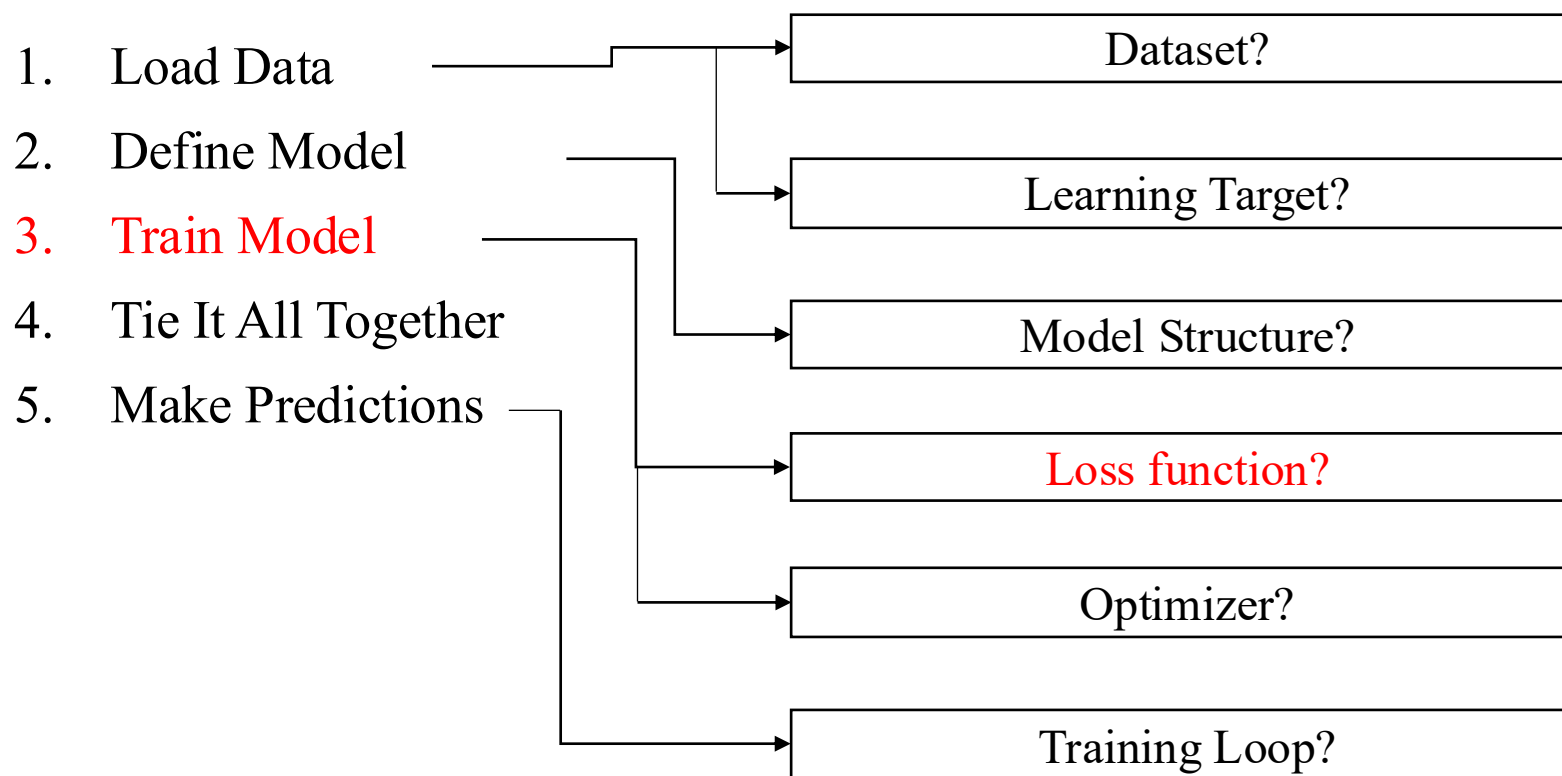
```
⇒ torch.Size([16, 4, 14, 14])  
   torch.Size([12544, 1])
```

Exercise

Define the following model using functional API. Kernel sizes are all 5×5 .



Key Components of Pytorch Pipeline



Loss Functions

Regression

Mean Squared Error (MSE)

- Calculates the average squared difference between predicted values and actual target values.
- Similar to SSR in traditional regression methods.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

```
mse_loss = torch.nn.MSELoss()
```

Loss Functions

Regression

Mean Squared Error (MSE)

```
import torch
input = torch.randn(5, 2)
target = torch.randn(5, 2)
# Create an instance of the MSELoss class
mse_loss = torch.nn.MSELoss()
# Compute the loss
loss = mse_loss(input, target)
# Print out the loss
print(loss.item())
```

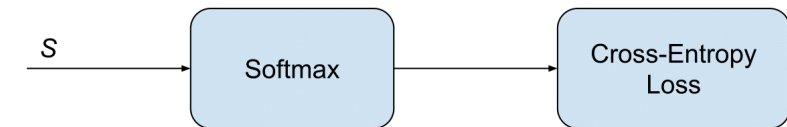
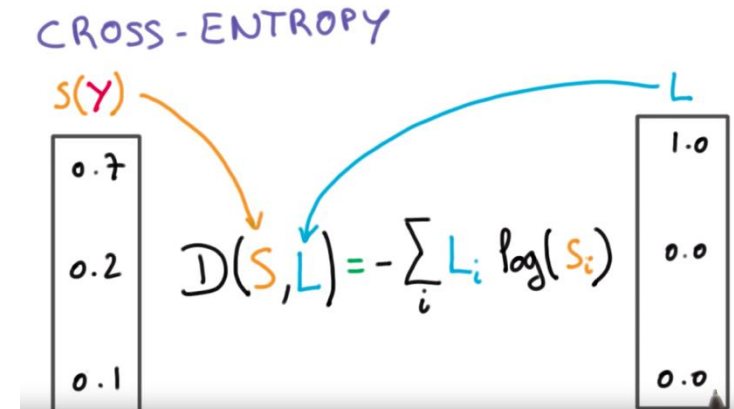
Loss Functions

Multi-Class Classification

Cross-Entropy (CE)

- A measure of the “surprise” from incorrect predictions, weighted by how often they occur.

$$E_p[\text{CE}_{\text{multi-class}}] = - \sum_{c=1}^M p(c) \log(\hat{p}(c))$$



```
cross_entropy_loss = nn.CrossEntropyLoss()
```

```
loss = cross_entropy_loss(predictions, target)
```

The “predictions” are passed through a softmax activation internally in `nn.CrossEntropyLoss`
Target should contain the indices of the correct class, not one-hot encoded

Loss Functions

Multi-Class Classification

Cross-Entropy (CE)

```
import torch
import torch.nn as nn
# size of predictions: (batch_size, num_classes), e.g., (5, 10)
predictions = torch.randn(5, 10)
# target should contain the indices of the correct class, not one-hot encoded
# size of target: (batch_size), e.g., (5). Generate 5 random integers in [0,10)
target = torch.randint(10, (5,))
# Create an instance of the CrossEntropyLoss class
cross_entropy_loss = nn.CrossEntropyLoss()
# Compute the loss
loss = cross_entropy_loss(predictions, target)
# Print out the loss
print(loss.item())
```

Loss Functions

Multi-Label Classification

Binary Cross-Entropy (BCE)

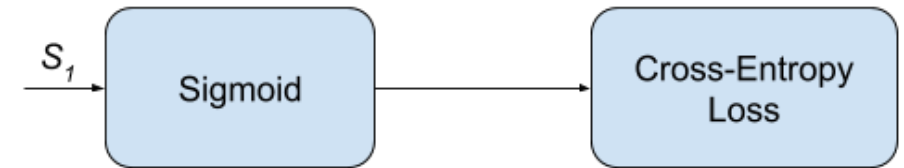
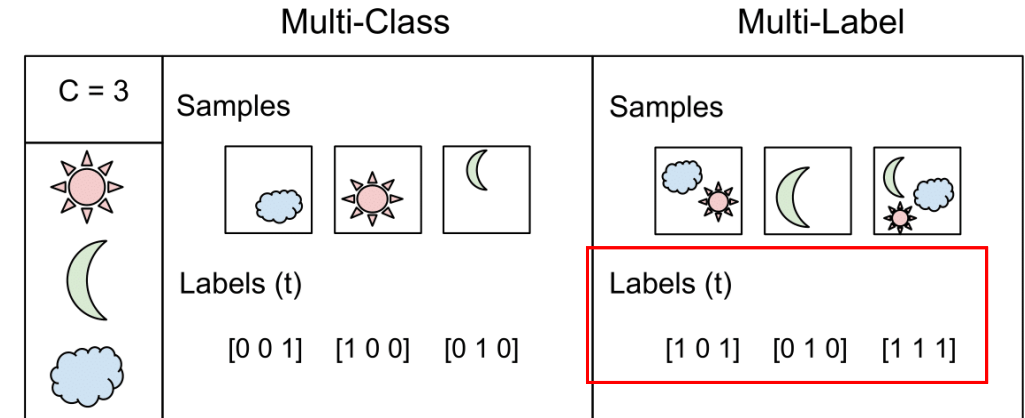
- Use multi-hot vector to indicate the existence of multiple objects.
- Use Sigmoid activation rather than Softmax activation to normalize the probability for each class to $[0,1]$

$$CE_{\text{binary}}(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

```
bce_loss = nn.BCELoss()
```

```
loss = bce_loss(probabilities, target)
```

A separate sigmoid is needed to convert prediction to probabilities.



Loss Functions

Multi-Class Classification

Binary Cross-Entropy (BCE)

```
import torch
import torch.nn as nn
# Assuming we have some predictions from the model and some target labels
# The predictions are logits, and we will apply sigmoid to convert them to probabilities
# For binary classification, the output size is (batch_size,) or (batch_size, 1), e.g., (5,)
predictions = torch.randn(5, requires_grad=True)
# Apply sigmoid to convert to probabilities
probabilities = torch.sigmoid(predictions)
```

Loss Functions

Multi-Class Classification

Binary Cross-Entropy (BCE)

Targets should be the same shape as the output of the sigmoid function and contain binary values

Here, we randomly generate binary targets, 5 instances being either 0 or 1.

```
target = torch.randint(0, 2, (5,)).float()
```

Create an instance of the BCELoss class

```
bce_loss = nn.BCELoss()
```

Compute the loss

```
loss = bce_loss(probabilities, target)
```

Print out the loss

```
print(loss.item())
```

Loss Functions

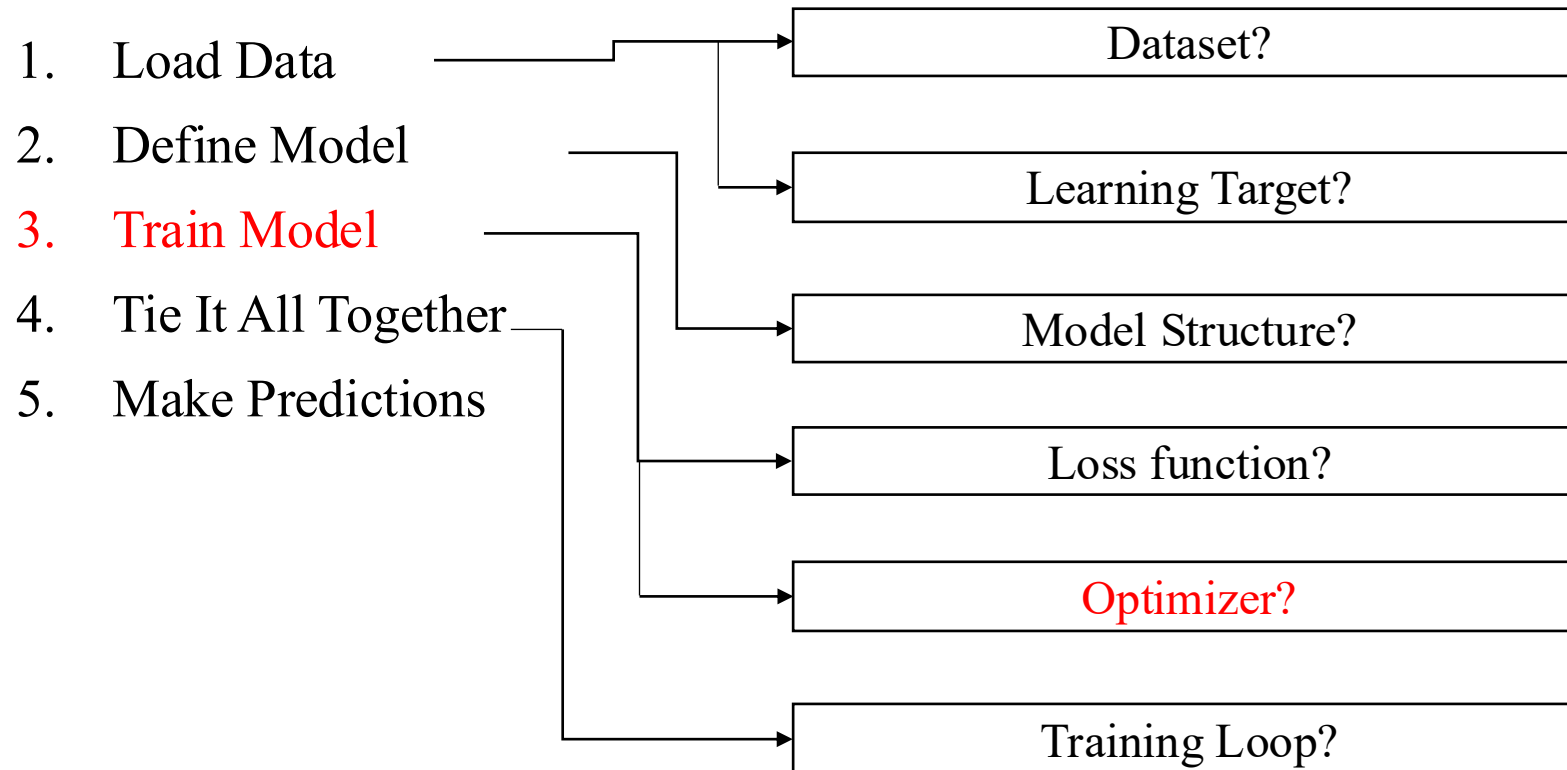
Custom Loss Functions

We can also define the loss as a class with forward function calculating the error between prediction and target.

For instance, to define L1 loss, we can have

```
# Custom L1 Loss Module
class CustomL1Loss(nn.Module):
    def __init__(self):
        super(CustomL1Loss, self).__init__()
    def forward(self, input, target):
        # Compute the absolute difference between input and target
        loss = torch.abs(input - target)
        # Take the mean over all the elements
        return loss.mean()
```

Key Components of Pytorch Pipeline



Optimizer

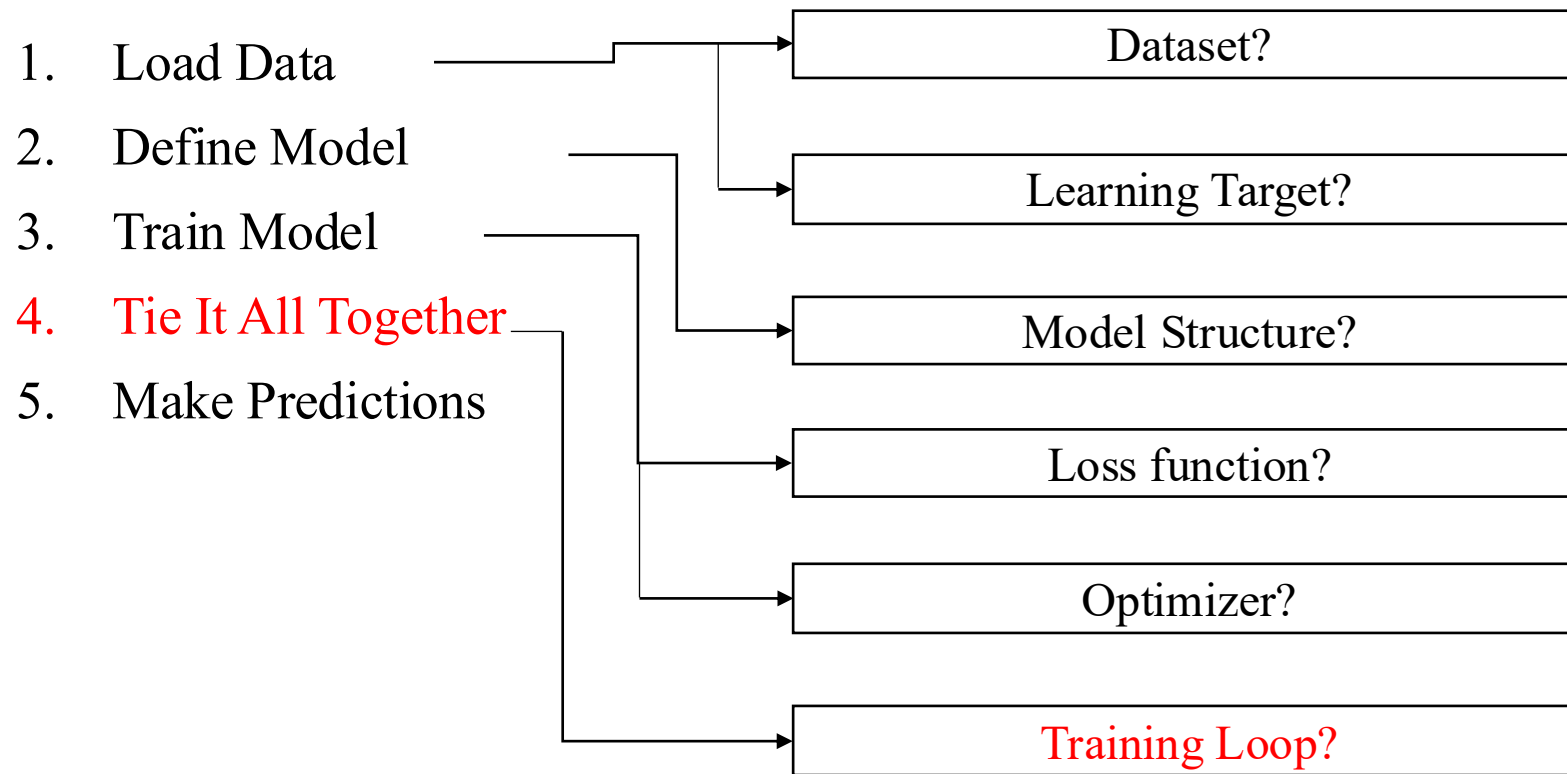
The optimizer specifies how to update the DNN weights according to the loss function values.

- SGD
- RMSprop
- Adam
- Adadelata
- Adagrad
- After we have defined the model, we can have

```
import torch.optim as optim
# Using the default learning rate
optimizer = optim.Adam(model.parameters())
```

- We can also specify the learning rate
Using a specified learning rate
optimizer = optim.Adam(model.parameters(), lr=1e-3)

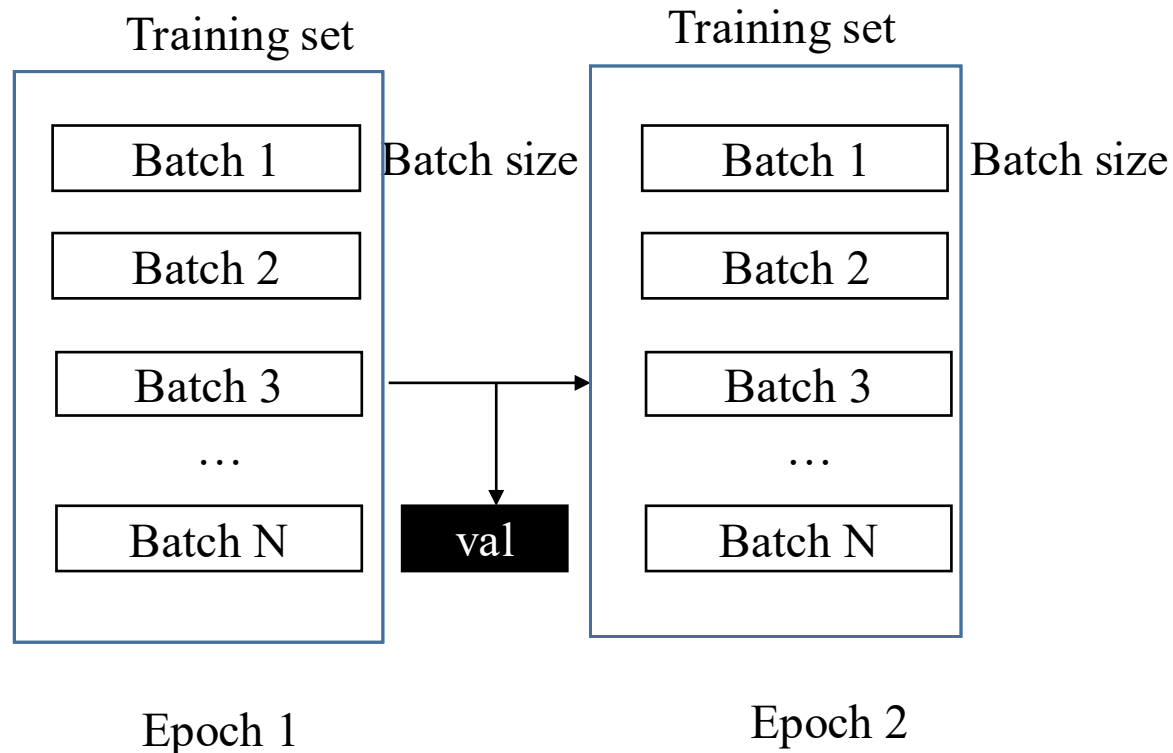
Key Components of Pytorch Pipeline



Training Loop

The training loop

- Takes batches of training samples to update the model
- Evaluate the model on the validation data to determine whether to save the model



Training Loop

Training loop code design: forward, get loss, update weights

```
# Set model to training mode
model.train()
# Assuming 'epochs' is defined and 'train_loader' is a PyTorch DataLoader for the training data
for epoch in range(epochs):
    for i step, (x batch train, y batch train) in enumerate(train_loader):
        # Forward pass: Compute predicted y by passing x to the model
        pred = model(x batch train)
        # Compute loss
        loss_value = loss_fn(pred, y_batch_train)
        # Zero the gradients before running the backward pass
        optimizer.zero_grad()
        # Backward pass: Compute gradient of the loss with respect to model parameters
        loss_value.backward()
        # Calling the step function on an Optimizer makes an update to its parameters
        optimizer.step()
```

Forward

Get Loss

Update Weights

Training Loop

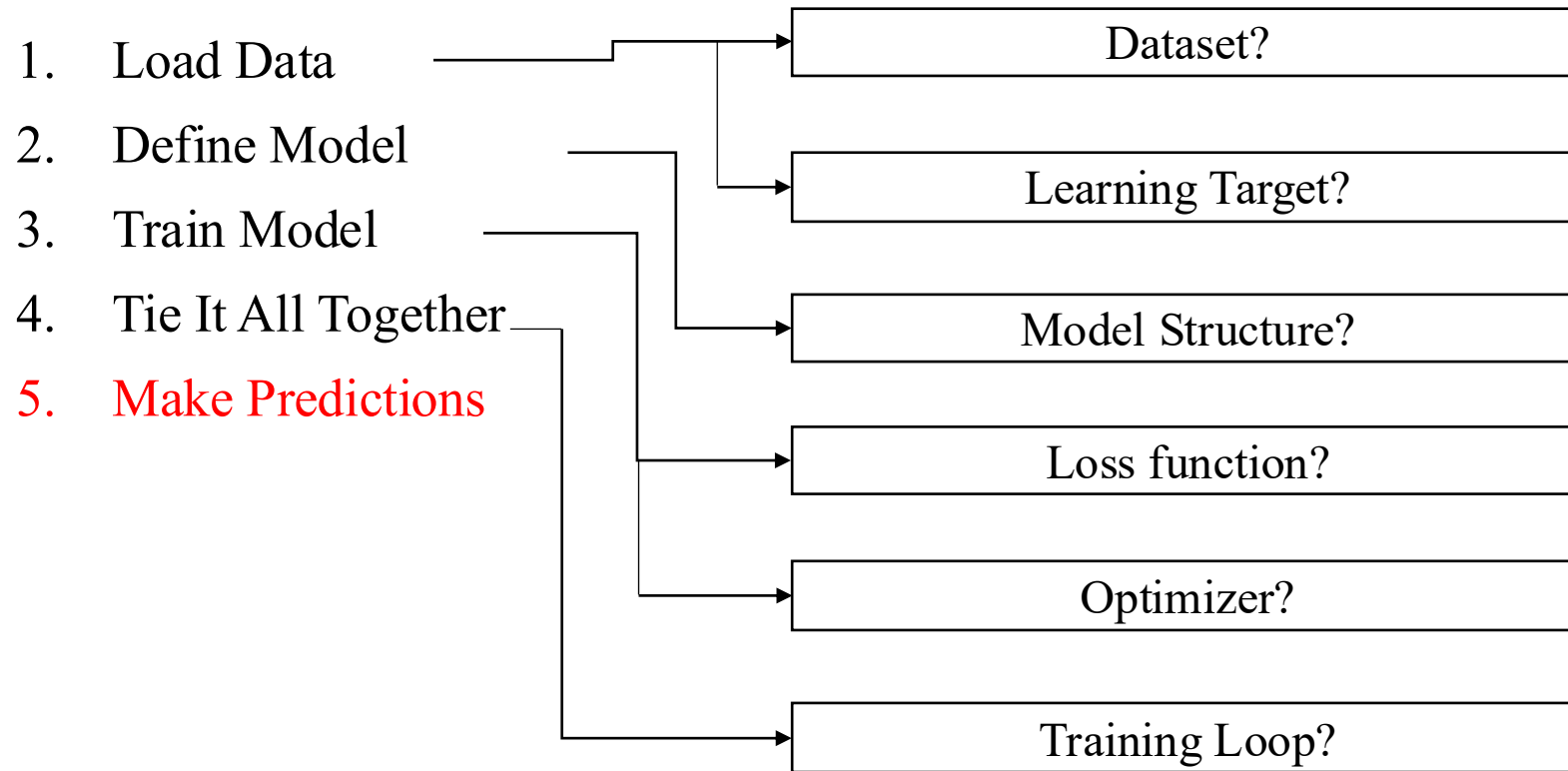
We can determine whether to save the model during training, by using the validation set

```
# Set model to training mode
model.train()
best_metric = float('inf')
# Assuming 'epochs' is defined and 'train_loader' is a PyTorch DataLoader for the training data
for epoch in range(epochs):
    for i_step, (x_batch_train, y_batch_train) in enumerate(train_loader):
        # Forward pass: Compute predicted y by passing x to the model
        pred = model(x_batch_train)
        # Compute loss
        loss_value = loss_fn(pred, y_batch_train)
        # Zero the gradients before running the backward pass
        optimizer.zero_grad()
        # Backward pass: Compute gradient of the loss with respect to model parameters
        loss_value.backward()
        # Calling the step function on an Optimizer makes an update to its parameters
        optimizer.step()

    with torch.no_grad(): # No need to track gradients during validation
        val_metric = 0.0
        for x_batch_val, y_batch_val in val_loader:
            val_pred = model(x_batch_val)
            # Calculate validation loss
            val_metric += loss_fn(val_pred, y_batch_val)

    if val_metric < best_metric:
        best_metric = val_metric
        # Save the model - the recommended way in PyTorch is to save the state_dict
        torch.save(model.state_dict(), 'path_to_location.pth')
```

Key Components of Pytorch Pipeline



Make Predictions

Once the model is trained, prediction can be achieved by conducting a forward propagation.

```
# Initialize the model defined in YourModel Class
model = YourModel()
# Load the state dict that was saved previously
model.load_state_dict(torch.load('path_to_location.pth'))
# Put the model in evaluation mode
model.eval()
with torch.no_grad(): # No gradients required for inference
    # Make a prediction
    prediction = model(x)
```

QA Time